МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ВОЛГОГРАДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

А.Е. Андреев, В.А. Егунов,
Д.Н. Жариков, В.А. Малолетков

РЕАЛИЗАЦИЯ ВЫЧИСЛИТЕЛЬНО-ИНТЕНСИВНЫХ АЛГОРИТМОВ НА ГИБРИДНЫХ СИСТЕМАХ С РЕКОНФИГУРИРУЕМЫМИ СОПРОЦЕССОРАМИ

Монография



Волгоград – 2013

УДК 004.27, 004.383

Р е ц е н з е н т ы :

кафедра «Информационные системы и технологии»
Волгоградского государственного аграрного университета (ВолГАУ),
зав. кафедрой - проректор по информатизации ВолГАУ,
д-р техн. наук, профессор *О.В. Кочеткова;*

зав. сектором нетрадиционных суперкомпьютерных архитектур
ОИВСиЛС ИПМ им. М.В. Келдыша РАН, д-р ф.-м. наук *А.О. Лацис;*

профессор кафедры «Информационные системы и технологии»
Новороссийского государственного морского университета
им. адмирала Ф.Ф.Ушакова д-р. техн. наук *Е.И. Духнич*

Печатается по решению редакционно-издательского совета
Волгоградского государственного технического университета

Андреев, А.Е.

Реализация вычислительно-интенсивных алгоритмов на гибридных системах с реконфигурируемыми сопроцессорами : монография / А.Е. Андреев, В.А. Егунов, Д.Н. Жариков, В.А. Малолетков; ВолгГТУ. – Волгоград, 2013. – 180 с.

ISBN

В монографии рассмотрены различные аспекты построения гибридных вычислительных систем с реконфигурируемыми сопроцессорами на базе программируемых логических интегральных схем (ПЛИС), структура таких систем, требования к алгоритмам для реализации на реконфигурируемых сопроцессорах, средства высокоуровневого проектирования для ПЛИС.

С использованием средств проектирования высокого уровня, а также языков описания аппаратуры реализован ряд алгоритмов и оценены характеристики полученных реализаций.

Ил. 63. Табл. 42. Библиогр.: 94 назв.

ISBN

© Волгоградский государственный технический университет, 2013
© А.Е. Андреев, В.А. Егунов, Д.Н. Жариков, В.А.Малолетков, 2013

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ……. 6
Глава 1. ПОНЯТИЕ О ГИБРИДНЫХ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМАХ С РЕКОНФИГУРИРУЕМЫМИ СОПРОЦЕССОРАМИ……. 8
1.1. Гибридные вычислительные системы…….. 8
1.2. Реконфигурируемые вычислительные системы…….. 10
1.3. Гибридные вычислительные системы начального уровня с реконфигурируемыми сопроцессорами (ГВС РС)……..…..... 14
1.4. Выводы……..……. 16
Глава 2. ХАРАКТЕРИСТИКА АЛГОРИТМОВ ДЛЯ РЕАЛИЗАЦИИ НА ГВС РС……..... 17
2.1. Общая характеристика алгоритмов, для которых возможна эффективная реализация на ГВС РС…… 18
2.2. Методики оценки целесообразности реализации алгоритмов на реконфигурируемых сопроцессорах……..…. 19
2.3. Прогнозирование производительности при реализации алгоритма QR-разложения матрицы……..... 26
2.4. Оценка производительности при реализации криптографического алгоритма AES…….. 31
2.5. Оценка производительности при реализации задачи восстановления строки по MD5 хэш-сумме……..... 34
2.6. Выводы…….. 40
Глава 3. СТРУКТУРА И ПОСТРОЕНИЕ ГИБРИДНОЙ СИСТЕМЫ С РЕКОНФИГУРИРУЕМЫМИ СОПРОЦЕССОРАМИ…… 41
3.1. Семейства ПЛИС для реконфигурируемых систем……... 41
3.1.1. Семейства ПЛИС Altera…….…. 41
3.1.2. Семейства ПЛИС Xilinx……...……..... 45
3.2. Реконфигурируемые сопроцессоры……..... 47
3.2.1. Анализ ресурсов FPGA Arria II GX EP2AGX125……...… 48
3.2.2. Комплект разработчика Terasic DE5-Net с поддержкой Altera OpenCL SDK……...……. 52
3.2.3. Интерфейс Avalon Memory Mapped……..…….. 53
3.3. Общая архитектура системы……..…..……. 55
3.4. Типовые структуры конфигурации сопроцессора для различных типов задач……..…..…… 56
3.4.1. Критерии для задач, решаемых с помощью реконфигурируемого сопроцессора……...……..…… 56
3.4.2. Реализация схемы взаимодействия для задач, осуществляющих многократные вычисления над небольшим объемом данных……...……..... 57
3.4.3. Реализация схемы взаимодействия для задач, обрабатывающих массивы данных……..……... 60
3.4.4. Реализация схемы взаимодействия для задач со сложной

управляющей логикой……...……. 63
3.5 Системное программное обеспечение сопроцессора……..… 66
3.6. Выводы……..……..... 67
Глава 4 СРЕДСТВА РАЗРАБОТКИ РЕШЕНИЙ ДЛЯ ГИБРИДНЫХ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ С РЕКОНФИГУРИРУЕМЫМИ СОПРОЦЕССОРАМИ…….……. 69
4.1 Состав средств разработки……..... 69
4.2 Подходы к разработке схемных решений и состав необходимых программных средств……... 70
4.3 Обзор и сравнительный анализ средств высокоуровневого проектирования для ПЛИС…….. 72
4.3.1. Обзор сравниваемых средств высокоуровневого проектирования……... 72
4.3.2. Методика сравнения высокоуровневых средств разработки……... 77
4.3.3. Результаты сравнения……..……..……..... 79
4.3.4. Выводы……..……..…… 82
Глава 5 ИССЛЕДОВАНИЕ РЕАЛИЗАЦИИ АЛГОРИТМОВ ДИСКРЕТНЫХ ЛИНЕЙНЫХ ПРЕОБРАЗОВАНИЙ НА РЕКОНФИГУРИРУЕМЫХ СОПРОЦЕССОРАХ……. 84
5.1 Алгоритмы дискретных линейных преобразований (ДЛП)..…..... 84
5.2 Многомерные алгоритмы ДЛП……...…..…… 90
5.3 Оценка характеристик реализации алгоритмов ДЛП на ПЛИС... 94
5.3.1. Реализация алгоритма CORDIC……..…… 96
5.3.2. Реализация алгоритма Householder-CORDIC…… 100
5.3.3. Реализация алгоритма ДЛП кронекеровского произведения вращений……..……. 111
5.3.4. Реализация ДЛП комплексного отражения……..……. 112
5.4 Выводы……..……... 117
Глава 6 РЕАЛИЗАЦИЯ КРИПТОЛОГИЧЕСКИХ АЛГОРИТМОВ НА ГВС-РС……..…… 119
6.1. Решение задачи поиска строки по MD5 хэш-сумме на FPGA сопроцессоре……..……... 119
6.1.1. Разработка общей структуры вычислительного блока….. 119
6.1.2. Реализация модуля хэширования по алгоритму MD5..…. 121
6.1.3. Реализация генератора строк……..….. 127
6.1.4. Реализация блока сравнения……..……... 128
6.1.5. Реализация управляющего устройства……..…….. 129
6.1.6. Результаты экспериментов……..…. 131
6.1.7. Практическая значимость разработки…….. 132
6.2. Реализация отдельных режимов криптоалгоритма ГОСТ……. 134
6.2.1. Реализация режима простой замены на FPGA……...…….. 135
6.2.2. Реализация режимов гаммирования на FPGA……...…….. 137
6.2.3. Результаты реализации алгоритма шифрования ГОСТ

28147-89 (с 32-битным ключом) в режиме простой замены на GPU и FPGA……. 138
6.2.3.1. Сравнение экспериментальной производительности с прогнозируемой…..…..… 141
6.2.4. Результаты реализации алгоритма шифрования ГОСТ 28147-89 (с 32-битным ключом) в режимах гаммирования на FPGA……. 142
6.2.4.1. Адаптация реализации на FPGA для интерфейсов Avalon…… 142
6.2.4.2. Сравнение экспериментальной производительности с прогнозируемой…… 145
6.3. Выводы……..…… 146
Глава 7 РЕАЛИЗАЦИЯ НА ПЛИС ПРОТОТИПА ТЕПЛОВОГО РАСЧЕТА ДЛЯ СИСТЕМЫ МЕЖДИСЦИПЛИНАРНОГО ИНЖЕНЕРНОГО АНАЛИЗА …… 147
7.1 Постановка задачи теплового расчета……...…….. 147
7.2 Реализация алгоритма расчета распределения тепла на ПЛИС……. 149
7.3 Реализация алгоритма с помощью средства Impulse C…..……... 154
7.4 Результаты работы теплового расчета для различных архитектур…….. 155
7.5. Выводы……...……... 158
ЗАКЛЮЧЕНИЕ……..……...…… 159
БИБЛИОГРАФИЧЕСКИЙ СПИСОК ……..……..... 160
ПРИЛОЖЕНИЕ А Исходный код приложения на C для реализации алгоритма CORDIC на LegUp…… 167
ПРИЛОЖЕНИЕ Б Исходный код ядра OpenCL для реализации алгоритма CORDIC на Altera OpenCL SDK……..……. 168
ПРИЛОЖЕНИЕ В Исходный код на C для реализации алгоритма Hausholder CORDIC в матричной форме для размерности 4 на LegUp …...……. 169
ПРИЛОЖЕНИЕ Г Исходный код ядра OpenCL для реализации алгоритма Hausholder CORDIC в матричной форме для размерности 3 на Altera OpenCL SDK…….…….. 171
ПРИЛОЖЕНИЕ Д Исходный код ядра OpenCL для реализации алгоритма Hausholder CORDIC в векторной форме для размерности 3 на Altera OpenCL SDK……..... 173
ПРИЛОЖЕНИЕ Е Исходный код ядра OpenCL для реализации алгоритма ДЛП кронекеровского произведения вращений на Altera OpenCL SDK 175
ПРИЛОЖЕНИЕ Ж Исходный код ядра OpenCL для реализации алгоритма комплексного ДЛП отражения на базе Hausholder CORDIC в векторной форме для размерности 3 на Altera OpenCL SDK 177

ВВЕДЕНИЕ

Развитие микропроцессоров в течение ряда последних лет направлено не столько на повышение тактовых частот (в этом направлении достигнут временный предел), сколько на увеличение количества вычислительных ядер и разрядности векторных регистров. Создание и массовое внедрение мультиядерных (от 2 ядер), а затем и многоядерных (с количеством ядер от 32 и выше) центральных процессоров (ЦПУ / CPU) резко повысило интерес к параллельным вычислениям и создало предпосылки для «параллельной революции» в области IT. Активизировались и производители традиционно многоядерных и хорошо приспособленных к обработке векторов и матриц графических процессоров (ГПУ / GPU), предоставив программистам удобные API для неграфических вычислений на графических ускорителях (GPGPU). В результате небольшой параллельный суперкомпьютер становится доступным настольным решением. В то же время разнообразные параллельные архитектуры по-прежнему не могут справляться с задачами принципиально последовательного характера, с итерационными задачами, время вычисления итераций в которых меньше временных затрат на создание и синхронизацию параллельных потоков. Другим вызовом для многоядерных архитектур и GPU является значительные потребление электроэнергии и выделение тепла, превращающие вычислительный комплекс (кластер) в клиента отдельной небольшой электростанции и специализированной системы охлаждения.

Одним из способов решения перечисленных проблем может являться применение еще одного вида ускорителей, построенного на базе микросхем программируемой логики (ПЛИС), работающих на частоте, меньшей на порядок, и потребляющих на порядок меньше энергии. Достигать ускорения вычислений они позволяют в основном за счет

специализации под конкретную задачу и возможности строить для специализированных решений эффективные конвейеры.

Применение таких ускорителей ограничивается рядом сложностей, среди которых можно выделить, прежде всего, доступность таких устройств, сложности сопряжения их с настольными системами и сложности создания специализированных схем для решения задач. Рассмотрению этих вопросов посвящена данная работа.

Глава 1

ПОНЯТИЕ О ГИБРИДНЫХ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМАХ С РЕКОНФИГУРИРУЕМЫМИ СОПРОЦЕССОРАМИ

1.1 Гибридные вычислительные системы

Под гибридными вычислительными системами еще относительно недавно понимали гибрид цифровых и аналоговых вычислительных подсистем в составе одной вычислительной системы или машины [1]. В настоящее время в связи с относительно небольшим распространением аналоговых вычислителей термин гибридная вычислительная система чаще приобретает смысл, синонимичный неоднородной (или, если использовать заимствование из английского - гетерогенной) системе. Гибридная вычислительная система состоит из разных по свойствам и даже по архитектуре вычислительных подсистем [2].

Необходимо отметить, что в контексте данной работы, прежде всего, нас интересуют гибридные вычислительные системы, ориентированные на высокую производительность, точнее, на повышение производительности за счет гибридизации в том числе. Характерной чертой высокопроизводительных вычислительных систем является активное использование различных форм параллельных вычислений, что проявляется и в гибридных системах.

Если попытаться дифференцировать гибридные и неоднородные системы, можно отметить, что в гибридной системе обычно сочетаются принципиально разные по архитектуре подсистемы, а в неоднородной – просто несколько отличающиеся. Скажем, неоднородный вычислительный комплекс может включать разные по производительности узлы, но одного типа. Примером неоднородной вычислительной системы также может служить система с ускорителями Intel MIC (Xeon Phi), в которой

сочетаются мощные ядра CPU Xeon и упрощенные ядра Xeon Phi, работающие на меньшей частоте и отделенные от CPU шиной PCI-E, но при этом и те, и другие являются ядрами x86. Гибридная система предполагает более глубокие различия между архитектурами подсистем. Скажем, гибридной является система, построенная на базе многоядерных центральных процессоров (CPU) и графических ускорителей (GPU). Также гибридной является система на базе CPU и ускорителей, оснащенных микросхемами программируемой логики (программируемые логические интегральные схемы - ПЛИС / Field Programmable Gateway Array - FPGA).

Также говоря о гибридных системах, иногда отмечают, что в них различные подсистемы более тесно интегрированы друг с другом, например, располагаются в одном кристалле, в одном корпусе микросхемы или на одной печатной плате, как в решениях AMD, Altera, Cray, СКИФ-ГРИД и в других. В этом случае под гибридными часто понимают неоднородные встраиваемые системы, системы на кристалле (SoC) [3].

Но в целом в литературе в настоящее время термины гибридная вычислительная система и неоднородная (гетерогенная) вычислительная система часто используются как взаимозаменяемые.

Что касается в целом причин рассмотрения в данной работе именно гибридных (или неоднородных) систем, они связаны с поиском путей эффективного использования высокопроизводительных вычислителей. Все чаще исследователи, специалисты приходят к выводу о том, что разные вычислительные задачи требуют разных архитектур. Гибридизация (или неоднородность) вычислительных систем стала практически нормой. Способствует этому развитие и доступность вычислительных ускорителей разного типа: графических (GPU), которые давно используются как вычислители общего назначения, ускорителей на базе ПЛИС (FPGA),

многоядерных ускорители MIC (Many Integrated Cores), представленных недавно Intel.

1.2 Реконфигурируемые вычислительные системы

Реконфигурируемые вычислительные системы (РВС) допускают изменение своей аппаратной конфигурации в процессе функционирования [4]. В широком смысле можно изменить аппаратную конфигурацию большинства вычислительных систем, но для этого чаще всего требуется внешнее вмешательство, в результате которого система будет иметь другой состав компонент или способ их соединения между собой, либо и то, и другое.

В реконфигурируемых системах изменение аппаратной конфигурации может не требовать внешнего вмешательства, но, пожалуй, важнее то, что при изменении конфигурации не меняется ни состав физических компонент системы, ни способ их соединения. Это связано с наличием в составе таких систем специфической элементной базы – программируемых логических интегральных схем (ПЛИС), аппаратная конфигурация которых задается прошивкой, хранящейся в том или ином виде памяти (обычно в статическом ОЗУ либо флэш).

ПЛИС появились и используются достаточно давно, но их применение в качестве основной элементной базы высокопроизводительных реконфигурируемых вычислительных систем, в качестве ускорителей, началось значительно позже, в результате развития технологии ПЛИС и роста их возможностей. Прежде всего, речь идет о степени интеграции и емкости ПЛИС в эквивалентых вентилях. В современных микросхемах FPGA емкость достигает десятков и даже сотен миллионов эквивалентных вентилей, что позволяет размещать в них достаточно сложные аппаратные конфигурации, сравнимые со сложными

CPU и GPU. Частота работы микросхем ПЛИС значительно уступает CPU и GPU (обычно на порядок), поэтому основное преимущество программируемой логики перед заказными большими интегральными схемами (БИС) CPU и GPU состоит в возможности получения аппаратной конфигурации, предназначенной для решения конкретной задачи, то есть в специализации вычислений. Специализация позволяет избавиться от многих накладных расходов, свойственных универсальным системам, максимально использовать (в рамках имеющихся ресурсов ПЛИС) параллелизм задачи, а также возможности конвейеризации вычислений. Последнее обстоятельство представляется особенно важным при сравнении реконфигурируемых ускорителей с другими. Как уже отмечалось выше, различные архитектуры высокой производительности эксплуатируют разные формы параллелизма, но именно конвейерный параллелизм – в меньшей степени, поскольку структура конвейера сильно связана с решаемой задачей. Специализация вычислений, сокращение накладных расходов на универсальность, конвейеризация и необходимый для конкретной задачи параллелизм отличают реконфигурируемые системы на базе ПЛИС при их использовании в качестве высокопроизводительных устройств.

Как и в случае использования другой элементной базы (CPU, GPU и MIC), высокопроизводительные вычислительные устройства на базе ПЛИС могут строиться как многокристальные одноплатные или многоплатные модули с общей памятью. Менее производительные устройства – однокристальные (или с небольшим числом кристаллов – от одного до 4) в виде плат с быстродействующим интерфейсом (в настоящее время PCI-E). Последние представляют собой типовые платы расширения, которые могут рассматриваться как сопроцессоры – ускорители CPU.

Многокристальные платы с несколькими GPU (больше двух) пока распространены редко и являются специализированными решениями для

узлов супер-ЭВМ (high-perfomance computing - HPC), использование нескольких MIC на одной плате пока не планируется. CPU устанавливаются на одну плату в количестве обычно не более 4, в основном это серверные платформы, платформы для высокопроизводительных рабочих станций или специализированные решения для HPC. В случае ПЛИС количество устройств на одной плате может быть относительно большим, поскольку это дает возможность строить достаточно крупные специализированные схемы для решения задач и достижения высокой производительности. Но, как и в случае с многокристальными решениями на базе CPU и GPU, многокристальные решения на ПЛИС достаточно дороги. Серийность их производства намного меньше, чем, например, многопроцессорных плат для CPU.

Тем не менее, такие решения производятся и находят применение в области HPC. Они могут иметь разный масштаб, зачастую позволяют наращивать количество блоков и реализовывать как ускорение одного вычислительного узла, так и построение большего или меньшего по масштабу реального «моря вентилей», состоящего из сотен и тысяч (и более) устройств ПЛИС.

Среди отечественных решений такого класса выделяются разработки НИИ МВС ТРТИ ЮФУ [4,5,6]. Работы по созданию масштабируемых реконфигурируемых вычислительных решений, комплексному обеспечению их применения и собственно внедрению в промышленность выполняются в НИИ МВС в течение уже нескольких десятков лет. За эти годы разработано и внедрено большое количество реконфигурируемых систем на базе различных поколений ПЛИС (рисунок 1.1), создано развитое специализированное программное обеспечение РВС и методика его применения при решении прикладных задач.

В то же время необходимо отметить, что разработки НИИ МВС, как правило, отличаются высокой стоимостью и зачастую недоступны большинству пользователей. Кроме того, они рассчитаны в основном на

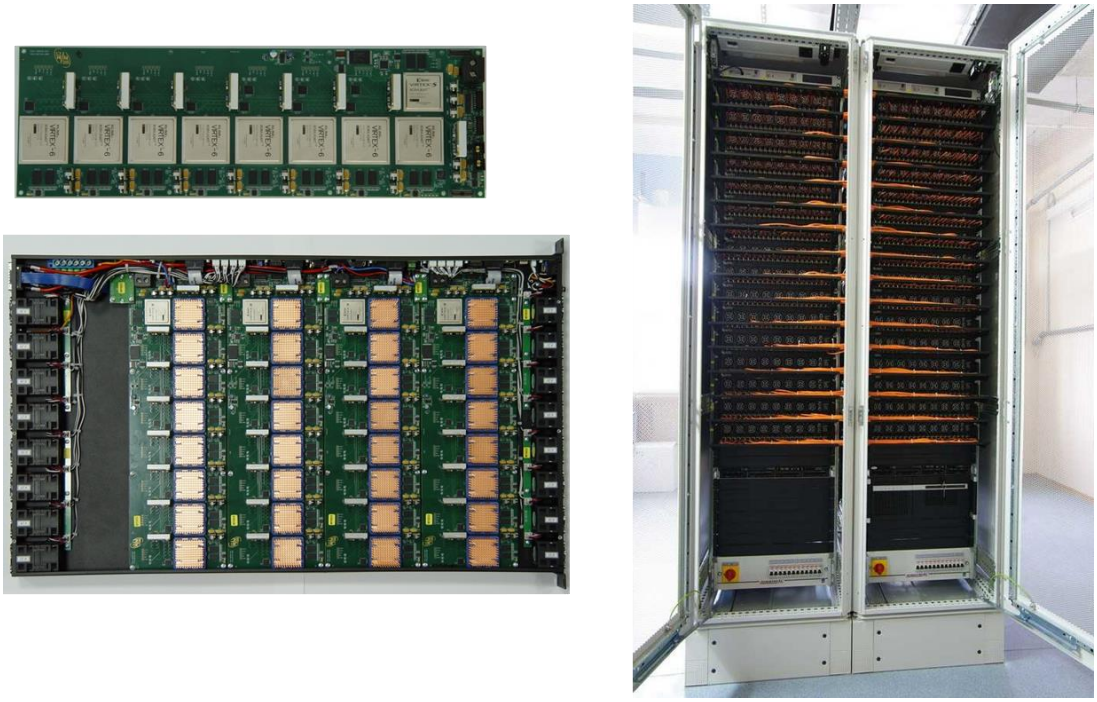


Рис.1.1 – Примеры реконфигурируемых систем, разработанных в НИИ МВС

комплексное применение подхода структурно-процедурных вычислений [4,5], который поддерживается на различных уровнях и требует от разработчиков глубоких специальных знаний всего стека предлагаемых технологий (языка высокого уровня COLAMO, специального ассемблера, средств разработки и так далее – рисунок 1.2 [6]). В силу этих причин использование решений НИИ МВС как ускорителей в составе гибридных многопроцессорных вычислительных систем начального уровня, построенных на базе штатных рабочих станций и серверов, представляется проблематичным.

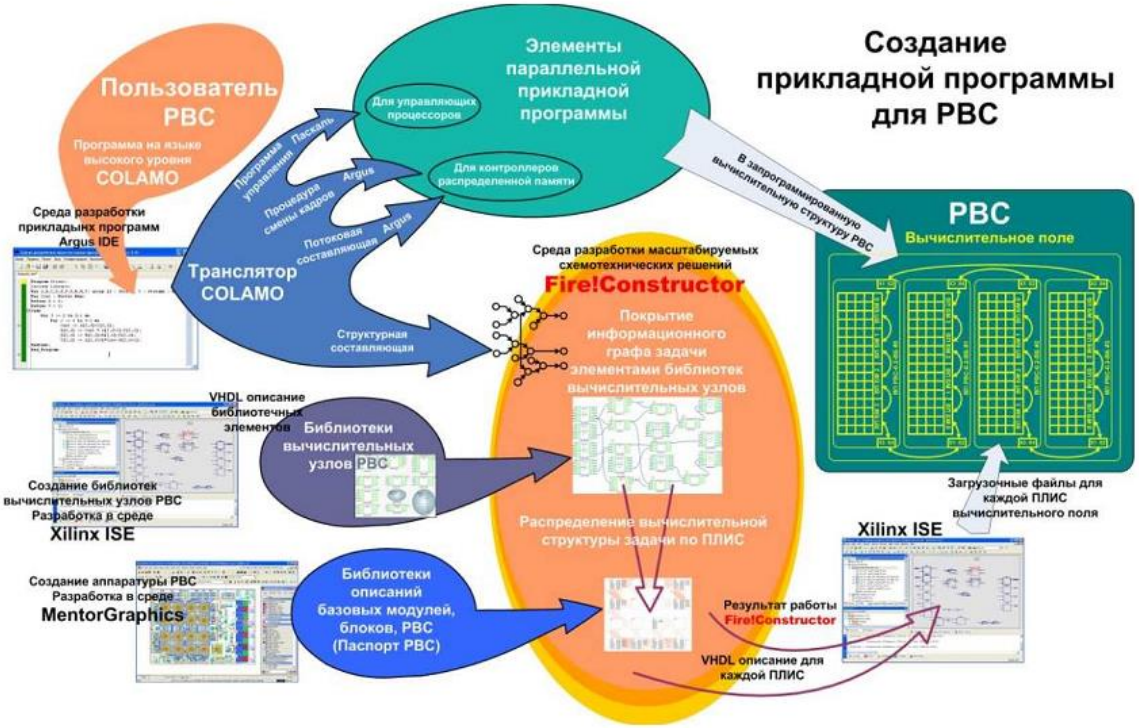


Рис. 1.2 Технология создания прикладных программ для РВС

1.3 Гибридные вычислительные системы начального уровня с реконфигурируемыми сопроцессорами (ГВС РС)

Если ставится задача построения гибридной вычислительной системы начального уровня с реконфигурируемым сопроцессором на базе штатных рабочих станций или серверов, по мнению авторов, целесообразно рассматривать сначала штатные, серийно выпускаемые устройства на базе современных ПЛИС высокой степени интеграции. Даже различные исследовательские наборы разработчика (Development Kit), оснащенные современными высокопроизводительными емкими ПЛИС и интерфейсом PCI-E, на начальном этапе могут служить подходящей элементной базой. Уже после освоения этих устройств и получения навыков их эффективного применения для решения прикладных задач возможна разработка своими силами или заказ на разработку более

специализированных или мощных устройств. Эти устройства должны обладать тем же интерфейсом и поддерживать те же способы работы с ними, что и штатные, но с улучшенными характеристиками (тип и количество ПЛИС, объем и тип памяти, количество каналов PCI-E, и так далее) или включать дополнительные специализированные модули. Но сразу стоит отметить, что затраты на разработку подобных более специализированных устройств довольно высоки и далеко не всегда оправданы.

Далее в данной работе рассматриваются именно гибридные вычислительные системы начального уровня на базе штатных рабочих станций и серверов, а также выпускаемых производителями ПЛИС (прежде всего – Altera) комплектов разработчика, выступающих в роли реконфигурируемых сопроцессоров (конкретно – Arria II GX и DE5-Net на базе ПЛИС компании Altera), и разработка решений с их помощью. Общая структура подобных систем представлена на рисунке 1.3.

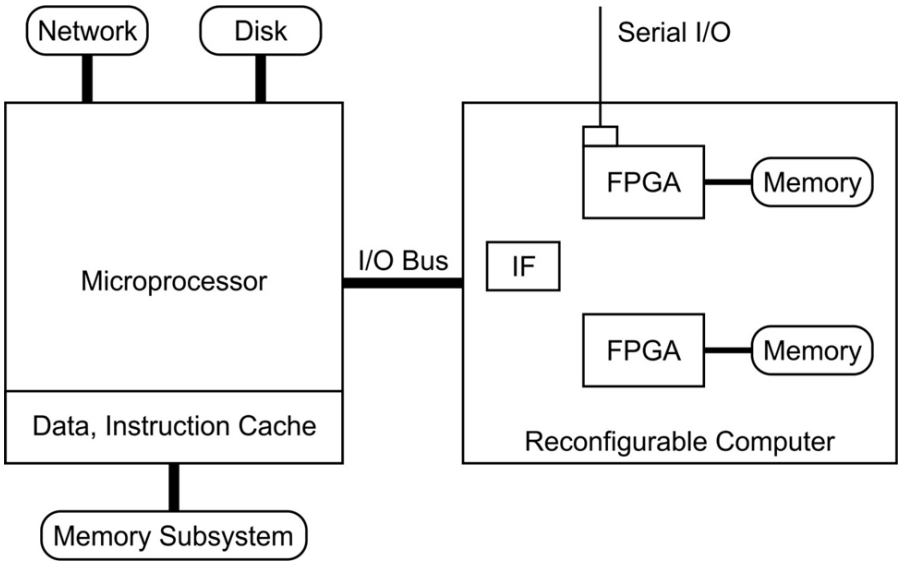


Рис. 1.3 – Общая структура системы с реконфигурируемым сопроцессором

1.4 Выводы

В первой главе рассмотрено понятие гибридной вычислительной системы на базе реконфигурируемых сопроцессоров (ГВС РС). В рамках данной работы основное внимание будет уделено системам на базе штатных рабочих станций или серверов, оснащенных комплектами разработчика в виде плат с программируемыми интегральными схемами (ПЛИС) и интерфейсом PCI-E.

Глава 2

ХАРАКТЕРИСТИКА АЛГОРИТМОВ ДЛЯ РЕАЛИЗАЦИИ НА ГВС РС

Анализ алгоритмов с точки зрения целесообразности их аппаратурной реализации вообще и на ПЛИС в частности проводился во многих работах [7-20]. В частности, в работах [15-20] рассматривается понятие аппаратурно-ориентированных алгоритмов, которые поддерживают эффективную аппаратную реализацию. В данной главе приводятся результаты оценки алгоритмов с точки зрения их возможной реализации на ПЛИС, приведенные в работе [21], и более подробно рассмотренные в магистерских исследованиях И.М. Силкина и Ю.В. Шафрана, выполненных под руководством А.Е. Андреева на кафедре ЭВМ и систем Волгоградского государственного технического университета.

Разработка приложений для сложных архитектур, в частности для FPGA, может быть условно разделена на четыре этапа: формулировка, проектирование, преобразование и реализация. Целью этапа формулировки является исследование алгоритмов, архитектуры и отображения реализации на архитектуру. Здесь стратегические решения принимаются до начала непосредственного кодирования и реализации. На трех других этапах реализация происходит итеративным способом: преобразование в исходные коды и ядра, отладка, проверка, оптимизация производительности и так далее. Процесс отладки и реализации алгоритмов для архитектуры FPGA более трудоемкий, требует наличия знаний в области схемотехники и дополнительных временных затрат по сравнению с другими архитектурами, например, CPU и GPU. Поскольку сложность архитектур FPGA продолжает увеличиваться, также увеличивается важность этапа формулировки, так как на данном этапе

обнаруживаются наиболее слабые места проекта, которые рано или поздно проявятся.

При разработке приложений на FPGA часто встречается ситуация, когда разработчик, имеющий реализованную версию приложения, например, для CPU, не уверен в потенциальном приросте производительности при реализации того же алгоритма на аппаратной платформе. В этом случае необходимо предварительно (до начала реализации) как можно точнее оценить возможный прирост производительности и целесообразность переноса реализации алгоритма на архитектуру FPGA.

Ввиду изложенных обстоятельств можно сделать вывод, что чрезвычайно важной задачей является предварительная оценка прироста производительности при реализации алгоритма с использованием FPGA в качестве сопроцессора, которая позволит удостовериться в целесообразности разработки.

2.1 Общая характеристика алгоритмов, для которых возможна эффективная реализация на ГВС РС

Для эффективной реализации на системах, имеющих в своем составе ПЛИС, алгоритм должен удовлетворять ряду требований, в частности:

1) Значительное время программа, реализующая алгоритм, выполняет собственно вычисления. Такие алгоритмы будем называть вычислительно-интенсивными (computationally intensive [22]) . В этом случае использование ПЛИС можно рассматривать как оптимизацию, которая заменяет эти вычислительные процедуры эффективной аппаратной реализацией.

2) Малый размер вычислительных ядер, которые реализуют процедуры алгоритма, обусловлен небольшим количеством

вычислительных ресурсов подобных систем, и как следствие - накладываемыми ограничениями на сложность реализуемых ядер.

3) Процедуры должны работать с собственными, а не разделяемыми данными (особенность систем, где ПЛИС стоят на отдельной плате). Подобные платы имеют свои банки памяти, обмен с которыми идет на значительно большей скорости, чем с ОЗУ ЦПУ. Поэтому для максимальной эффективности все данные должны помещаться в эту память.

4) Передача данных между платой с установленной ПЛИС и ОЗУ ЦПУ должна быть минимальной по сравнению с объемом вычислений (также является особенностью систем, где ПЛИС установлены на отдельной плате). Другими словами, задача должна обладать высокой вычислительной мощностью, чтобы эффективно обрабатываться на ПЛИС. Это требование перекликается с первым и также характеризует вычислительно-интенсивные алгоритмы.

5) Ограниченное пространство для работы самих ядер вытекает из сравнительно небольшого объема ОЗУ ПЛИС и из невысокой скорости обмена ПЛИС - ЦПУ. Другими словами, ядро не должно порождать больших вспомогательных массивов данных (т.е. больших по порядку, нежели размер входных или выходных данных).

6) Параллельная декомпозиция задачи для ПЛИС должна хорошо ложиться на топологию, по которой соединены отдельные платы с ПЛИС, если их несколько.

2.2 Методики оценки целесообразности реализации алгоритмов на реконфигурируемых сопроцессорах

В различных работах было предложено несколько методик, так или иначе затрагивающих прогнозирование производительности решений для

архитектуры программируемых логических интегральных схем (ПЛИС / FPGA).

В работе [7] приводится методика количественного анализа факторов ускорения при реализации алгоритмов на FPGA. Недостатками данной методики являются отсутствие учёта передачи данных и концентрация внимания только на параллельном выполнении операций и получаемом от этого ускорении. Для оценки производительности приложений на FPGA данная методика мало пригодна.

Предложенная в [8] Performance Prediction Model включает в рассмотрение 19 параметров, на основании которых рассчитываются частота работы схемы и пропускная способность вычислений при различной организации вычислений. При этом точность методики прогнозирования составляет около 20%. Авторы Performance Prediction Model рекомендуют использовать их методику скорее для выбора схемы вычислений, чем для оценки производительности.

Методика, изложенная в работе [9], также включает в рассмотрение большое количество параметров и сосредотачивается в основном на работе схемы с подсистемой памяти, поэтому данная методика также не подходит для анализа предполагаемой производительности FPGA как сопроцессора.

В работе [10] предложена методика оценки времени выполнения реализаций алгоритмов на распределённой разнородной вычислительной системе, имеющей в своём составе реконфигурируемые устройства. При анализе времени выполнения методика предполагает использование уже известных показателей производительности отдельных задач, выполняемых на узлах вычислительной системы. Это делает ее непригодной для анализа применимости сопроцессоров.

Для проведения быстрого и простого исследования реализации алгоритмов на FPGA можно воспользоваться методикой «Reconfigurable Amenability Test» (RAT), описанной в [11] и [12].

RAT использует три критерия для определения целесообразности реализации алгоритма на FPGA: пропускную способность, числовую точность и необходимые ресурсы. Данные факторы являются доминирующими при оценке полной эффективности реализации алгоритмов на конкретной элементной базе. Анализ пропускной способности позволяет предсказать производительность приложения, используя несколько простых уравнений, основываясь на известных параметрах (например, скорость обмена данными между CPU и сопроцессором) и значениях, характерных для конкретного алгоритма (например, объем передаваемых данных). Анализ числовой точности оценивает пропускную способность, исходя из требований алгоритма к формату данных и связанных с ним возможных погрешностей. Анализ необходимых ресурсов подразумевает оценку аппаратных средств, которые потребуются для реализации выбранного алгоритма.

Методология RAT предполагает, что первоначально определяется необходимая точность приложения, затем проводится тест пропускной способности для получения прогнозируемой производительности алгоритма. Числовая точность должна быть сбалансирована относительно типа и количества доступных ресурсов для поддержки требуемого формата.

На рисунке 2.1 представлена схема процесса разработки приложений для архитектуры FPGA с использованием методики RAT.

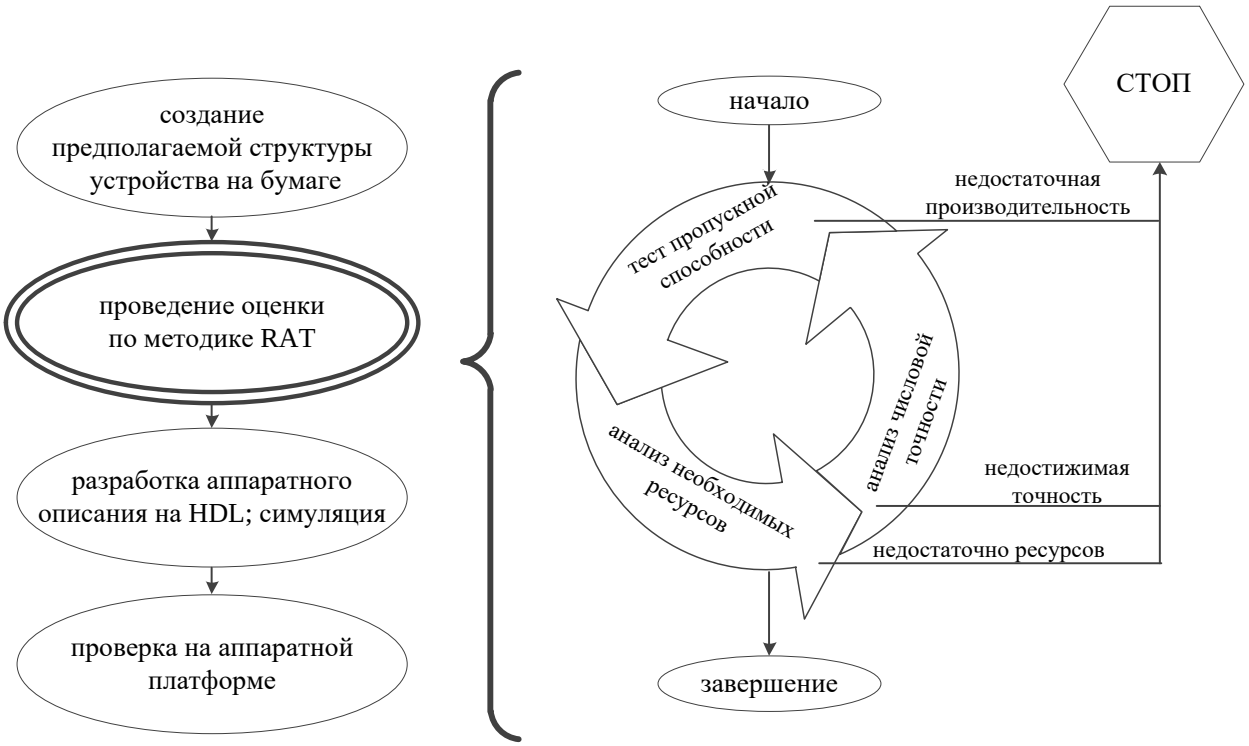


Рис. 2.1. Использование RAT в процессе разработки приложений

Разрабатываемые аппаратные схемы проектов для архитектуры FPGA будут ограничены в размере количеством трех основных ресурсов: памятью на микросхеме, встроенными функциональными модулями (например, умножителями) и основными логическими элементами. Количество необходимой памяти можно достаточно точно оценить, исходя из реализуемого алгоритма. Кроме того необходимо определиться с количеством требуемых функциональных блоков (например, DSP), которые имеются в большинстве современных FPGA.

Прогнозируемое время выполнения алгоритма определяется исходя из двух показателей: времени обмена данными между CPU и сопроцессором и времени непосредственных вычислений. Время передачи данных определяется в соответствии с выражениями (2.1), (2.2) и (2.3).

$$t_{comm} = t_{read} + t_{write}, \tag{2.1}$$

$$t_{read} = \frac{N_{elements} \cdot N_{bytes/element}}{\alpha_{read} \cdot throughput_{ideal}}, \quad (2.2)$$

$$t_{write} = \frac{N_{elements} \cdot N_{bytes/element}}{\alpha_{write} \cdot throughput_{ideal}}, \quad (2.3)$$

где

t_{comm} – время обмена данными между CPU и сопроцессором;

t_{read} – время чтения данных из сопроцессора;

t_{write} – время записи данных в сопроцессор;

$N_{elements}$ – количество передаваемых элементов данных;

$N_{bytes/element}$ – размер элемента данных в байтах;

$throughput_{ideal}$ – идеальная пропускная способность при чтении и записи;

α_{write}, α_{read} – коэффициенты, характеризующие отношение реальной и идеальной пропускной способности при записи и чтении соответственно.

Оценка времени непосредственных вычислений на сопроцессоре производится согласно формуле (2.4).

$$t_{comp} = \frac{N_{elements} \cdot N_{operations/element}}{f_{clock} \cdot throughput_{process}}, \quad (2.4)$$

где

t_{comp} – время непосредственных вычислений на сопроцессоре;

$N_{elements}$ – количество обрабатываемых элементов данных;

$N_{operations/element}$ – количество операций над элементом данных, необходимых для вычисления конечного значения;

f_{clock} – тактовая частота вычислителя;

$throughput_{process}$ – количество операций за такт.



Рис.2.2 Схема процесса передачи данных и вычислений
при двойной и одиночной буферизации данных

В зависимости от типа буферизации данных, одиночной или двойной, общее время выполнения алгоритма определяется по формулам (2.5) и (2.6).

$$t_{RC_{SB}} = N_{iterations} \cdot \left(t_{comm} + t_{comp}\right) \tag{2.5}$$

$$t_{RC_{DB}} \approx N_{iterations} \cdot MAX\left(t_{comm}, t_{comp}\right) \tag{2.6}$$

где

$N_{iterations}$ - количество итераций передачи данных и вычислений, необходимых для решения задачи;

$t_{RC_{SB}}$ - общее время вычислений на сопроцессоре при использовании одиночной буферизации (SB);

$t_{RC_{DB}}$ - общее время вычислений на сопроцессоре при использовании двойной буферизации (DB).

Двойная буферизация позволяет одновременно осуществлять передачу данных и непосредственные вычисления (рис.2.2) [12].

Выражения (2.7), (2.8), (2.9) и (2.10) представляют собой метрики, которые определяют часть полного времени, используемую на пересылку данных, и часть, используемую для выполнения непосредственных вычислений на FPGA. Формулы различаются для сценариев одиночной и двойной буферизации данных. При одиночной буферизации данных метрики могут использоваться для оценки возможности получения дополнительного ускорения. Например, если показатели метрики, определяющей непосредственные вычисления, высокие, то FPGA редко простаивает, и можно сделать заключение о приближении к максимально достижимому ускорению. Низкие показатели данной метрики указывают на существующий потенциал увеличения ускорения, при условии, если алгоритм будет переформулирован таким образом, что будет иметь меньшее время обмена данными и/или больше наложений обмена данными и непосредственных вычислений за счет реализации двойной буферизации.

$$util_{comp_SB} = \frac{t_{comp}}{t_{comm} + t_{comp}} \tag{2.7}$$

$$util_{comm_SB} = \frac{t_{comm}}{t_{comm} + t_{comp}} \tag{2.8}$$

$$util_{comp_DB} = \frac{t_{comp}}{Max(t_{comm}, t_{comp})} \tag{2.9}$$

$$util_{comm_DB} = \frac{t_{comm}}{Max(t_{comm}, t_{comp})} \tag{2.10}$$

Интерпретация значений метрик для двойной буферизации данных немного отличается от варианта с одиночной буферизацией. Наибольшее значение, которое может принимать метрика (передачи данных или вычислений) равно 1. Если время вычислений больше времени передачи данных (как показано в третьем сценарии на рис.2.2), то это означает, что для текущих данных достигнуто максимальное ускорение и ресурсы исчерпаны. Если время передачи данных больше времени вычислений, то соответствующая метрика отражает часть доступной пропускной

способности, которая может быть реализована при добавлении дополнительных параллельных вычислений.

Данная методика успешно применялась в процессе реализации на архитектуре FPGA таких алгоритмов, как вычисление функции плотности распределения вероятностей [12], задача Коммивояжёра [12], задача молекулярной динамики [13] и другие.

2.3 Прогнозирование производительности при реализации алгоритма QR-разложения матрицы

Проведем прогнозирование реализации алгоритма QR разложения для матриц большой размерности с помощью методики RAT на архитектуре FPGA [21].

QR-разложение матрицы A — представление матрицы A в виде произведения унитарной (или ортогональной матрицы) Q и верхнетреугольной матрицы R (формулы 2.11, 2.12, 2.13, 2.14).

$$A \cdot Y = Z \tag{2.11}$$

$$(Q \cdot R) \cdot Y = Z \tag{2.12}$$

$$R \cdot Y = Q^{-1} \cdot Z \tag{2.13}$$

$$R \cdot Y = Z^{'}, \text{ где } Z^{'} = Q^{-1} \cdot Z \tag{2.14}$$

где

A – матрица размерностью N·N;

Y и Z – векторы-столбцы размерностью N.

Применяется при решении систем линейных уравнений, при отыскании собственных значений и в других задачах.

Для реализации разложения на FPGA выбран метод вращений Гивенса, ввиду преимуществ параллелизма, которые предлагает данный метод. Стандартная архитектура устройства, реализующего данный алгоритм на аппаратной архитектуре, представляет собой систолический

массив спецпроцессоров CORDIC в режиме вращения и векторном режиме (рис.2.3) [23].

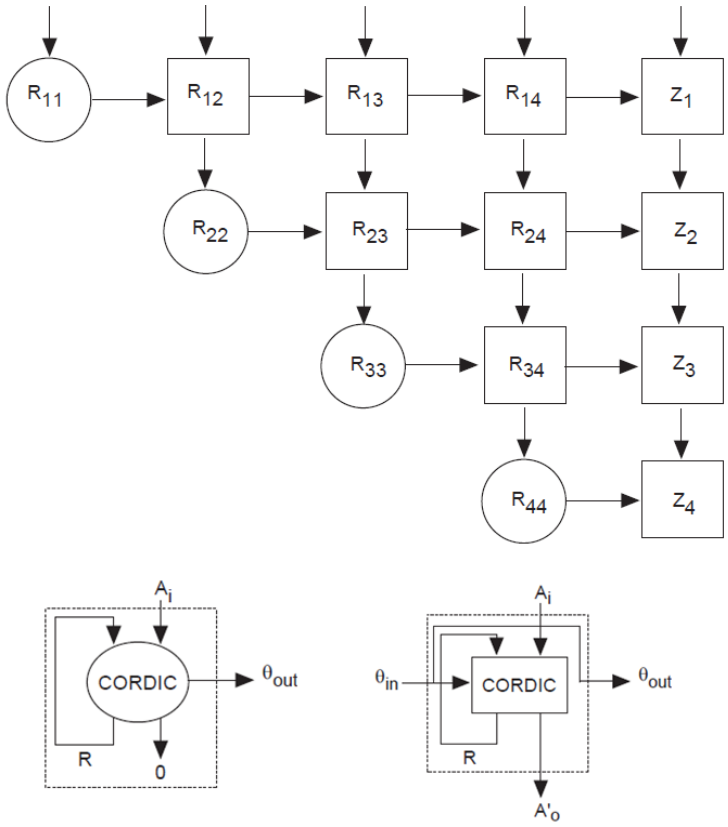


Рис.2.3. Систолический массив для QR разложения

Спецпроцессор CORDIC в векторном режиме производит поворот вектора, таким образом, что ордината вектора становится равной 0. В результате вычисляется значение угла поворота. CORDIC в режиме вращения производит поворот вектора на заданный угол. Выходными данными являются новые координаты вектора. Данные преобразования представлены на рис.2.4.

Проведем оценку аппаратных ресурсов, требующихся для реализации CORDIC процессоров. Ввиду имеющихся в открытом доступе ядер, разработанных на языке описания аппаратных схем и реализующих

алгоритмы CORDIC, проанализируем их средствами САПР Quartus II. Результат представлен в табл.2.1.

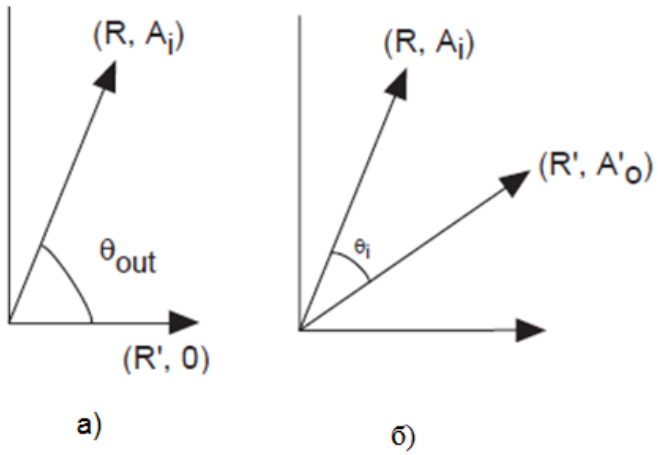


Рис.2.4. Преобразования CORDIC: а) векторный режим; б) режим вращения

Таблица 2.1

Анализ аппаратных ресурсов спецпроцессоров CORDIC

Режим спецпроцессора	Количество необходимых логических блоков (ALUT)		
	16 бит	32 бита	64 бита
векторный	2068	6014	18568
вращение	2485	6882	19681

Ввиду того, что данные устройства работают с числами формата с фиксированной запятой, для обеспечения точности, аналогичной типу float языков высокого уровня, необходимо использовать устройства, работающие с 64-разрядными данными (хотя для некоторых приложений достаточно и меньшей разрядности).

Оценим количество CORDIC процессоров, которое возможно разместить в FPGA различной емкости. Результат оценки представлен в табл.2.2.

Таблица 2.2

Анализ аппаратных ресурсов спецпроцессоров CORDIC

Емкость FPGA в логических блоках	Количество устройств CORDIC (64 бита)
140000	7
500000	24
800000	40

Анализ ресурсов показывает, что ёмкости современных FPGA недостаточно для того, чтобы вместить весь систолический массив для матриц большой размерности. Поэтому для прогнозирования производительности используем структуру устройства, представляющую собой несколько CORDIC устройств, предназначенных для обработки части строки матрицы (рис.2.5). Произведем прогнозирование производительности реализации данного алгоритма, согласно описанной в пункте 2.1 методике RAT на диапазоне размерностей матриц. При оценке времени передачи данных примем значение пропускной способности равной пропускной способности шины PCI Express 1Gen 4x. В качестве элемента принимаем всю обрабатываемую матрицу. Количество операций за такт принимаем равным (CORDIC – 1)*64, ввиду того, что рассматриваемые устройства CORDIC конвейеризованы.

Результаты расчета сведены в табл.2.3.

По результатам прогнозирования видно, что ускорение зависит от емкости кристалла FPGA и, соответственно, от количества CORDIC процессоров. При использовании отладочной платы с кристаллом FPGA Altera Arria II GX EP2AGX125, емкость которого составляет около 150000 логических блоков, реализация рассматриваемой задачи не имеет большого смысла, поскольку прогнозируемое ускорение не превышает 10-13 % по сравнению с не самым мощным CPU.

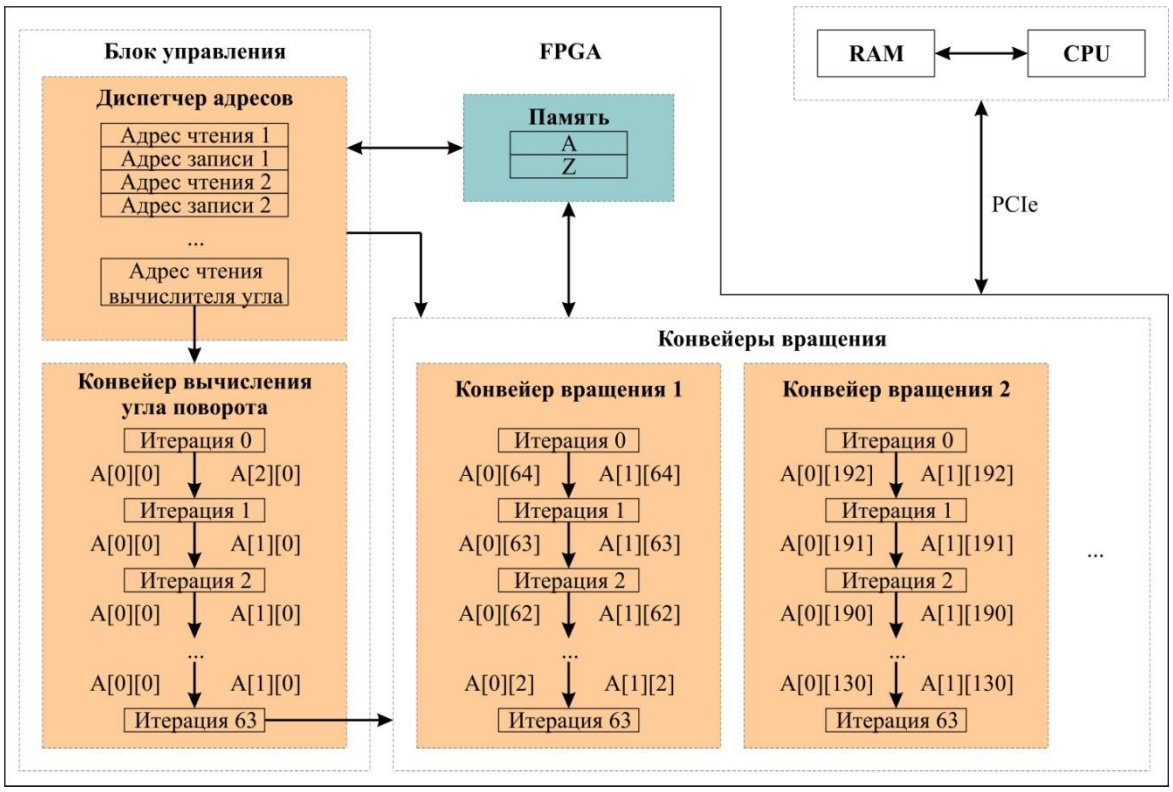


Рис.2.5. Структура устройства, реализующего QR разложение

Таблица 2.3

Результаты прогнозирования производительности реализации QR декомпозиции

Емкость кристалла, логические блоки	Количество процессоров CORDIC	Прогнозируемое ускорение (сравнение с Intel Core i3-2125)				
		Размерность матрицы NxN, N				
		512	1024	4096	8192	16384
140 000	7	1,10	1,12	1,13	1,13	1,13
500 000	24	3,42	3,81	4,20	4,22	4,22
800 000	40	4,92	5,87	6,92	7,05	7,27

Кроме того, обеспечение одновременной загрузки данных из памяти на конвейеры и записи выходных данных всех конвейеров в память представляется достаточно сложной задачей без использования памяти на чипе. Однако ресурсы чипа полностью отданы конвейерам. Более того, при расчете не учитывались аппаратные затраты на организацию диспетчера. Все это позволяет утверждать, что для платы Arria II GX реализация рассмотренного алгоритма не имеет смысла. Но при наличии более емких кристаллов (как например, в DE5-Net) ускорение возможно.

2.4 Оценка производительности при реализации криптографического алгоритма AES

В качестве следующего примера применения представленной в пункте 2.2.1 методики проведём расчёт прогнозируемого ускорения при реализации криптографического алгоритма AES.

AES представляет собой алгоритм шифрования 128-битных блоков данных ключами по 128, 192 и 256 бит. Над каждым блоком исходного текста производится определенное количество раундов шифрования. В зависимости от размера ключа варьируется и число раундов. Для 128 битного ключа – 10 раундов, для 192 битного – 12, для 256 битного – 14 раундов. Пусть в данной реализации длина ключа 128 бит, режим работы выберем ECB (Electronic Codebook). Предполагаемая структура устройства, реализующего рассматриваемый алгоритм, представлена на рис.2.6.

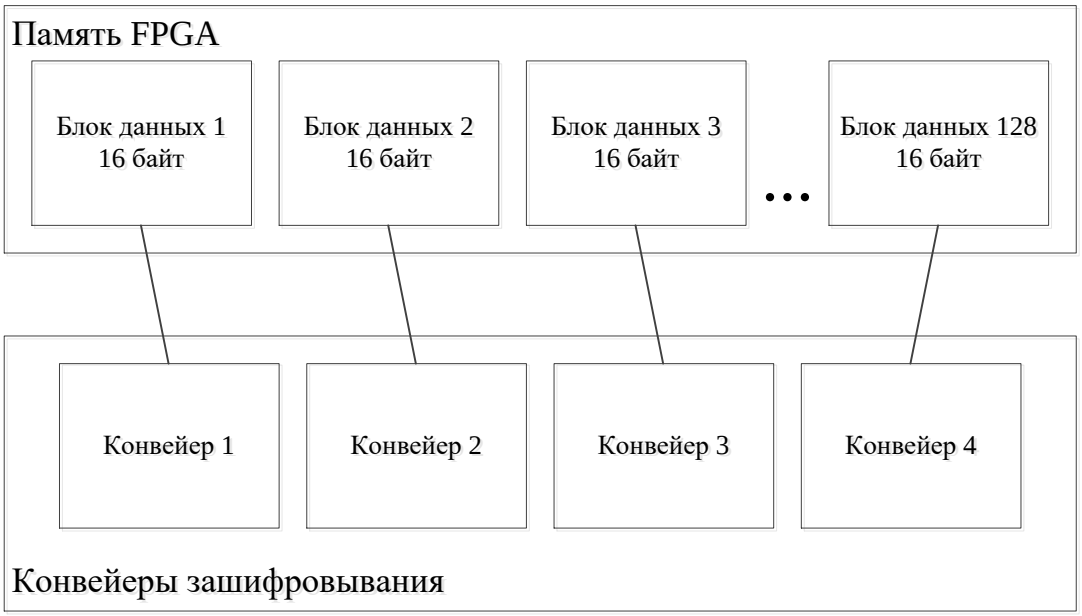


Рис.2.6. Предполагаемая структура устройства, реализующего шифрование

Оценим задачу зашифровывания данных объемом 256 Мб = 268435456 байт с помощью криптографического алгоритма AES. Из структуры предполагаемого устройства видно, что на каждой итерации алгоритма шифруется порция данных в 128 блоков по 16 байт. Для того чтобы зашифровать 256 Мб текста потребуется 131072 итераций. После окончания каждой итерации данные возвращаются в CPU. Т.е. если следовать терминологии RAT, получаем $N_{elements_input}$ = 128, $N_{elements_output}$, $N_{byte/elements}$=16, $N_{iterations}$=131072.

Структура конвейеров зашифровывания представлена на рис.2.7. Блок имеет «развернутую» архитектуру AES шифрования. Это позволяет увеличить пропускную способность за счет дополнительных аппаратных затрат на организацию конвейера.

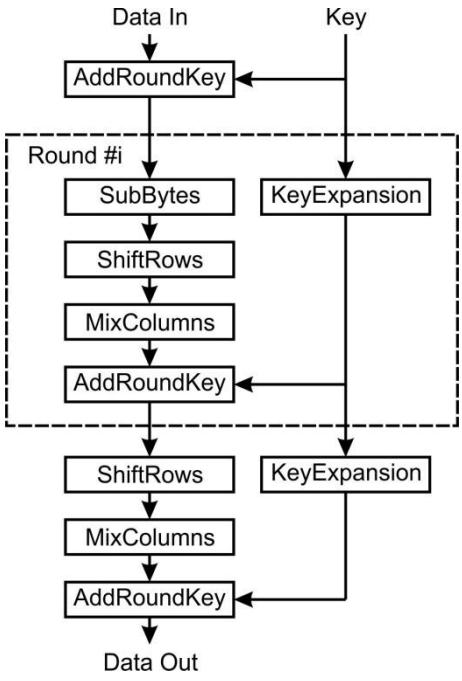


Рис.2.7. Структура конвейера зашифровывания

Процедура Key Expansion, формирующая ключи для раундов, не выделяется в отдельный блок: ключи вычисляются на каждом раунде.

Понятие «операция» в RAT достаточно субъективное и определяется разработчиком. Единственное требование – величина (число операций за цикл) должна вычисляться в том же самом контексте. Под количеством операций будем понимать количество процедур, аналогичных высокоуровневому алгоритму, которые надо провести над блоком данных, чтобы его зашифровать, тогда нам необходимо выполнить 44 операции над блоком. Учитывая, что у нас имеется 4 параллельных конвейера, каждый из которых после полной загрузки выполняет 44 операций за цикл, то общее количество операций – 44*4 = 176. Уменьшим данное число до 150, тем самым попытаемся учесть задержки конвейера и издержки вычислений и составить более пессимистичную оценку.

Входные параметры для проведения теста RAT сведены в табл.2.4.

Таблица 2.4

Параметры для теста RAT

Параметр	Размерность	Значение
$N_{elements}$, input	(элементов)	128
$N_{elements}$, output	(элементов)	128
$N_{bytes/element}$	(байт/ элемент)	16
$throughput_{ideal}$	(Гб/с)	1
α_{write}	$0 < \alpha < 1$	0,7
α_{read}	$0 < \alpha < 1$	0,7
$N_{ops/element}$	(операций/элемент)	120
$throughput_{proc}$	(операций/цикл)	450
f_{clock}	(МГц)	75/100/150
t_{soft}	(секунды)	16,56
N_{iter}	(итераций)	131072

Результаты расчетов согласно методике RAT приведены в табл.2.5.

Таблица 2.5

Результаты оценки AES

Результаты оценки			
f_{clock} (МГц)	75	100	150
t_{comm} (с)	0,714	0,714	0,714
t_{comp} (с)	0,056	0,042	0,028
t_{RC} (с)	0,771	0,757	0,742
ускорение	12,81	13,05	13,30

Как видно из результатов оценки, время непосредственных вычислений на FPGA на порядок меньше времени передачи данных. Но даже при таком ограничении пропускной способности получены значительные показатели ускорения, по сравнению с реализацией на CPU.

При увеличении пропускной способности возможно получение больших значений ускорения.

Данная задача приведена только в качестве примера применения методики RAT и реализации не подлежит, ввиду того, что в современных процессорах Intel Core (например, Intel Core i5-661), Intel Xeon и даже Atom уже предусмотрена аппаратная поддержка данного алгоритма [24].

2.5 Оценка производительности при реализации задачи восстановления строки по MD5 хэш-сумме

Произведем оценку производительности реализации задачи восстановления строки по MD5 хэшу на архитектуре FPGA. Для расчетов используем показатели ресурсов, которыми обладает чип Arria II GX EP2AGX125 (подробнее они рассмотрены в следующей главе).

Данная задача решается следующим образом. Производится полный перебор возможных строк входного алфавита нахождением их хэша по алгоритму MD5, сравнение полученного хэша с искомым [25].

Предполагаемая структура устройства восстановления строки представлена на рисунке 2.8.

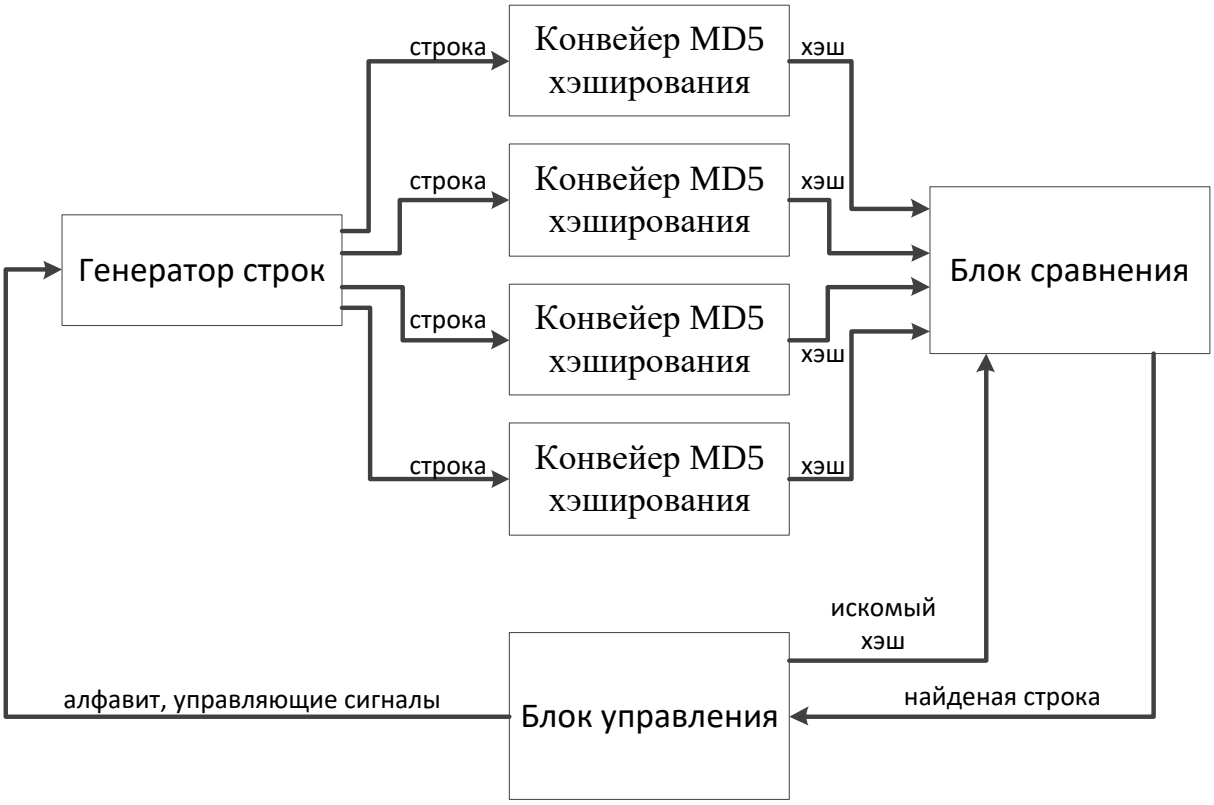


Рис.2.8. Предполагаемая структура устройства

Все элементы структуры представленной схемы конвейерные. Рассмотрим подробнее назначение каждого из модулей устройства. Генератор строк формирует строки из заданного алфавита и передает их на устройства хэширования. Конвейер MD5 хэширования вычисляет значение хэш-суммы входной строки по алгоритму MD5. Предполагается наличие четырех таких модулей, работающих параллельно. В блоке сравнения происходит сопоставление хэша искомой строки и хэш-сумм, полученных с выходов конвейеров хэширования. В случае совпадения в устройство управления передается искомая строка. Управляющее устройство

контролирует процесс поиска, инициализирует управляющие регистры данными, полученными по каналу PCIE от CPU, и сообщает результат.

Ввиду того, что в данной задаче количество вычислений зависит от входных данных, методику RAT в её эталонном виде применить невозможно, так как заранее нельзя вычислить количество выполняемых операций над входными данными. Исходя из этого, используя только некоторые формулы RAT, произведем расчет количества хэш-сумм, которое способно проверить разрабатываемое устройство за секунду. Аналогичный показатель для CPU был получен экспериментально и составляет приблизительно 45 миллионов хэш-сумм в секунду. Данная величина получена экспериментальным путем.

Рассмотрим подробнее алгоритм MD5 (Message Digest 5). MD5 – это 128-битный алгоритм хеширования. Предназначен для создания «дайджестов» сообщений произвольной длины и последующей проверки их подлинности.

Алгоритм MD5 состоит из последовательных 5 этапов.

Первый этап – выравнивание потока: дописывается единичный бит в конец потока (байт 0x80), затем необходимое число нулевых бит; входные данные выравниваются так, чтобы их новый размер был сравним с 448 по модулю 512 (512*N+4).

Второй этап – добавление длины сообщения: в оставшиеся 64 бита дописывают 64-битное представление длины данных (количество бит в сообщении) до выравнивания; сначала записывают младшие 4 байта; после этого длина потока станет кратной 512.

Третий этап – инициализация буфера: для вычислений инициализируются 4 переменных размером по 32 бита и задаются начальные значения (A = 0x01234567; B = 0x89ABCDEF; C = 0xFEDCBA98; D = 0x76543210).

Четвертый этап – вычисления в цикле. Для данного этапа потребуются логические функции, заданные выражениями (2.15, 2.16, 2.17, 2.18).

$$FunF(X,Y,Z) = (X \wedge Y) \vee (\neg X \wedge Z) \quad (2.15)$$

$$FunG(X,Y,Z) = (X \wedge Z) \vee (\neg Z \wedge Y) \quad (2.16)$$

$$FunH(X,Y,Z) = X \oplus Y \oplus Z \quad (2.17)$$

$$FunI(X,Y,Z) = Y \oplus (\neg Z \vee X) \quad (2.18)$$

Помимо этого понадобится 64-элементная таблица констант, построенная по формуле 2.19:

$$T[i] = \text{int}(4\,294\,967\,296 * |\sin(i)|) \quad (2.19)$$

Далее над блоком производятся операции, представленные в табл.2.6. Данные операции разделены на 4 раунда.

После того, как над блоком произведутся операции на всех раундах, необходимо просуммировать значения переменных A, B, C, D с результатом предыдущего цикла.

Пятый этап – результат вычислений. Результат вычислений находится в буфере ABCD, это и есть хэш-сумма. Если выводить побайтово, начиная с младшего байта A и закончив старшим байтом D, то получим MD5-хеш.

Так как устройства хэширования конвейеризованы, то после заполнения конвейера на каждом такте будет доступна хэш-сумма новой строки. Производительность такого конвейера будет зависеть лишь от тактовой частоты, на которой он сможет работать. Тактовая частота, в свою очередь, зависит от размера ступени конвейера в логических элементах.

Таблица 2.6

Операции на раундах

№ раунда	Операции
1	/*[abcd k s i] a = b + ((a + F(b,c,d) + X[k] + T[i]) <<< s). */ [ABCD 0 7 1][DABC 1 12 2][CDAB 2 17 3][BCDA 3 22 4] [ABCD 4 7 5][DABC 5 12 6][CDAB 6 17 7][BCDA 7 22 8] [ABCD 8 7 9][DABC 9 12 10][CDAB 10 17 11][BCDA 11 22 12] [ABCD 12 7 13][DABC 13 12 14][CDAB 14 17 15][BCDA 15 22 16]
2	/*[abcd k s i] a = b + ((a + G(b,c,d) + X[k] + T[i]) <<< s). */ [ABCD 1 5 17][DABC 6 9 18][CDAB 11 14 19][BCDA 0 20 20] [ABCD 5 5 21][DABC 10 9 22][CDAB 15 14 23][BCDA 4 20 24] [ABCD 9 5 25][DABC 14 9 26][CDAB 3 14 27][BCDA 8 20 28] [ABCD 13 5 29][DABC 2 9 30][CDAB 7 14 31][BCDA 12 20 32]
3	/*[abcd k s i] a = b + ((a + H(b,c,d) + X[k] + T[i]) <<< s). */ [ABCD 5 4 33][DABC 8 11 34][CDAB 11 16 35][BCDA 14 23 36] [ABCD 1 4 37][DABC 4 11 38][CDAB 7 16 39][BCDA 10 23 40] [ABCD 13 4 41][DABC 0 11 42][CDAB 3 16 43][BCDA 6 23 44] [ABCD 9 4 45][DABC 12 11 46][CDAB 15 16 47][BCDA 2 23 48]
4	/*[abcd k s i] a = b + ((a + I(b,c,d) + X[k] + T[i]) <<< s). */ [ABCD 0 6 49][DABC 7 10 50][CDAB 14 15 51][BCDA 5 21 52] [ABCD 12 6 53][DABC 3 10 54][CDAB 10 15 55][BCDA 1 21 56] [ABCD 8 6 57][DABC 15 10 58][CDAB 6 15 59][BCDA 13 21 60] [ABCD 4 6 61][DABC 11 10 62][CDAB 2 15 63][BCDA 9 21 64]

Ниже представлено планируемое построение конвейера для блока вычислений MD5 хэш (рис.2.9).

Проанализируем возможную производительность устройства на диапазоне частот, на которых реально может работать FPGA Arria II GX EP2AGX125 (табл.2.7). Ввиду конвейеризации блоков хэширования, каждый такой блок после полной загрузки конвейера способен выдавать хэш-сумму новой строки на каждом такте. То есть при частоте 125 МГц данное устройство должно производить (4 * 125) млн хэш-сумм за 1 с.

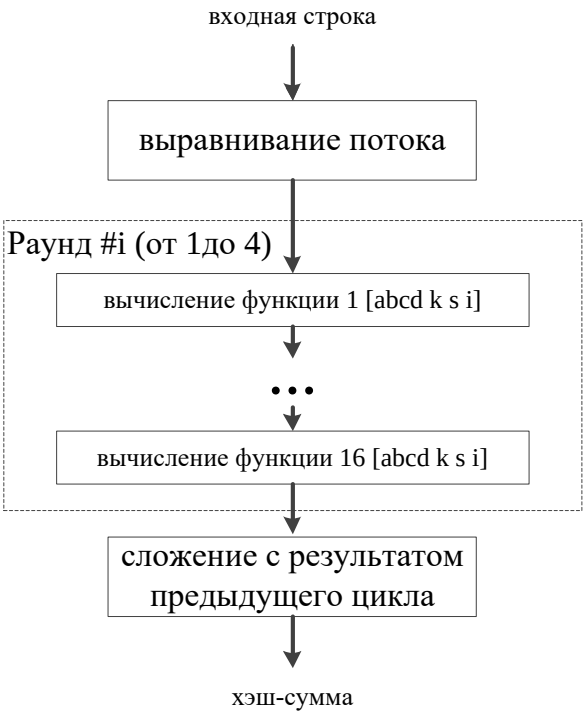


Рис.2.9. Конвейер блока вычислений хэша

Таблица 2.7

Прогнозируемая производительность

	CPU	Arria II GX EP2AGX125		
		100 МГц	150 МГц	200 МГц
Производительность, (млн хэш-сумм)/с	45	400	600	800
Прогнозируемое ускорение	1	8,89	13,33	17,78

Результаты прогнозирования позволяют сделать вывод, что данное устройство подходит для реализации на FPGA Arria II GX EP2AGX125 (табл.2.7). Как известно, заранее предсказать тактовую частоту разрабатываемой схемы невозможно. Она будет известна после схемной реализации и временного анализа САПР Quartus II.

2.6 Выводы

В данной главе рассмотрен вопрос прогнозирования производительности реализаций алгоритмов на архитектуре FPGA. Предварительное прогнозирование производительности до начала реализации алгоритмов на FPGA является одним из важных моментов создания приложений для данных архитектур. Кроме того, что оценка дает возможность с достаточной точностью предсказать результат конкретной реализации алгоритма, она позволяет уже на первом этапе разработки выделить задачи, неперспективные с точки зрения повышения производительности на рассматриваемых архитектурах.

В этом разделе рассмотрено применение методики «Reconfigurable Amenability Test» для оценки производительности реализаций алгоритмов на реконфигурируемой архитектуре. Произведено прогнозирования для следующих задач: QR-декомпозиция матрицы с помощью алгоритма CORDIC, криптографического алгоритма AES и задачи восстановления строки по MD5 хэш-сумме. Реализация задачи QR-декомпозиции матрицы для устройств уровня Arria II GX неперспективна, ввиду невозможности получения требуемых значений ускорения. Аппаратная реализация AES позволяет получить ускорение при сравнении с последовательной версией для CPU, но так как в ряде современных процессоров уже предусмотрена аппаратная поддержка данного алгоритма, реализация данного алгоритма на платформе FPGA нецелесообразна. Рассмотренный вариант реализации алгоритма восстановления строки по MD5 хэш-сумме является перспективной задачей с точки зрения повышения производительности и позволяет отразить возможности применения FPGA в качестве сопроцессора, как и криптологические задачи в целом.

Глава 3

СТРУКТУРА И ПОСТРОЕНИЕ ГИБРИДНОЙ СИСТЕМЫ С РЕКОНФИГУРИРУЕМЫМИ СОПРОЦЕССОРАМИ

3.1 Семейства ПЛИС для реконфигурируемых систем

На текущий момент в мире существует два ведущих производителя ПЛИС: компании Altera и Xilinx. Продукты этих фирм последних поколений имеют схожие характеристики, приведенные далее для основных семейств ПЛИС [26, 27].

3.1.1 Семейства ПЛИС Altera

Младшее семейство микросхем Cyclone V компании Altera выполнено по оптимизированному 28 нм техпроцессу и является самым недорогим, функциональным и малопотребляющим решением со встроенными трансиверами на рынке.

Таблица 3.1

Эволюция микросхем семейства Cyclone

	Cyclone FPGA	Cyclone II FPGA	Cyclone III FPGA	Cyclone IV FPGA	Cyclone V FPGA
Год выпуска	2002	2004	2007	2009	2011
Технологический процесс, нм	130	90	65	60	28

Основной составляющей архитектуры микросхем Cyclone V является логический элемент (рисунок 3.1).

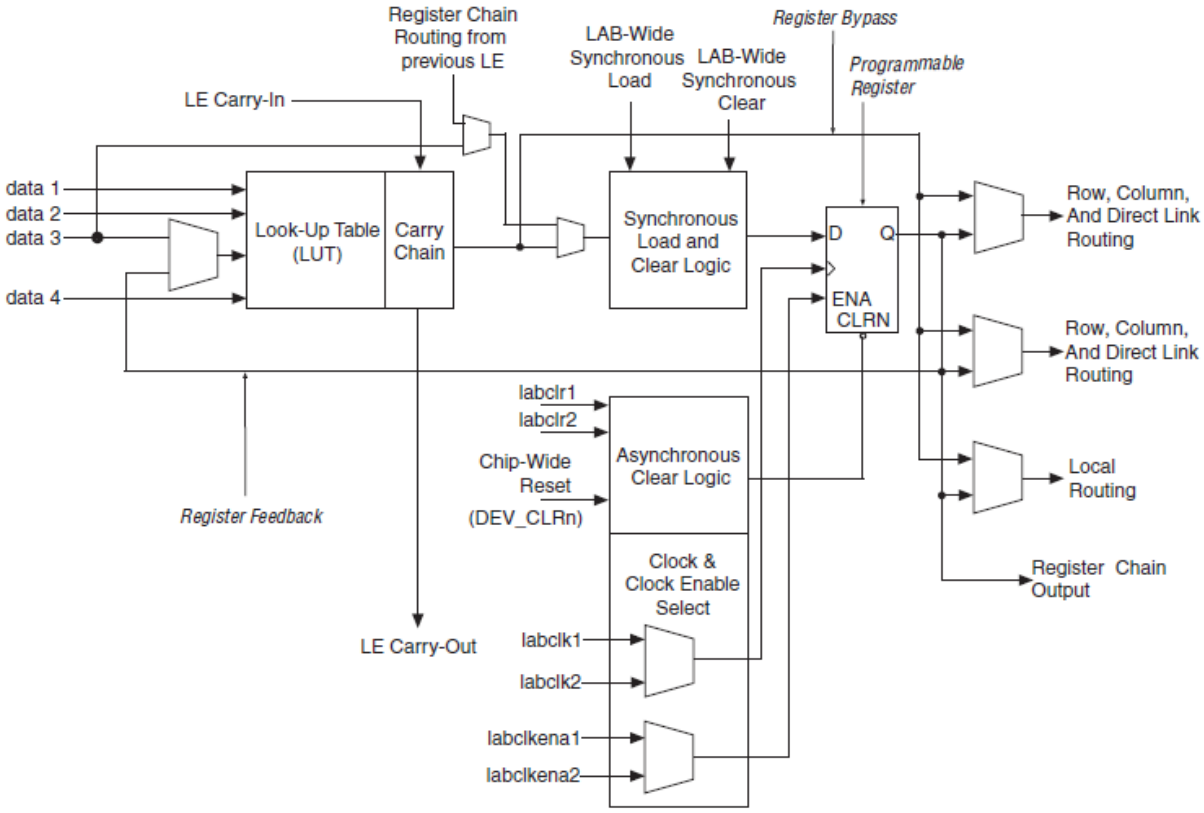


Рис. 3.1. Структура логического элемента в микросхемах Altera

Таблица 3.2

Характеристики семейства ПЛИС Cyclone V E

Характеристики	Устройства				
	5CEA2	5CEA4	5CEA5	5CEA7	5CEA9
Логические элементы (LEs)	25000	49000	77000	149500	301000
Блоки памяти M10K	176	308	446	686	1220
Умножители 18-бит x 18-бит	50	132	300	312	684
Блоки variable-precision DSP	25	66	150	156	342
Кол-во PLL	4	4	6	7	8
Контроллеры памяти	1	1	2	2	2

Среднее семейство микросхем Arria V востребовано в проектах, предназначенных к массовому производству, и в отраслях, чувствительных к стоимости конечной продукции. Среди типичных задач, которые могут

выполнять ПЛИС Arria GX – подключение к компьютеру устройств по шине PCI-Express. Другое применение – подключение промышленных контроллеров к оптическим сетям через протокол Gigabit Ethernet.

Таблица 3.3

Эволюция микросхем семейства Arria

	Arria GX	Arria II GX	Arria II GZ	Arria V GX, GT, SX	Arria V GZ	Arria 10 GX, GT, SX
Год выпуска	2007	2009	2010	2011	2012	2013
Технологический процесс, нм	90	40	40	28	28	20

Основой архитектуры ПЛИС Arria V GX является массив адаптивных логических модулей (Adaptive Logic Modules, ALM), который впервые был применен в ПЛИС семейства Stratix II, выпускаемых с 2004 года. Адаптивный логический модуль состоит из восьмивходовой таблицы перекодировок (look-up table, LUT), двух специализированных сумматоров и двух триггеров.

Входящая в состав адаптивного логического модуля восьмивходовая LUT может использоваться для реализации логических функций большого числа входных переменных, а также в режиме разделения ресурсов — для реализации двух разных логических функций меньшего числа переменных. Например, на одной LUT может быть реализована любая логическая функция шести переменных, некоторые логические функции семи переменных, и различные комбинации двух логических функций от двух до шести переменных. Такая гибкая архитектура позволяет упаковывать большее количество функций в тот же логический объем.

Каждый ALM имеет и комбинаторные, и регистровые выходы. Соответственно, появляется возможность применять LUT и триггеры ALM независимо друг от друга, что особенно удобно в приложениях, требующих большого количества регистров.

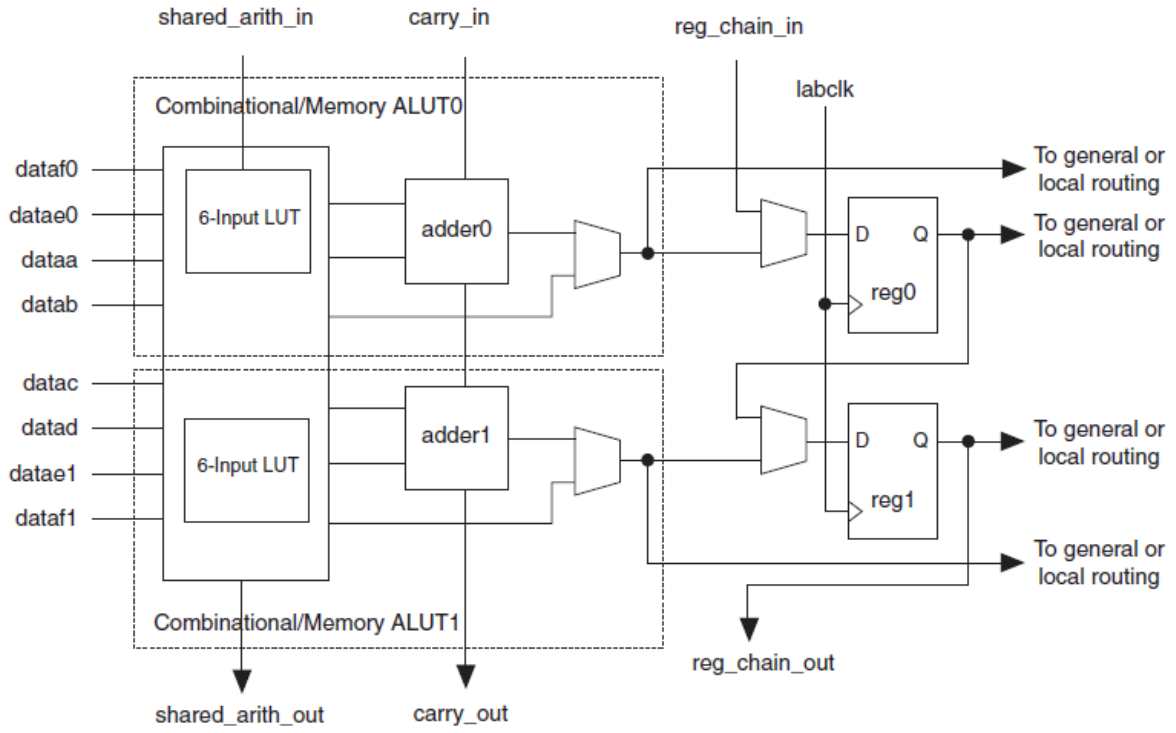


Рис. 3.2. Схема блока ALM

Таблица 3.4

Характеристики семейства ПЛИС Arria V GX

Характеристики	Устройства				
	5AGXA1	5AGXA3	5AGXB1	5AGXB5	5AGXB7
Логические элементы (LEs)	75000	156000	300000	420000	504000
Адаптивные логические модули (ALMs)	28302	58900	113208	158491	190240
Блоки памяти M10K	800	1051	1510	2054	2414
Умножители 18-бит x 18-бит	480	792	1840	2184	2312
Блоки variable-precision DSP	240	396	920	1092	1156
Трансиверы 6.5536 Gbps	9	9	24	36	36
Контроллеры памяти	1	1	2	2	2

Старшее семейство микросхем Stratix V разработано для проектов, требовательных к ресурсам ПЛИС и быстродействию. Микросхемы

подсемейства GX имеют логическую емкость до 952000 логических элементов. В микросхемах применяется технология логических блоков (LAB), состоящих из 10 адаптивных логических модулей (ALM), обеспечивающая лучшее использование логических ресурсов кристалла и сокращение путей между задействованными логическими блоками.

Таблица 3.5

Эволюция микросхем семейства Stratix

	Stratix	Stratix GX	Stratix II	Stratix II GX	Stratix III	Stratix IV	Stratix V	Stratix 10
Год выпуска	2002	2003	2004	2005	2006	2008	2010	2013
Технологический процесс, нм	130	130	90	90	65	40	28	14

Таблица 3.6

Характеристики семейства ПЛИС Stratix V GS

Характеристики	Устройства				
	5SGXA7	5SGXA9	5SGXAB	5SGXB5	5SGXBB
Логические элементы (LEs)	622000	840000	952000	490000	952000
Адаптивные логические модули (ALMs)	234720	317000	359200	185000	359200
Регистры	938880	1268000	1436800	740000	1436800
Трансиверы 14.1 Gbps	24, 36, или 48	36 или 48	36 или 48	66	66
Блоки памяти M20K	2560	2640	2640	2100	2640
Умножители 18-бит x 18-бит	512	704	704	798	704
Блоки DSP 27 x 27	256	352	352	399	352

3.1.2. Семейства ПЛИС Xilinx

Старшее семейство ПЛИС Virtex-7 представляет собой программируемые аппаратные решения последнего поколения выполненных по 28 нм технологии и предназначенные для применения как

в конечных проектах, так и для разработки и моделирования устройств, используя последние технологии в области программируемой логики. Микросхемы Virtex-7 имеют архитектуру ASMBL (расширенные кремниевые модульные блоки).

Основой архитектуры логического поля микросхем семейства Virtex-7 являются конфигурируемые логические блоки (CLBs), позволяющие реализовывать последовательностные и комбинационные схемы. Каждый CLB блок соединен с матрицей переключателей, соединяющей в свою очередь блок CLB с коммутационной матрицей (рисунок 3.3).

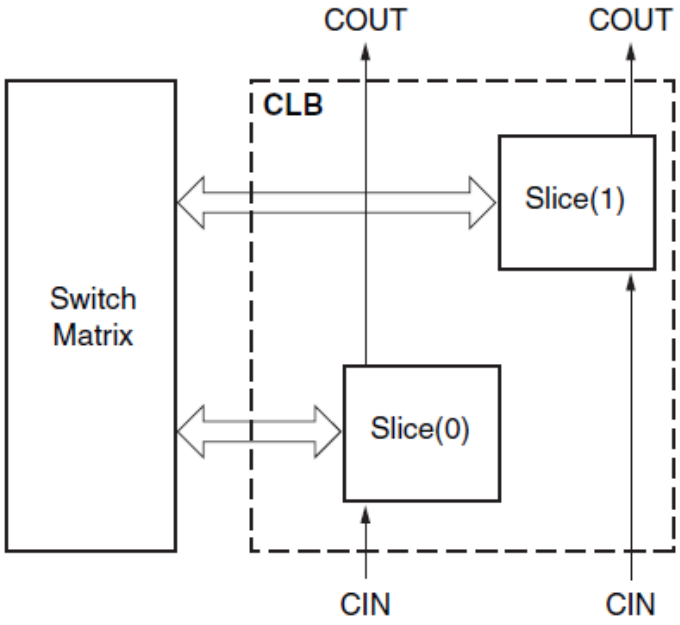


Рис. 3.3. Структура блока CLB

Обычный сегмент (SLICEL) состоит из четырех таблиц перекодировок (LUT), восьми регистров, мультиплексоров и комбинационной логики. Все эти компоненты позволяют использовать сегмент для логических и арифметических операций, а также для хранения данных. Расширенные сегменты (SLICEM), в дополнение к основным компонентам, оборудованы ячейками памяти (RAM) и сдвиговыми 32-разрядными регистрами.

Таблицы перекодировки (LUT) реализованы как 6-входовые табличные функции с двумя выходными значениями. Одна таблица

перекодировки позволяет реализовать 6-входовую логическую функцию или две 5-входовые булевые функции при условии, что у них несколько общих входов. В расширенных сегментах таблицы перекодировки могут реализовывать функции синхронной (на запись) памяти с ячейками разрядностью от 32 до 128 бит. Расширенный сегмент также может быть сконфигурирован как сдвиговый 32-разрядный регистр. При этом не задействуются основные регистры-защелки, что позволяет организовать на основе сегмента линию задержки до 32 тактов или буфер типа FIFO.

Таблица 3.7

Характеристики семейства ПЛИС компании Xilinx

Характеристики	Устройства			
	Spartan-6	Artix-7	Kintex-7	Virtex-7
Логические ячейки (Logic Cells)	150000	215000	480000	2000000
Техпроцесс, нм	45	28	28	28
Блоки RAM, Mb	4,8	13	34	68
DSP Slices	180	740	1920	3600
Количество трансиверов	8	16	32	96
Скорость трансивера, Gb/s	3,2	6,6	12,5	28,05
Интерфейс памяти, Mb/s	800	1066	1866	1866
Интерфейс PCI Express®	x1 Gen1	x4 Gen2	x8 Gen2	x8 Gen3

3.2. Реконфигурируемые сопроцессоры

Для того чтобы приступить к разработке аппаратно-программного комплекса для реализации конкретных алгоритмов на архитектуре FPGA, необходимо решить следующие задачи:

1) проанализировать ресурсы, имеющиеся в составе конкретного реконфигурируемого вычислителя;

2) разработать схемы взаимодействия приложения, выполняемого на CPU персонального компьютера, с аппаратной реализацией алгоритма, расположенной на кристалле FPGA.

В качестве реконфигурируемого вычислителя в большей части работ мы использовали отладочную плату с расположенным на ней кристаллом FPGA Arria II GX EP2AGX125. При оценке реализаций алгоритмов на Altera OpenCL SDK использовались конфигурации средства разработки для устройства Terasic DE5 с FPGA Stratix V.

Далее в данной главе рассматривается состав ресурсов Arria II GX EP2AGX125 и Terasic DE5, а затем представлены различные варианты схем систем взаимодействия CPU и FPGA (раздел 3.4), подробно рассмотренные в магистерской работе И.М. Силкина, выполнявшейся под руководством А.Е. Андреева. Под «системой взаимодействия» будут подразумеваться аппаратные реализации схем, позволяющие осуществлять передачу данных между центральным процессором хоста и кристаллом FPGA на реконфигурируемом вычислителе.

3.2.1. Анализ ресурсов FPGA Arria II GX EP2AGX125

Внешний вид реконфигурируемого вычислителя, для которого разрабатывались схемы систем взаимодействия CPU и FPGA, представлен на рисунке 3.4 [28].

Помимо кристалла FPGA Arria II GX EP2AGX125 данное устройство оснащено следующей периферией:

1) блок памяти 128-Mbyte 16-bit DDR3;

2) блок памяти 1-Gbyte 64-bit DDR2;

3) блок памяти 2-Mbyte SSRAM;

4) блок памяти 64-Mbyte flash memory;

5) 5 генератор тактовых импульсов:

а) 50 МГц;

б) 100 МГц;

в) 155.52 МГц;

г) программируемый генератор (по умолчанию 125 МГц);

д) программируемый генератор (по умолчанию 100 МГц);

6) LCD дисплей (2 линии по 16 символов);

7) PCI Express разъем стандартного размера (8.48” x 4.376”);

и другая периферия (рисунок 3.5).

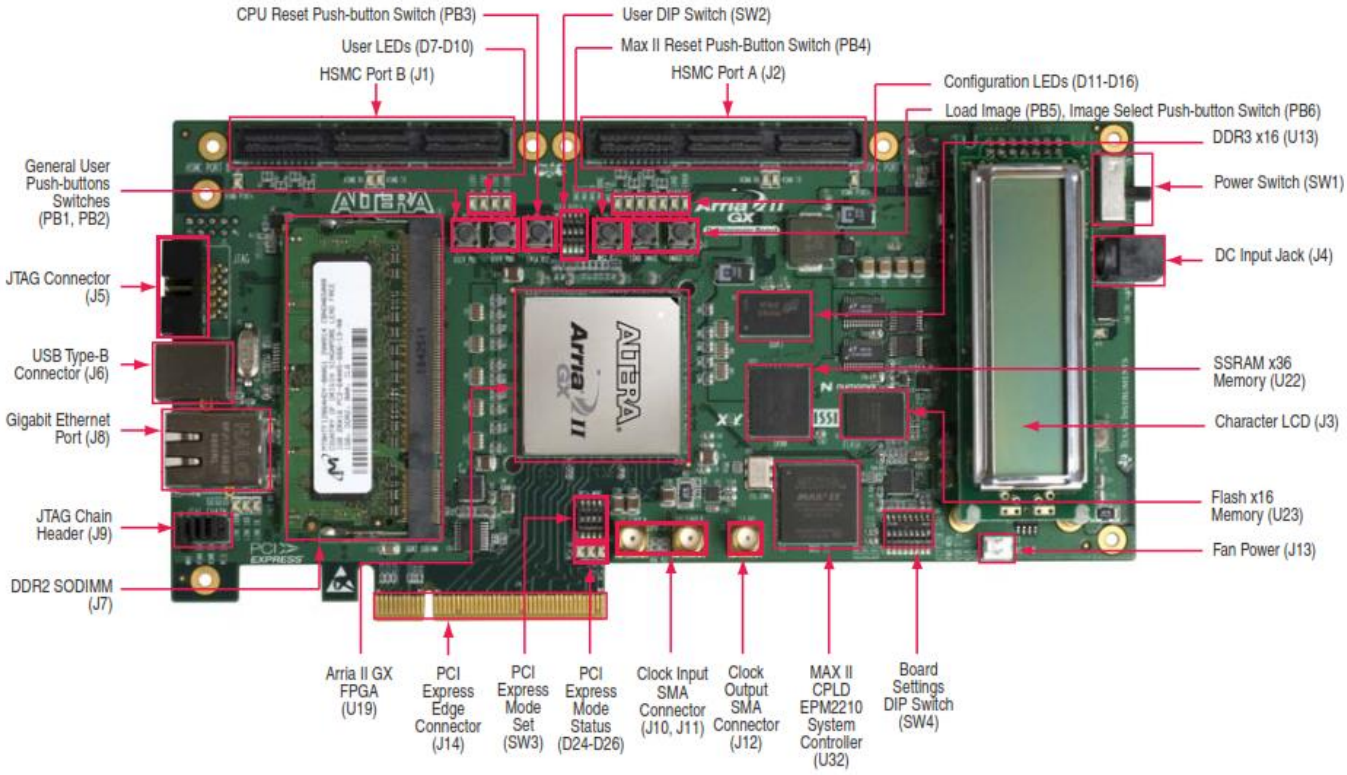


Рис. 3.4. Внешний вид используемого реконфигурируемого ускорителя

Arria II GX представляют собой недорогие FPGA с приёмопередатчиками, выполненные по технологии 40 нм. Благодаря "тонкому" техпроцессу достигается значительное снижение энергопотребления по сравнению с предыдущими семействами. Данное

семейство имеет полноценную архитектуру, включающую в себя адаптивные логические модули (ALM), блоки ЦОС (DSP blocks), встроенную память и PCI Express ядро (hard PCIe IP core). Это семейство FPGA с трансиверами оптимально сбалансировано по логике, памяти и умножителям.

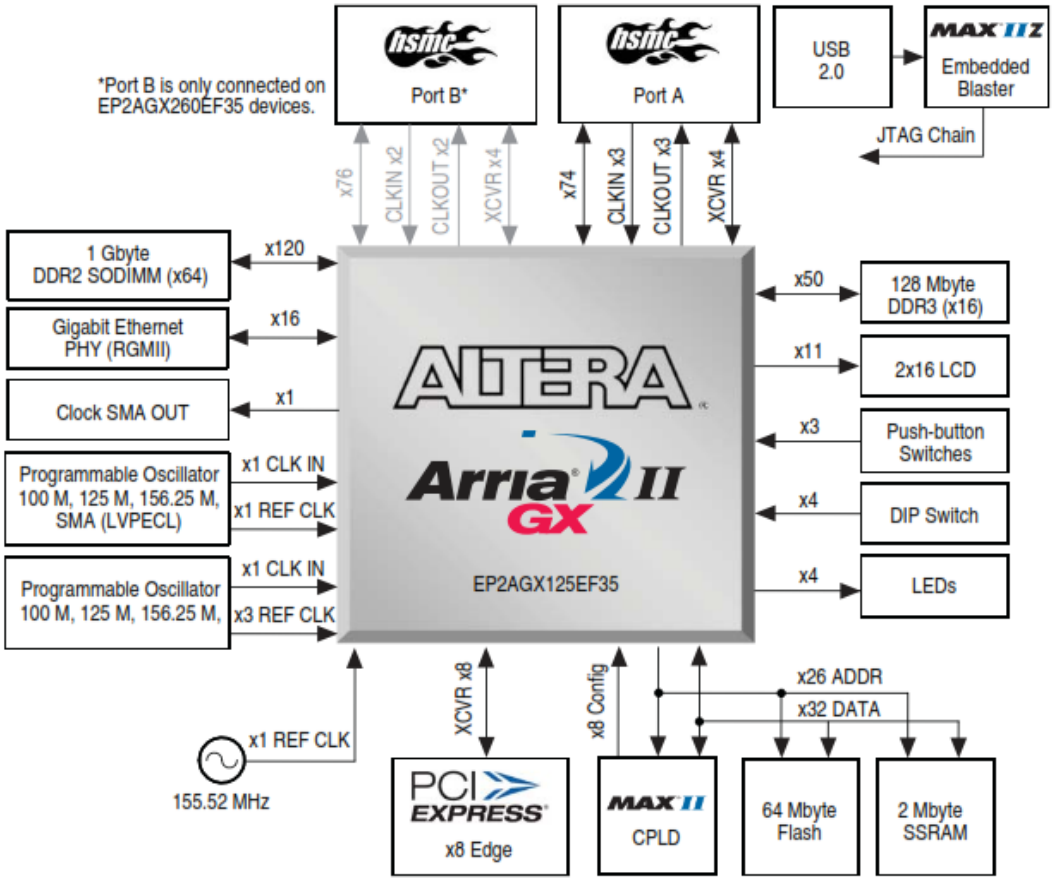


Рис. 3.5. Подключение периферии к выходам FPGA

Используя средства разработки компании Altera Quartus II и QSys, можно создавать систему на программируемом кристалле (SOPC), а благодаря наличию 3.75 Гбит/с приемопередатчиков FPGA Arria II GX могут применяться в таких областях телекоммуникации, как проводная и беспроводная связь, системы телерадиовещания, обработки и передачи контента и другие.

Основные параметры и характеристики кристалла FPGA Arria II GX EP2AGX125 приведены в таблице 3.8.

Таблица 3.8

Состав основных ресурсов Arria II GX EP2AGX125

Группа характеристик	Название характеристики	Значение
Логический объем и быстродействие	Кол-во эквивалентных логических элементов (LEs)	124,100
	Кол-во адаптивных логических модулей (ALMs)	49,640
	Кол-во блоков встроенного ОЗУ M9K RAM Blocks (8 Кбит + 512 битов четности)	730
	Объем встроенного ОЗУ M9K (Кбит)	6,570
	Общий объем ОЗУ, M9K + MLAB (Кбит)	8,121
	Кол-во встроенных умножителей 18 x 18-бит / 9 x 9-бит	576
Особенности	Поддержка синтезируемых процессорных ядер	Nios II
	Кол-во PLL / выходов PLL	6 / 42
	Встроенные средства защиты проекта от несанкционированного копирования	+
	Размер конфигурационного файла (Мбит)	140
Подсистема ввода-вывода	Поддерживаемые уровни напряжения ввода-вывода (В)	3.0, 2.5, 1.8, 1.5, 1.2
	Поддерживаемые стандарты ввода-вывода	LVDS, LVPECL, Differential SSTL-18, Differential SSTL-2, Differential HSTL, SSTL-18 (I and II), SSTL-2 (I and II), 1.5-V HSTL (I and II), 1.8-V HSTL (I and II), PCI, PCI-X 1.0, LVTTL, LVCMOS
	Кол-во встроенных аппаратных блоков PCI Express	1
Поддержка интерфейсов внешней памяти	Поддерживаемые интерфейсы внешней памяти	DDR3, DDR2, DDR, QDR II

3.2.2. Комплект разработчика Terasic DE5-Net с поддержкой Altera OpenCL SDK

Комплект разработчика Terasic DE5-Net, представленный на рисунке 3.6, построен на базе ПЛИС Stratix V EP5SGXEA7 (рассмотрены в разделе 3.1.1) и имеет следующие характеристики: поддержка PCI Express x8, четыре порта 10-Gigabit Ethernet SFP+, четыре порта Serial ATA, два слота памяти DDR3, 32 Mb QDRII+ SRAM, 256 Mb флэш [29].

Набор функций из DE5-Net в полной мере поддерживает реализацию вычислительно-интенсивных алгоритмов, приложений облачных вычислений, высокопроизводительных коммуникаций и обработки сигналов. Для данного устройства производителем (компанией Terasic) реализована поддержка современного средства разработки Altera OpenCL SDK, позволяющего разрабатывать решения для ПЛИС в виде ядер OpenCL.

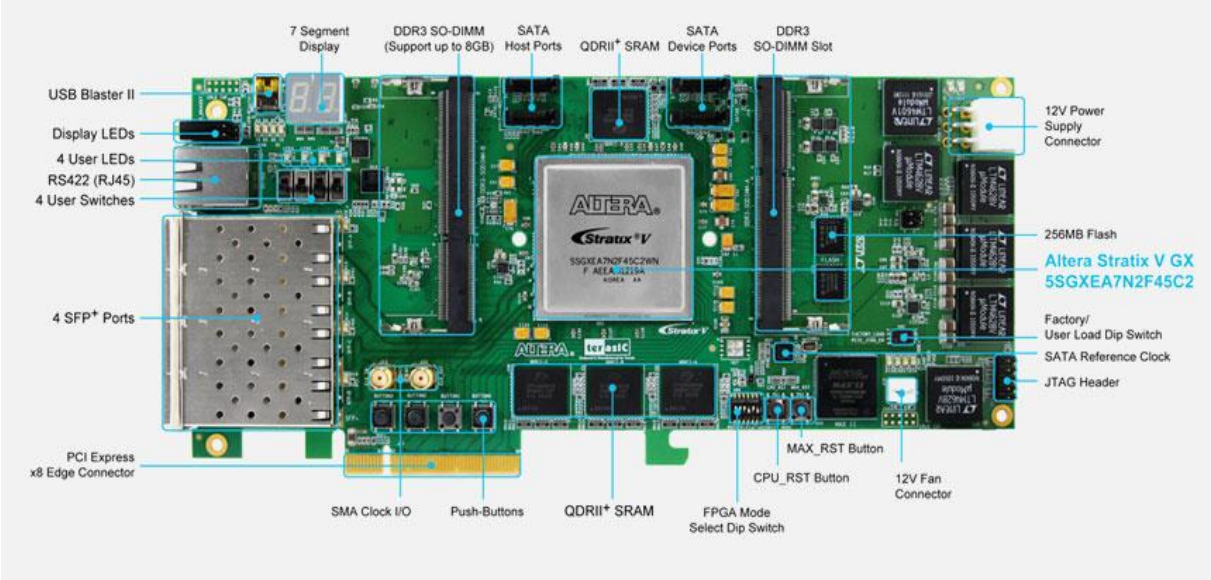


Рис. 3.6. Внешний вид платы Terasic DE5-Net Stratix V GX FPGA Development Kit

3.2.3. Интерфейс Avalon Memory Mapped

Ввиду того, что для реализации схем систем взаимодействия CPU и FPGA используется программное обеспечение Altera QSys [30], входящее в состав Quartus II, необходимо определиться с интерфейсами, с помощью которых соединяются компоненты системы. Для связи компонентов системы между собой и передачи данных между ними программное обеспечение QSys предполагает, что данные устройства реализуют интерфейс Avalon Memory Mapped. Рассмотрим особенности данного интерфейса.

Avalon - простая шинная архитектура, предназначенная для объединения устройств на кристалле в «system-on-a-programmable chip» (SOPC). Avalon - интерфейс, который определяет подключение порта шины между мастером и подчиненными компонентами, определяет синхронизацию, которой эти компоненты связываются.

Основные цели проекта шины Avalon:

1) простота - обеспечивает легкий для понимания и изучения протокол;

2) оптимизированное использование ресурса кристалла при генерации логики шины – минимизирует затраты логических элементов (LEs) внутри программируемого логического устройства (PLD).

3) синхронные операции - хорошо интегрируется с пользовательской логикой, которая сосуществует на том же самом PLD, и у пользователя не возникают проблемы с анализом синхронизации работы с процессором [31].

Основные операции шины Avalon передают блок данных (8, 16, 32, 64 бита) между мастером и подчиненным внешним устройством. Мастер шины Avalon и подчиненные взаимодействуют друг с другом в соответствии с методикой, называемой арбитражем на стороне

подчиненного устройства. Арбитраж на стороне подчиненного устройства определяет, какой из мастеров получит доступ к подчиненному, когда несколько мастеров пытаются одновременно обратиться к одному и тому же подчиненному устройству.

Основные особенности архитектуры шины Avalon:

1) До 4GB адресное пространство: память и внешние устройства могут быть отображены в пределах 32-х битного адресного пространства.

2) Синхронный интерфейс: все сигналы Avalon синхронизированы с синхрочастотой шины Avalon. Это упрощает работу шины Avalon, и облегчает интеграцию с быстродействующими внешними устройствами.

3) Встроенное декодирование адреса: шина Avalon автоматически генерирует сигналы "Выбор Устройства" для всех внешних устройств, что значительно упрощает проектирование внешних устройств.

4) Архитектура мультимастер: несколько внешних устройств типа мастер может постоянно находиться на шине Avalon. Шина Avalon автоматически генерирует логику арбитража.

5) Конфигурация при помощи Wizard'a: удобные в работе графические Wizard'ы помогают пользователю произвести конфигурацию шины Avalon (добавление внешних устройств, определение мастер / подчиненные, определение карты памяти). Архитектура шины Avalon будет сгенерирована автоматически на основании данных, введенных пользователем при работе Wizard'a.

6) Динамическое установление размеров шины: шина Avalon автоматически обрабатывает те случаи передачи данных между внешними устройствами, когда разрядность шины данных внешнего устройства не соответствует разрядности шины процессора. [31]

Подробное описание операций передачи данных подчиненного устройства и мастера данных приведено в спецификации [31].

При разработке собственных вычислительных блоков, реализующих конкретный алгоритм, необходимо иметь в виду, что данные блоки должны поддерживать как минимум интерфейс Avalon Slave. Иначе разработанное устройство невозможно будет добавить в систему QSys.

3.3. Общая архитектура системы

Общую архитектуру гибридной системы с реконфигурируемым сопроцессором можно представить, как показано на рисунке 3.7.

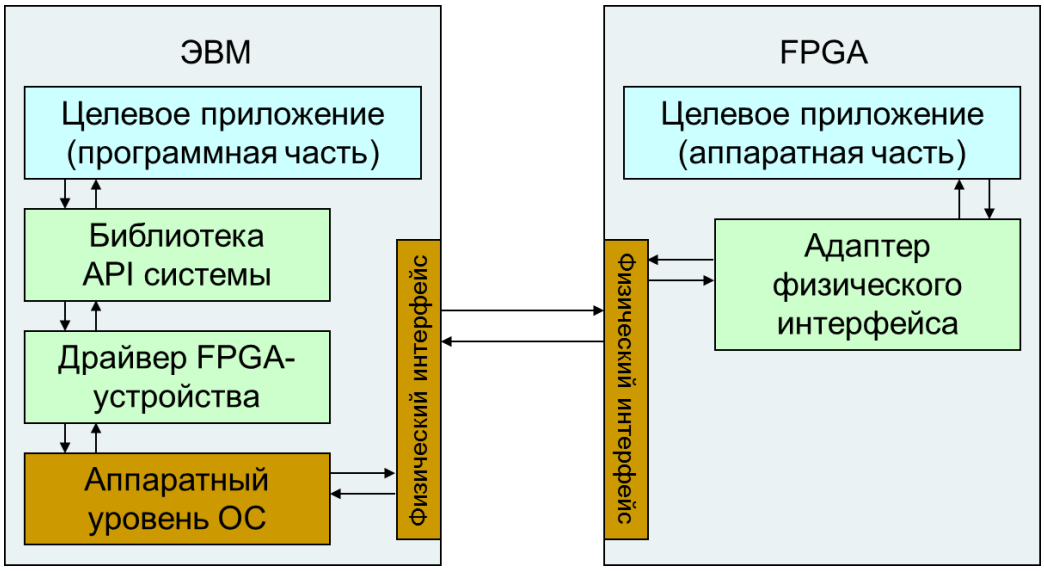


Рис. 3.7. Общая архитектура гибридной системы с ПЛИС – сопроцессором

На рисунке 3.7 при этом в достаточно общем виде представлена структура аппаратной части системы, функции которой, на самом деле, не сводятся только к функции адаптера физического интерфейса и целевого приложения (в нашем случае как правило – PCI-E). В ее задачи может входить управление памятью сопроцессора, реализация режима прямого доступа к памяти (ПДП-DMA), а особенности ее реализации определятся используемой аппаратной платформой и предлагаемыми для нее типовыми решениями, что описано подробнее далее в разделе 3.4.

3.4. Типовые структуры конфигурации сопроцессора для различных типов задач

3.4.1. Критерии для задач, решаемых с помощью реконфигурируемого сопроцессора

Одним из наиболее важных вопросов при использовании FPGA в качестве сопроцессора является реализации схем систем взаимодействия центрального процессора и блоков, реализующих вычисления на FPGA. Помимо блоков, осуществляющих непосредственные вычисления, схемы систем взаимодействия должны включать в себя дополнительную периферию для передачи данных, для хранения исходных данных и результата, для управления вычислительными блоками. В зависимости от определенных особенностей реализуемых алгоритмов системе требуются ресурсы разного типа. Исходя из этого, выделены следующие наиболее значимые при разработке данных систем критерии для решаемых задач:

1) Объем пересылаемых данных между FPGA и CPU:

а) обработка массивов данных (например, пример, блоки в криптографических алгоритмах);

б) многократные вычисления над небольшим объемом данных (пример, задача восстановления строки по md5-хэшу).

2) Наличие сложной логики, управляющей процессом решения задачи (например, QR-разложение).

Для алгоритмов, производящих многократные вычисления над небольшим объемом данных, в состав системы взаимодействия необходимо включить, как минимум, контроллер PCI Express.

Для алгоритмов, осуществляющих обработку массивов данных, в состав системы взаимодействия потребуется добавить устройства памяти для хранения входных, обрабатываемых данных и результата вычисления. Кроме того, для повышения производительности обмена данными между

FPGA и CPU логично использовать устройство Scatter-Gather Direct Memory Access (SGDMA), которое позволяет перемещать буферы памяти, заданного размера, из одного местоположения (области памяти) в другое.

При реализации алгоритмов, имеющих сложную управляющую логику, следует использовать систему, в состав которой включен встраиваемый в FPGA процессор Nios II. Это позволит упростить и ускорить создание управляющей логики, путем написания программы на языке высоком уровне, которая будет выполняться на Nios II и контролировать процесс непосредственных вычислений.

3.4.2. Реализация схемы взаимодействия для задач, осуществляющих многократные вычисления над небольшим объемом данных

Аппаратные схемы систем взаимодействия реализуются с помощью средства разработки Altera Qsys (версия 11.1). Altera Qsys представляет собой программное обеспечение для автоматического проектирования различных программно-аппаратных средств компьютерной техники и представляющее собой различные HDL модули: контроллеры памяти, интерфейсы и периферийное оборудование.

Итак, первый вариант схемы предназначен для задач, не требующих пересылки больших объемов данных (например, задача восстановления строки по MD5-хэшу). Предполагаются многократные вычисления над небольшим объемом данных. Данная система может быть использована для тестирования схем устройств. Разработанная система позволяет осуществлять контроль над вычислительными блоками путем записи/чтения управляющих регистров устройств. Основные блоки, входящие в состав данного варианта системы, представлены на рисунке 3.8.

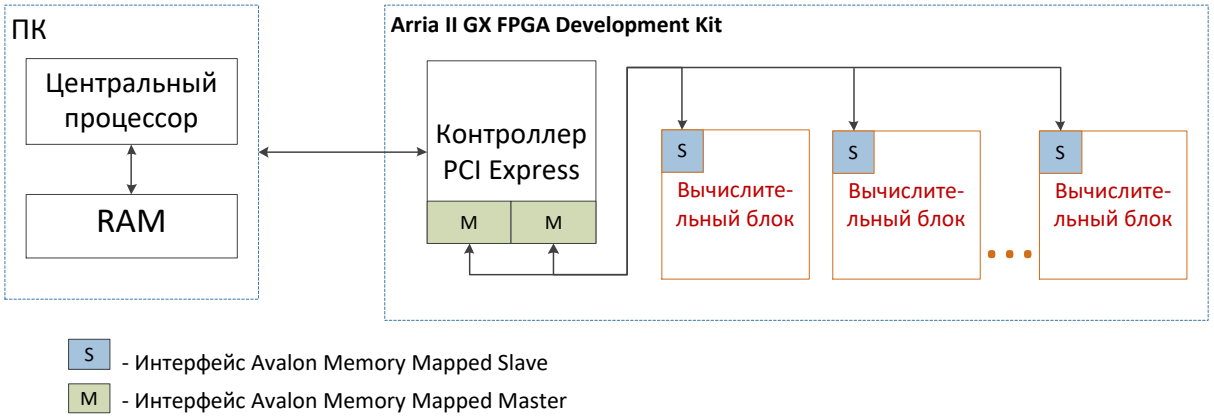


Рис. 3.8. Структура схемы взаимодействия для задач, осуществляющих многократные вычисления над небольшим объемом данных

В данной системе предусмотрено наличие только PCI Express-контроллера и блоков, осуществляющих вычисления согласно реализуемому алгоритму. Приложение, выполняемое на CPU, имеет доступ к управляющим регистрам устройства, к регистрам данных через PCI Express-контроллер. При этом данные могут иметь размер 8, 16, 32 или 64 бита. Экспериментально полученные значения скорости передач чтения/записи данных 64-битной разрядности из CPU в регистры вычислительного блока представлены в таблице 3.9. Как видно из данных таблицы, скорости чтения и записи регистров устройств в данном случае не зависят от объемов пересылаемых данных. Для чтения среднее значение пропускной способности составляет 6,7 Мб/с, для записи – 208 Мб/с.

Объемы ресурсов, требующихся для реализации данной системы, представлены в таблице 3.10. Данные значения получены средствами анализа САПР Quartus II. Из представленной таблицы видно, что для реализации данной системы потребуется 5% всех ресурсов рассматриваемой FPGA.

Таблица 3.9

Значения скорости передачи данных в регистры вычислительного блока

Направление передачи	Объем пересылаемых данных, Кб	Экспериментально полученное значение пропускной способности, Мб/с					
		1	2	3	4	5	среднее
чтение	2	6,71	6,79	6,77	6,79	6,78	6,77
	8	6,75	6,72	6,80	6,69	6,78	6,74
	32	6,68	6,75	6,79	6,77	6,74	6,75
	64	6,79	6,72	6,75	6,79	6,78	6,77
запись	2	213,25	202,53	203,17	208,56	210,10	207,52
	8	210,70	208,13	211,32	202,23	206,75	207,84
	32	212,00	212,67	207,89	208,96	205,15	209,33
	64	212,33	212,86	210,53	211,32	209,36	211,28

Таблица 3.10

Объем ресурсов, требуемый для реализации системы взаимодействия

Тип ресурса	Количество
Combinational ALUTs	3,145 / 99,280 (3 %)
Memory ALUTs	27 / 49,640 (< 1 %)
Dedicated logic registers	4,224 / 99,280 (4 %)
Logic utilizations	5%

Компоненты в системах QSys соединяются между собой с помощью интерфейсов Altera Avalon [32]. Разрабатываемые вычислительные блоки, реализующие выбранный алгоритм, также должны придерживаться спецификации интерфейсов Avalon Memory Mapped. В данном варианте системы для добавления вычислительного блока необходимо, чтобы он имел интерфейс Avalon Memory Mapped Slave, описанный в разделе 3.2.3.

В качестве примера задачи, при реализации которой можно использовать данный вариант системы взаимодействия, можно привести задачу восстановления строки по md5-хэшу.

3.4.3. Реализация схемы взаимодействия для задач, обрабатывающих массивы данных

Рассмотрим вариант схемы взаимодействия, предназначенный для задач, обрабатывающих массивы данных.

Разработанная структура схемы взаимодействия позволяет передавать большие объемы данных в режиме прямого доступа как направлении из CPU на FPGA, так и в обратном направлении. Как и в первом варианте структуры, представленной в пункте 3.4.2, данная реализация предусматривает возможность осуществления контроля над вычислительными блоками путем записи/чтения управляющих регистров устройств. Помимо PCI Express контроллера в состав данной системы включены устройство SGDMA, позволяющее осуществлять пересылки данных в режиме прямого доступа к памяти. Прямой доступ к памяти (Direct Memory Access, DMA) — режим обмена данными между устройствами или же между устройством и основной памятью (RAM) без участия центрального процессора (CPU). В результате скорость передачи увеличивается, так как данные не пересылаются в центральный процессор и обратно. Кроме того, данные пересылаются сразу для многих слов, расположенных по подряд идущим адресам, что позволяет использование «пакетного» режима работы шины — 1 цикл адреса и следующие за ним многочисленные циклы данных. Также в систему добавлены контроллеры памяти DDR3, DDR2 и блок памяти на чипе (on-chip memory). Данные устройства позволяют использовать разные виды памяти для хранения входных и обрабатываемых данных, а также результатов вычисления.

При выборе данного варианта системы необходимо учитывать, что в вычислительном блоке, реализующем алгоритм, помимо интерфейса Avalon Memory Mapped Slave, должен быть реализован интерфейс Avalon Memory Mapped Slave. Данный интерфейс необходим для подключения вычислительного блока к устройствам памяти. Описание данного интерфейса приведено в [31].

Таблица 3.11

Объем ресурсов, требующихся для реализации схемы взаимодействия

Тип ресурса	Количество
Combinational ALUTs	7,921 / 99,280 (8 %)
Memory ALUTs	65 / 49,640 (< 1 %)
Dedicated logic registers	11,633 / 99,280 (12 %)
Logic utilization	14 %

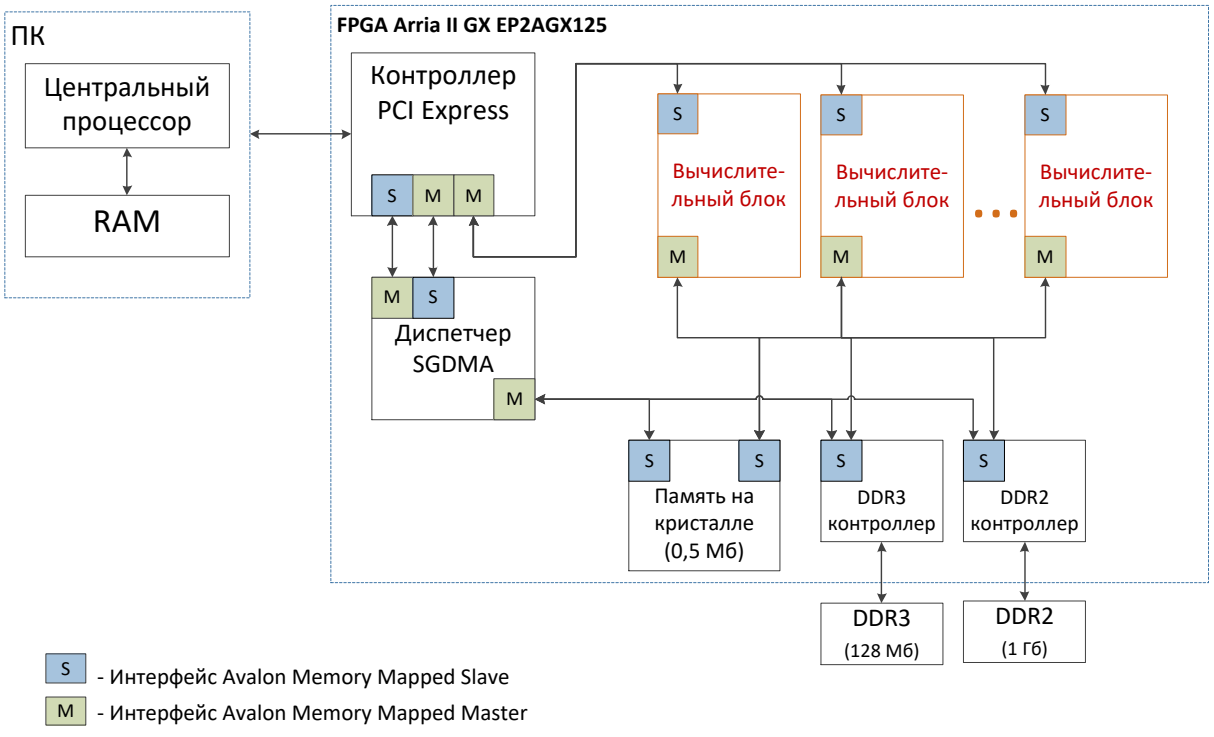


Рис. 3.9. Структура схемы взаимодействия для алгоритмов, предусматривающих обработку массивов данных

Проанализируем средствами Quartus II объем ресурсов, необходимых, для реализации данной системы. Результаты анализа необходимых ресурсов представлены в таблице 3.11. Из представленной таблицы видно, что для реализации данной системы потребуется 14% ресурсов рассматриваемой FPGA.

Пропускная способность передачи данных через интерфейс PCI Express в режиме DMA зависит от размеров пересылаемых блоков. Экспериментальные результаты замеров показателей скорости передачи данных на диапазоне размеров блоков представлены в таблице 3.12.

Таблица 3.12

Экспериментально полученные результаты пропускной способности PCI Express в режиме DMA

Направление передачи	Объем пересылаемых данных, Кб	Экспериментально полученное значение пропускной способности, Мб/с					
		1	2	3	4	5	среднее
чтение	2	266.67	284.44	272.69	275.63	268.76	273.63
	8	533.34	537.52	535.36	532.62	534.81	534.70
	32	682.67	685.38	675.91	682.98	678.63	681.11
	64	714.84	711.12	711.89	716.53	713.25	713.52
	256	749.85	741.03	739.02	743.65	748.62	744.43
запись	2	328.20	304.76	323.21	321.63	325.52	320.66
	8	588.51	568.89	580.32	578.36	577.62	578.74
	32	718.60	726.24	725.23	723.21	726.83	724.02
	64	750.19	751.32	748.53	748.96	749.36	749.67
	256	772.47	768.32	774.56	774.10	770.55	772.00

Анализ экспериментальных данных таблицы позволяет сделать вывод, что скорость передачи данных с использованием режима DMA зависит от объема пересылаемого блока. Также можно отметить, что с

точки зрения повышения производительности пересылок данных между CPU и FPGA следует передавать информационные блоки как можно большего размера.

В качестве примеров задач, при реализации которых на архитектуре FPGA рекомендуется использовать данную схему, можно привести криптографический алгоритм AES, криптографический алгоритм ГОСТ 28147-89.

3.4.4. Реализация схемы взаимодействия для задач со сложной управляющей логикой

Зачастую при разработке аппаратных схем для архитектуры FPGA возникают трудности при реализации сложной логики управления вычислениями на языках аппаратного описания. Данную проблему можно решить, используя в своих системах встраиваемый в кристалл FPGA процессор общего назначения Nios II [33]. Данный процессор реализуется непосредственно с помощью логических элементов в самом кристалле FPGA. Сложная логика управления процессом решения задачи разрабатывается с помощью языка высокого уровня и выполняется на встраиваемом процессоре.

Подробнее рассмотрим, что собой представляет процессор Nios II. Семейство программных процессоров Nios II - это второе поколение встраиваемых процессоров общего назначения с перестраиваемой конфигурацией. Эти процессоры используются в FPGA компании Altera. Системы на базе Nios II позволяют программно создавать и использовать процессоры, соответствующие потребностям встраиваемой системы. Встраиваемое ядро процессора Nios - конвейерный RISC процессор, в котором команды выполняются за один цикл частоты синхронизации. На рисунке 3.10 представлена обобщенная схема ядра процессора Nios II.

Выделим наиболее значимые особенности процессора Nios II:

1) загружаемая RISC архитектура, с перестраиваемой конфигурацией;

2) полностью синхронный адрес и интерфейс шины данных;

3) разрядность данных 16 или 32 бита;

3) адресное пространство 128 Кб и 8 Гб, соответственно;

4) 16-битная система команд;

5) малые требования к памяти;

6) совместимость со стандартными FLASH устройствами;

7) поддержка памяти на кристалле и внешней памяти;

8) архитектура конвейера с 5 стадиями;

9) одна команда выполняется за один цикл частоты;

10) до 512 регистров общего назначения.

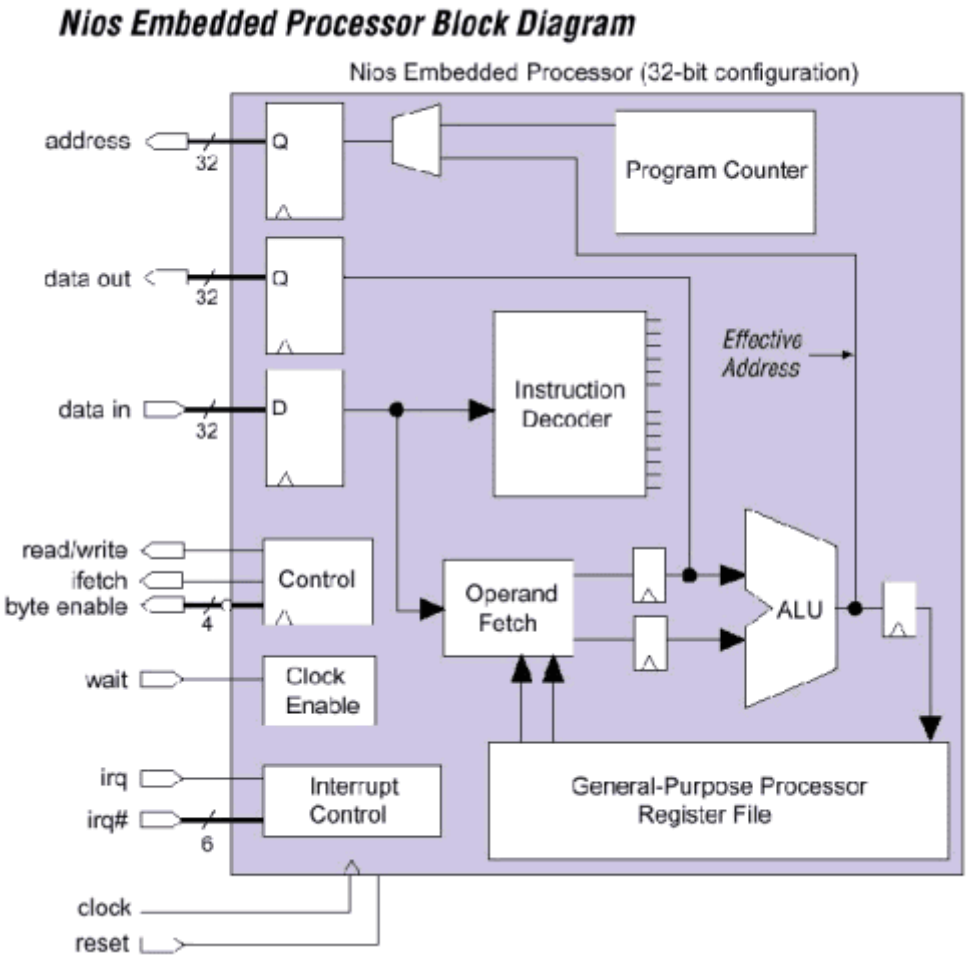


Рис. 3.10. Ядро процессора Nios II

Структура реализованной схемы системы взаимодействия представлена на рисунке 3.11. Кроме встраиваемого процессора, в архитектуру добавилось несколько устройств периферии, необходимой для работы с Nios II, а именно: устройство памяти программ, реализованное на кристалле, интервальный таймер и контроллер JTAG UART, необходимый для отладки программ.

Проанализируем средствами Quartus II объем ресурсов, необходимых, для реализации данной системы. Результаты анализа необходимых ресурсов представлены в таблице 3.13. Из представленной таблицы видно, что для реализации данной системы потребуется 18% ресурсов рассматриваемой FPGA.

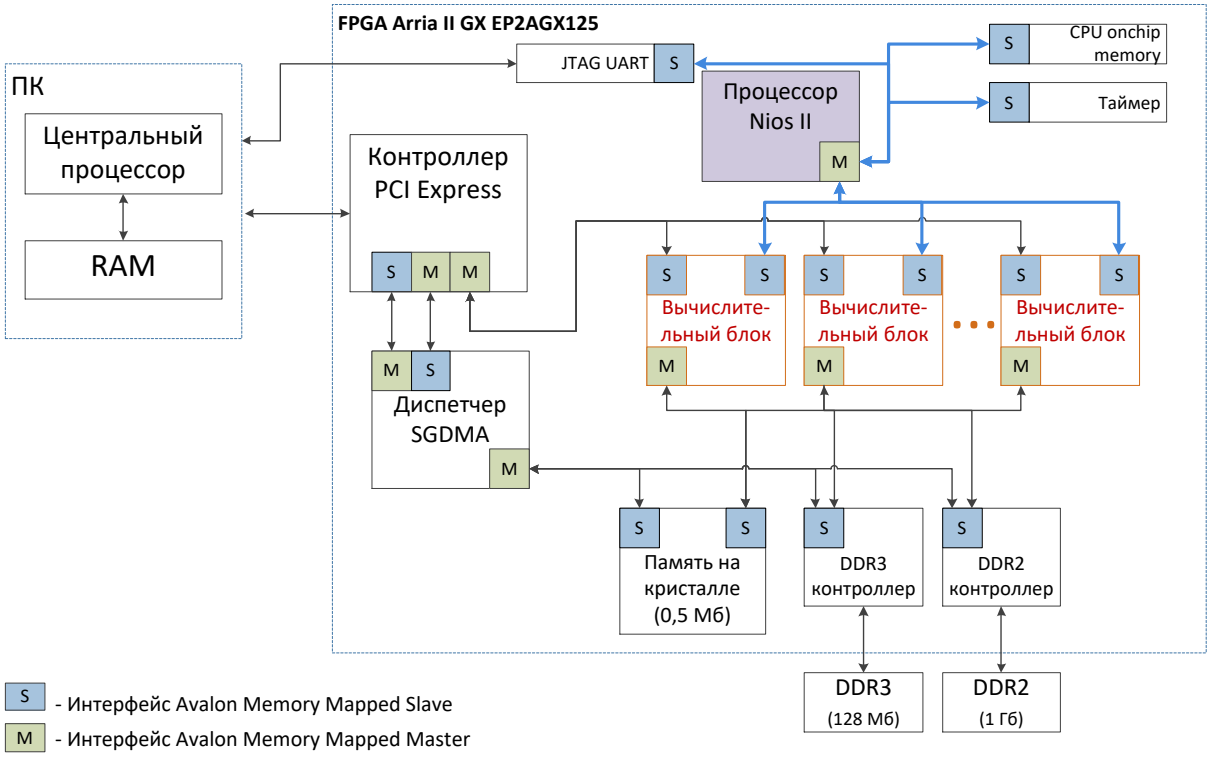


Рис. 3.11. Структура схемы взаимодействия для задач со сложной управляющей логикой

Таблица 3.13

Объем ресурсов, требующихся для реализации схемы взаимодействия

Тип ресурса	Количество
Combinational ALUTs	10,362 / 99,280 (10 %)
Memory ALUTs	65 / 49,640 (< 1 %)
Dedicated logic registers	15,397 / 99,280 (16 %)
Logic utilization	18 %

3.5 Системное программное обеспечение сопроцессора

К системному программному обеспечению сопроцессора в соответствии с рисунком 3.6 относится как минимум драйвер устройства, а также, возможно, библиотека доступа, предоставляющая прикладным программам более удобный прикладной интерфейс API.

В основном при работе с устройствами, описанными в пункте 3.2, нами использовался драйвер, разработанный с помощью коммерческого средства разработки и библиотеки WinDriver. Драйвер для платы DE-5, предоставленный производителем (Terasic), также построен на базе WinDriver. Для отказа от проприетарного средства разработки WinDriver в работе студента Трофимова В.В. под руководством Андреева А.Е. была предпринята попытка создания своего собственного драйвера для платы Arria II GX средствами WDK8. В результате был создан полностью функциональный драйвер, поддерживающий режим программного обмена с устройством. Основным недостатком драйвера является отсутствие поддержки режима ПДП и низкая производительность. В настоящее время ведется работа по совершенствованию драйвера.

Также в рамках данной работы была разработана небольшая объектно-ориентированная библиотека API для работы с драйвером, диаграмма классов которой приведена на рисунке 3.12.

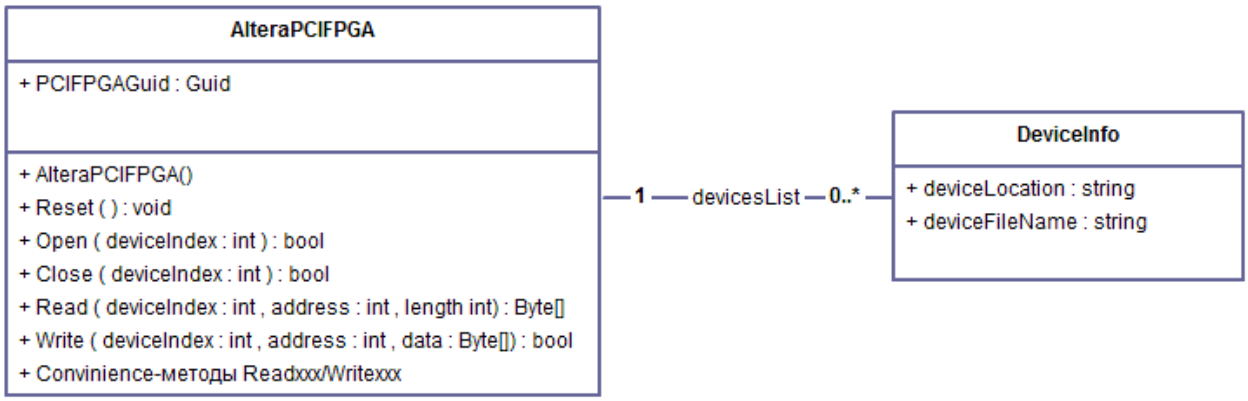


Рис. 3.12 – Диаграмма классов библиотеки API для работы с драйвером

3.6. Выводы

В данной главе рассмотрено аппаратное обеспечение реконфигурируемых сопроцессоров, включая семейства ПЛИС, две платы из комплектов разработчика Arria II GX и DE5-Net, общая архитектура ГВС РС, особенности используемого системного программного обеспечение. Основное внимание уделено вопросам реализации схем взаимодействия центрального процессора персонального компьютера и вычислительного блока, расположенного внутри кристалла FPGA. Представлено описание особенностей используемого интерфейса Avalon Memory Mapped.

При разработке архитектуры схем взаимодействия центрального процессора и FPGA необходимо учитывать критерии алгоритмов, для реализации которых планируется использовать систему. К таким критериям относится объем пересылаемых данных и сложность управляющей логики.

При реализации схемы взаимодействия для решения задач, осуществляющих многократные вычисления над небольшим объемом данных, не следует перегружать систему дополнительной периферией. Схема взаимодействия в этом случае может содержать лишь контроллер PCI Express, подсоединенный непосредственно к вычислительным блокам.

При реализации схемы взаимодействия для решения задач, обрабатывающих массивы данных, необходимо учесть в ее составе устройства памяти для хранения входных, обрабатываемых данных и результата. Приведенная в данной главе архитектура включает в себя контроллеры для работы с внешней памятью DDR3 и DDR2. Кроме того, для повышения производительности операций обмена данными в состав системы добавлено устройство SGDMA, позволяющее осуществлять пересылки данных в режиме прямого доступа к памяти.

При реализации схемы взаимодействия для решения задач со сложной управляющей логикой предлагается использовать встраиваемый процессор. Рассмотренная в данной главе система имеет в своем составе встраиваемый в FPGA процессор Nios II и необходимую для работы с ним периферию. Предполагается, что сложно реализуемая управляющая логика будет описываться с помощью языка высокого уровня и выполняться на процессоре общего назначения Nios II (возможно, даже под управлением встраиваемой операционной системы).

Кроме того, прежде чем использовать тот или иной вариант схемы, необходимо проанализировать требуемые для его построения ресурсы. Чем больше дополнительных периферийных устройств включает в себя система, тем больше требуется ресурсов для ее реализации, и, соответственно, тем меньше ёмкости ПЛИС остается для размещения блоков, занимающихся непосредственно вычислениями.

Глава 4

СРЕДСТВА РАЗРАБОТКИ РЕШЕНИЙ ДЛЯ ГИБРИДНЫХ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ С РЕКОНФИГУРИРУЕМЫМИ СОПРОЦЕССОРАМИ

4.1 Состав средств разработки

Разработка решений для систем с реконфигурируемыми сопроцессорами в соответствии со структурой таких систем и самих решений предполагает использование целого комплекса средств. Как минимум, в состав такого комплекса входят средства создания хост приложений, выполняемых на ЦП, и средства создания аппаратной части решения, размещаемой в ПЛИС реконфигурируемого сопроцессора. Поскольку хост приложение, как правило, работает под управлением операционной системы, в состав программных средств решения входят драйверы устройств для работы с сопроцессором, а комплекс средств разработки может включать средства создания драйверов.

На практике в состав программных средств могут входить дополнительные системные программные средства, такие, например, как промежуточные API, предоставляющие программный интерфейс более высокого уровня по отношению к драйверу для прикладных программ, работающих с сопроцессором.

Средства разработки хост приложений чаще всего традиционны и представляют собой обычные компиляторы, трансляторы, библиотеки, интегрированные среды разработки с поддержкой выбранного языка высокого уровня и необходимых технологий программирования.

Поскольку наиболее трудоемкой частью разработки, в значительной мере определяющей эффективность всего решения, является именно разработка аппаратных, схемных конфигураций, размещаемых в ПЛИС,

основной интерес представляют именно компоненты комплекса средств разработки, отвечающие за создание этих схемных решений. Кроме того, они могут сильно влиять на состав всего комплекса средств разработки. В следующем пункте рассмотрим основные подходы к разработке схемных решений и состав необходимых программных средств.

4.2 Подходы к разработке схемных решений и состав необходимых программных средств

Схемные решения для РС строятся либо непосредственно путем создания схем с помощью графических редакторов САПР (довольно редко), либо, чаще всего, с помощью языков описания аппаратуры (Hardware Descriptive Languages–HDL), либо – с использованием средств высокоуровневого проектирования (High-Level Synthesis – HLS). В последнем случае используется либо непосредственно язык высокого уровня (ЯВУ, High Level Language – HLL), либо специализированное расширение такого языка, либо – технология программирования, поддерживающая язык высокого уровня и адаптированная для генерации схемных решений. Кроме того, схемные решения для РС обычно встраиваются в какое-то стандартное аппаратное окружение, включающее локальную память, систему контроллеров и адаптеров, предназначенное для управления памятью, аппаратным интерфейсом платы сопроцессора и взаимодействием между сопроцессором и хостом (подобное описанному в предыдущей главе).

Поскольку аппаратное обеспечение РС строится на базе ПЛИС, средства разработки схемных решений строятся на базе соответствующих САПР ПЛИС. Такие САПР (например, Altera Quartus II или Xilinx ISE) поддерживают создание схемных решений непосредственно с помощью своих графических редакторов, генераторов IP ядер, на HDL с помощью

HDL-редакторов и трансляторов, а также, как правило, обеспечивают стандартное системное окружение для схемных решений с помощью готовых блоков подсистем и средств интеграции (например, средствами Qsys в случае Altera). Большинство САПР ПЛИС поддерживают стандартные языки описания аппаратуры VHDL и Verilog, а также собственные (например, AHDL для Altera).

Несмотря на то, что разработка схемных решений для РС средствами HDL с использованием САПР ПЛИС и предоставляемых ими средств интеграции позволяет в принципе решить любую возникающую задачу и является стандартом для реконфигурируемых систем, такой подход к разработке имеет как минимум три существенных недостатка. Во-первых, он требует хорошего знания выбранного HDL, во-вторых, он требует достаточно длительного времени, в-третьих, он требует от разработчика знаний в области цифровой схемотехники. В конечном итоге эти три недостатка можно свести к одному – значительные затраты времени при разработке. Это ограничивает количество создаваемых решений для РС и мешает такому же широкому применению РС на базе ПЛИС, которое находят, например, графические ускорители (GPU). Для использования последних также необходимы дополнительные знания – нужно изучить технологию CUDA или OpenCL, но это, как правило, требует меньшего времени и данные технологии лучше воспринимаются прикладными программистами.

В результате все больший интерес программистов вызывают различные средства высокоуровневого проектирования для FPGA (HLS for FPGA).

Рассмотрим некоторые из них в следующем разделе.

4.3 Обзор и сравнительный анализ средств высокоуровневого проектирования для ПЛИС

4.3.1 Обзор сравниваемых средств высокоуровневого проектирования

Высокоуровневые средства разработки для FPGA можно классифицировать по используемому языку программирования, по ориентации на конкретное аппаратное обеспечение сопроцессора. Кроме того, можно выделить свободно распространяемые и коммерческие средства.

Для выбора средства HLS на начальном этапе нами были рассмотрены следующие средства высокоуровневого проектирования: C-to-Verilog, Impulse-C, LegUp, Mitrion Platform, Altera OpenCL SDK, Автокод-HDL [34, 35].

Веб-компилятор C-to-Verilog работает с программами, написанными на языке C, накладывая на них ряд ограничений: требуется отсутствие рекурсивных функций, указателей на функции, структур и вызовов библиотечных функций (printf, malloc и других), операций с плавающей запятой. При использовании нескольких функций их необходимо объявлять как static inline. Поддерживаются только одномерные массивы. Использовать в функции можно только те массивы, которые были переданы в качестве параметров функции. Кроме генерации кода функции на Verilog, компилятор создаёт соответствующее окружение для тестирования, которое авторы проекта предлагают провести в коммерческих (ModelSim) или бесплатных (Icarus verilog simulator) средствах.

C-to-Verilog производит оптимизацию с помощью распараллеливания и конвейеризации. Кроме того, возможна оптимизация

под архитектуру нескольких модельных рядов FPGA от Xilinx и Altera. Разработчики позволяют использовать их компилятор и локально, однако это возможно только на платформах Linux [36].

Среда разработки Impulse CoDeveloper компании Impulse Accelerated Technologies включает в себя компилятор Impulse C software-to-hardware, интерактивный параллельный оптимизатор, пакеты Platform Support Packages, обеспечивающие поддержку широкого набора систем основанных на FPGA. Impulse C позволяет описывать приложения и алгоритмы с глубокой степенью параллелизма, при этом не являясь новым языком программирования. Impulse C расширяет стандарт ANSI C за счет дополнительных библиотек функций. Среда CoDeveloper предоставляется в нескольких вариантах: для FPGA компании Xilinx и компании Altera. Разработка приложений с использованием Impulse C производится в рамках программной модели CSP (communicating sequential processes). Основу данной программной модели составляют процессы и потоки (processes and streams). Процессы независимо синхронизированы и параллельно выполняют разные части приложения. Передача данных между процессами производится через механизм потоков, сигналов или с помощью общей памяти. При описании части алгоритма, реализуемой аппаратно, на базовый язык C накладывается ряд ограничений. Например, запрет на использование рекурсии, сильно ограниченный набор поддерживаемых функций и структур. Контроль над генерируемым аппаратным описанием реализуется с помощью директив компилятора, в частности, существуют директивы для построения конвейера, для управления задержкой ступеней конвейера и другие. Главным недостатком проекта является его высокая стоимость, но имеется возможность получить временную лицензию для ознакомления с продуктом, для использования в научных исследованиях и для изучения [32, 37].

LegUp представляет собой свободный каркас ПО с открытым исходном кодом для синтеза схемных решений для FPGA из программ на языке C, разрабатываемый группой ученых из университета Торонто [38]. Последняя на момент написания данной работы версия каркаса 3.0 обеспечивает частичную поддержку ANSI C, включая многомерные массивы, структуры, адресную арифметику и арифметику с плавающей точкой. Динамическое выделение памяти и рекурсия не поддерживаются.

LegUp предоставляет транслятор байт-кода LLVM (low level virtual machine) в Verilog и использует clang для компиляции кода на C в байт-код LLVM.

Необходимо также отметить, что каркас работает только под Linux, не имеет графического интерфейса и поддерживает только семейства FPGA Cyclone II и Stratix IV.

Наибольший интерес среди российских разработок в области высокоуровневых средств синтеза схем вызывает решение Автокод HDL, создаваемое в институте прикладной математики им. М.В. Келдыша РАН [39, 40].

Авторы проекта пришли к выводу о непригодности традиционных языков программирования, ориентированных на архитектуру фон Неймана, для высокоуровневого проектирования для ПЛИС. Они разрабатывают новый язык высокоуровневого проектирования логических схем для реконфигурируемых устройств – Автокод-HDL – который объединяет в себе краткость языков высокого уровня и ориентированность на доступную в FPGA аппаратную архитектуру (схемотехническую модель программирования), характерную для языков описания аппаратуры HDL [39].

По мнению его авторов, написание программы на языках VHDL и Verilog представляет собой некоторое подобие ручного кодирования команд процессора, а программа на автокоде – ассемблер для этого

процессора. Спроектированная версия языка Автокод HDL позволяет описывать разрабатываемые схемы с помощью описания операторов, выполняющихся на каждом такте, при этом синхронность исполнения тактов обеспечивается самим языком [39].

В текущей версии языка отсутствуют некоторые базовые логические операторы, а также нет возможности создавать пользовательские типы данных.

Авторы рассчитывают, что после завершения работы над “ассемблером” для ПЛИС, следующим шагом станет появление более высокоуровневых языков, подобных С/Фортран, которые будут поддерживать традиционные подходы к программированию.

Проект Mitrion Platform предполагает следующие шаги разработки приложения: реализовать вычислительно сложные функции на MitrionC, выполнить вызовы этих функций в основной программе специальными вызовами Mitrion Platform API, отладить с помощью Mitrion SDK приложение, сгенерировать конфигурационные файлы для FPGA. MitrionC расширяет обычный С с помощью собственных объявлений типов, списков, векторов и потоков, конструкций циклов и функций. При этом длина программы ограничена объёмом программируемой логики, за адресацию памяти в FPGA отвечает пользователь, отсутствуют исключения (деление на ноль, выход за пределы массива). Mitrion Platform API предоставляет следующие возможности: резервирование FPGA для приложения, загрузка логической схемы, запуск и остановка выполнения, пересылка данных между платой и системной памятью, а также наблюдение за выполнением алгоритма. Серьёзным недостатком Mitrion Platform является поддержка ограниченного круга аппаратных платформ, оснащенных FPGA [37, 41].

С лета 2008 года ведётся разработка стандарта межплатформенного параллельного языка программирования OpenCL. Цель разработки –

перенос параллельных алгоритмов с платформы на платформу с минимальными затратами. В основу OpenCL положен язык C с добавлением специфических для параллельных программ расширений. Фактически OpenCL ориентирован на использование сопроцессоров.

OpenCL предполагает построение приложения из двух блоков. Хост-часть выполняется на микропроцессоре (CPU или встроенном в FPGA) – это обычная программа на стандартном C/C++. Вычислительно сложные части приложения предлагается располагать во втором блоке – так называемых ядрах (kernels) – которые и будут выполняться на ускорителе (стандарт подразумевает возможность использования и GPU, и FPGA). Для написания ядер используется всё тот же C, но со специальными расширениями для использования параллелизма ускорителя. Для связи двух частей приложения используется OpenCL API – набор функций для хост-части, позволяющий обмениваться данными с ускорителем, а также запускать ядра на выполнение. Один из ведущих мировых производителей FPGA – компания Altera в ноябре 2012 года опубликовала первые результаты разработки компилятора OpenCL для создания ядер в своих FPGA семейства Stratix IV/V [42].

Краткая характеристика средств высокоуровневого программирования приведена в табл.4.1.

В результате проведенного качественного сравнения наиболее перспективным решением представлялись проекты Impulse C, LegUp и Altera OpenCL SDK. Также вызывает интерес (прежде всего своим альтернативным необычным подходом, а также отечественным происхождением) Автокод HDL [34].

На следующем этапе был проведен сравнительный анализ трех отобранных высокоуровневых средств разработки для FPGA (LegUp, Impulse C и Автокод HDL) на ряде тестовых задач. К сожалению, на

момент проведения данного анализа средство Altera OpenCL SDK еще не было доступно.

Таблица 4.1

Сравнение средств высокоуровневого проектирования

	Язык	ОС	Поддержка плавающей запятой	Поддержка FPGA компании Altera	Возможность получить некоммерческую лицензию	Возможность создания систем со встроенным процессором
Mitrion Platform	extended C	linux/ Windows	+	-	-	+
C-to-Verilog	extended C	linux, web-компилятор	-	Неспециализированный hdl	+	-
LegUp	extended C	Linux	+	+	+	+
Impulse-C-To-FPGA	extended C	Linux/ Windows	+	+	(по запросу)	+
Автокод HDL	специализированный	Linux	+	+/-	(по запросу)	-
Altera OpenCL SDK	C+ OpenCL	Linux/ Windows	+	+	для участников программы	-

4.3.2 Методика сравнения высокоуровневых средств разработки

Существует немало приемов и методик для оценки эффективности высокоуровневых средств разработки (например, работы [43], [44] и [45]). В частности, в работе [45] авторами предложена методика, позволяющая получать интегральные оценки по интересующими метрикам для HLS. Эту методику, за исключением алгоритма вычисления результирующих оценок для HLS, мы использовали в нашей работе [35]. Не меньшее значение имеет и подбор алгоритмов, используемых в качестве тестовых. Проблеме бенчмаркинга систем высокоуровнего синтеза посвящена работа [46], в ней предложен набор тестовых алгоритмов из разных областей

информатики (преимущественно ЦОС, криптография и архивация), которые, по мнению авторов, имеют хорошие перспективы для реализации на ПЛИС. В работе [35] тестирование HLS проводилось на алгоритмах FFT(БПФ) (64-х точечное, целочисленное, отсчеты и коэффициенты в комплексной форме, разрядность мнимой части = разрядности вещественной части = 16 бит), шифрования AES (длина блока = длине ключа = 128 бит), CRC16 (разрядность входных данных 8 бит) и функции-заглушке (dummy function), возвращающей входные значения назад без изменений (размерность передаваемых данных 8 бит), для оценки накладных расходов от использования того или иного средства разработки. Схемные решения, синтезированные при помощи описанных выше HLS, сравнивались между собой с помощью следующих критериев:

- максимальная тактовая частота, на которой может работать схема;
- пропускная способность схемы;
- рассеиваемая мощность ПЛИС с запрограммированной схемой;
- количество задействованных в схеме логических элементов;
- время, затраченное на разработку схемы.

Данные собирались при помощи Quartus II 9.1 Web Edition. Проекты собирались для ПЛИС EP2C7oF89618 семейства Cyclone II, так как только ПЛИС этого семейства поддерживаются как Quartus II 9.1 Web Edition, так и всеми исследуемыми HLS. Также были задействованы в сравнительном анализе схемы, описанные на Verilog.

После проведения всех испытаний каждой HLS по каждому атрибуту выставлялась оценка по следующему правилу [45]:

$$S_{l,f} = \frac{1}{N_a} \sum_{a=1}^{N_a} \frac{v_{a,l,f} - v_{a,l_0,f}^{min}}{v_{a,l_1,f}^{max} - v_{a,l_0,f}^{min}} \qquad (4.1)$$

где

N_a – количество тестовых задач;

$v_{a,l,f}$ – значение критерия f для HLS l при тестовой задаче a;

$v^{min}_{a,l_0,f}, v^{max}_{a,l_1,f}$ – минимальное и максимальное значение критерия f для тестовой задачи a.

Таким же способом осуществлялась оценка необходимого для начала работы с HLS времени, за исключением того, что для этой оценки $N_a = 1$.

4.3.3 Результаты сравнения

Результаты тестирования HLS представлены в табл.4.2. Вычисленные для них по приведенному выше правилу интегральные оценки – на рис.4.1.

Таблица 4.2

Результаты тестирования

Средство	Пропускная способность (Мб/с)	Максимальная тактовая частота (МГц)	Количество логических блоков	Затраты на разработку (ч)	Рассеиваемая мощность (мВ)	
Автокод HDL	11,9	143	135	4	36	CRC16
Impulse C	12,5	167	190	1	195	
Verilog	250,9	258	44	5	200	
LegUp(PH)	34,8	146	186	1	195	
Автокод HDL	66,3	70	25412	20	221	AES
Impulse C	1	69	2842	3	195	
Verilog	89,3	112	45325	25	239	
LegUp(PH)	2,7	102	3301	2	199	
Автокод HDL	263	67	51071	16	220	FFT
Impulse C	122	142	4245	2	197	
Verilog	242	99	29073	30	201	
LegUp(PH)	5,5	56	5588	1,5	197	
Автокод HDL	476	500	8	0,5	30	Dummy function
Impulse C	181	333	19	0,25	31,9	
Verilog	476	500	8	0,5	30	
LegUp(PH)	238	500	2	0,25	33	

Наиболее “низкоуровневым” из рассматриваемых нами средств разработки является Автокод HDL, поэтому временные затраты на синтез схем ожидаемо больше, по сравнению с ImpulseC и LegUp, но все же меньше, чем при разработке на Verilog.

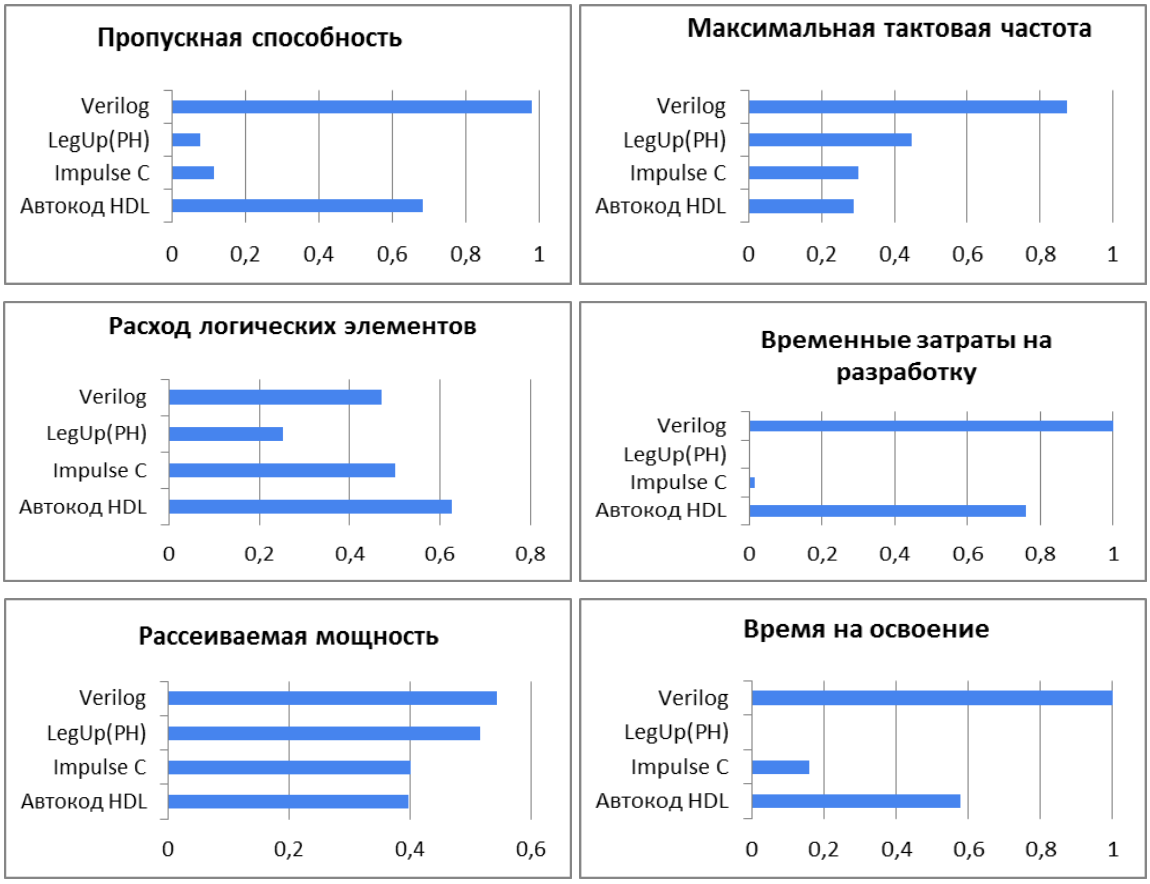


Рис.4.1. Интегральные оценки для HLS по результатам анализа

Impulse C является наиболее гибким инструментом среди рассматриваемых нами, но это коммерческий проект, кроме того для освоения парадигмы программирования, предлагаемой разработчиками данной системы, потребуется дополнительное время.

LegUp, с его уровнем поддержки ANSI C и набором скриптов, автоматизирующих процесс генерации прошивок, пожалуй, является наиболее удобным и простым для программиста, не знакомого с языками

описания аппаратуры. К тому же, это единственная HLS из рассмотренных, которая распространяется с открытым исходным кодом.

Чтобы оценить накладные расходы, создаваемые интерфейсом модуля, генерируемого исследуемыми HLS, и определить максимально достижимую тактовую частоту модулей, использовались тестовые схемы, возвращающие входные данные разрядностью 8 бит без изменений. В этом тесте как Verilog, так и Автокод, и Legup сгенерировали регистр, но в схеме, синтезированной Legup, присутствуют вспомогательные схемы, из-за которых она выдает данные один раз в 2 такта, а не в 1, что объясняет вдвое меньшую пропускную способность. Схема, синтезированная при помощи Impulse C, даже для такого простого примера включает в себя аппаратную реализацию базовых абстракций языка - входных и выходных потоков, которые заметно снижают общую производительность системы.

Следующим проведенным тестом была реализация алгоритма CRC-16. Вычислительная сложность данного алгоритма не высока и при последовательной подаче байтов входных данных вычисления 16-ти битного значения CRC16 занимает 1 такт при ручной реализации на Verilog. Схемы, сгенерированные участвующими в тестировании HLS, тратят на вычисление значительно больше тактов и не могут сравниться с эталонной ни по пропускной способности, ни по занимаемой площади кристалла, ни по рассеиваемой мощности.

64-х точечный алгоритм комплексного БПФ, использованный нами в следующем тесте, позволяет оценить возможности HLS по конвейеризации схем. Специфика алгоритма такова, что значительное время работы схемы тратится на прием и передачу входных и выходных данных. 64 отсчета/коэффициента занимают 64*2*2 = 256 байт и принимаются / выдаются в течение 64-х тактов, сам же этап вычислений может быть проведен за 60-100 тактов. При организации 3-х ступенчатого конвейера можно получить пиковую пропускную способность 1 коэффициент за такт.

Результаты, полученные в этом тесте, показывают что оптимальными средствами для организации вычислительных конвейеров обладает Автокод, а Legup не особенно подходит для организации высокопроизводительных схем.

Последний тест с наибольшей вычислительной сложностью - блочное шифрование по алгоритму AES со 128-ми битным ключом и блоком. Объем передаваемых данных для операции шифрования блока (128+128 бит на вход и 128 бит на выход) не слишком высок и позволяет организовать схемное решение с входными/выходными шинами соответствующей разрядности. Также алгоритм включает повторяющиеся действия над данными в процессе шифрования (10 раундов), что обеспечивает возможность конвейеризации. Этот тест оказался самым тяжелым для всех HLS, кроме Автокода, отставшего по производительности от эталона на Verilog всего на 33%. Хуже всего в этом тесте показал себя Legup, очевидно из-за того, что перенос модели памяти программы на C непосредственно на ПЛИС малоэффективно, и это особенно заметно на задачах, требующих интенсивных вычислений, таких как AES.

4.3.4 Выводы

Проведённое небольшое исследование позволяет утверждать, что свободные системы высокоуровнего синтеза могут показывать результаты, сравнимые с коммерческими системами. Среди рассмотренных HLS наиболее производительные модули можно создавать при помощи Автокод HDL, что, однако, достигается за счет максимального среди всех рассмотренных HLS расхода логических элементов. LegUp, в свою очередь, обеспечивает минимальное расходование ресурсов кристалла. При этом время, затрачиваемое программистом на разработку схемы, как и время, необходимое на освоение системы, при использовании

LegUp минимально, т.к. его входной язык практически полностью соответствует стандарту C99. Что касается рассеиваемой мощности синтезируемых модулей, то здесь лидером является Impulse C.

Обзор средств высокоуровневого проектирования был проведен магистрантами И.М. Силкиным и Ю.В. Шафраном под руководством А.Е. Андреева, а сравнительный анализ – магистрантами Д.А. Чернояровым, Г.О. Фроловым, М.А. Оболониным под руководством и при участии В.А. Малолеткова и А.Е. Андреева

Глава 5

ИССЛЕДОВАНИЕ РЕАЛИЗАЦИИ АЛГОРИТМОВ ДИСКРЕТНЫХ ЛИНЕЙНЫХ ПРЕОБРАЗОВАНИЙ НА РЕКОНФИГУРИРУЕМЫХ СОПРОЦЕССОРАХ

5.1 Алгоритмы дискретных линейных преобразований (ДЛП)

К алгоритмам, ориентированным на аппаратную реализацию (аппаратурно-ориентированным), относят, в частности, итерационные алгоритмы, известные под названием "цифра за цифрой", в которых применяются простые арифметические операции. Обобщение этих методов на реализацию процедур линейной алгебры приводит к классу алгоритмов, называемых алгоритмами дискретных линейных преобразований (ДЛП) [15-17, 47-48]. В алгоритмах ДЛП матрица заданного линейного преобразования представляется в виде сходящегося бесконечного произведения матриц простого вида:

$$\mathbf{A} = \prod_{i=0}^{\infty} \mathbf{A}_i \,, \tag{5.1}$$

где матрица A содержит только нули и суммы целых степеней основания системы счисления.

В этом случае вместо линейного преобразования вектора x матрицей A выполняется итерационный процесс, который можно легко реализовать на конвейере :

$$\mathbf{y}_{i+1} = \mathbf{A}_i \cdot \mathbf{y}_i \tag{5.2}$$

где $i = 0,\ 1,\ \ldots,\ n\text{-}1$, $\mathbf{y}_0 = \mathbf{x}$.

Матрицы преобразования на каждой итерации представляют собой ортогональные (унитарные) преобразования вращения, отражения, сдвига, причем параметры преобразования (элементарные углы, векторы отражения) для каждой итерации заранее известны (с точностью до знаков) и выбираются такими, чтобы матрицы Ai содержали только нули и суммы (разности) целых степеней основания системы счисления (для большинства вычислительных систем - степени двойки).

В результате при $n \to \infty$, $\mathbf{y}_n \to \mathbf{y}$, а бесконечное произведение матриц сойдется к матрице A. Как правило, итерационный процесс строится таким образом, чтобы обеспечивалась линейная сходимость, и поэтому количество операций n сравнимо с разрядностью операндов. При этом после каждой итерации уточняются очередные цифры результата. Матрицы Ai в алгоритмах ДЛП на каждой итерации известны с точностью до знаков элементов, поэтому нет необходимости вычислять параметры преобразования на каждой итерации, а нужно только определить знаки, которые зависят от значений компонент преобразуемого вектора, полученных на предыдущей итерации. А так как матрица содержит только нули и суммы целых степеней основания системы счисления, то вычисления на каждой итерации сводятся к выполнению простых («коротких») операций сложения и сдвига, что сокращает время преобразования и уменьшает аппаратурные затраты.

Наиболее известным алгоритмом ДЛП является алгоритм Дж. Волдера для реализации матрицы ортогонального преобразования плоского вращения

$$\mathbf{R}(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix} \qquad (5.3)$$

в двумерном случае [49]. Алгоритм Волдера, как и устройства на его основе, в литературе принято называть CORDIC - сокращение от Coordinate Rotation DIgital Computer - цифровой компьютер координатного вращения. По этому алгоритму вращение выполняется по итерационной процедуре без вычисления матрицы (5.3):

$$\mathbf{y}_{i+1} = \mathbf{R}_i \cdot \mathbf{y}_i \; ;$$

$$\mathbf{R}_i = \begin{bmatrix} 1 & -\xi_i \cdot 2^{-i} \\ \xi_i \cdot 2^{-i} & 1 \end{bmatrix}; \qquad (5.4)$$

$$\alpha_{i+1} = \alpha_i - \xi_i \Delta\alpha_i \; ; \quad \Delta\alpha_i = arctg 2^{-i} \; ;$$

$$i = \overline{0, n-1} \; ; \; \mathbf{y}_0 = \mathbf{x},$$

где $\xi_i = \pm 1$ и задает направление вращения на i-той итерации. На каждой итерации поворот осуществляется на фиксированный угол $\Delta\alpha_i$. Выбор ξ_i и α_0 зависит от режима алгоритма. При повороте вектора x на заданный угол (режим вращения CORDIC или режим “поворот”):

$$\xi_i = sgn\, \alpha_i \; ; \; \alpha_0 = \alpha \; . \qquad (5.5)$$

Если требуется вращать вектор **x** до совмещения с осью абсцисс (режим векторизации, вычисления модуля или режим “вектор”) :

$$\xi_i = -sgn\, y_{2i} \; ; \quad \alpha_0 = 0 \; . \qquad (5.6)$$

В результате “векторизации” в идеальном (без учета погрешности) случае вектор $\mathbf{y}_n$ будет иметь координаты

$$y_{2n} = 0, \quad y_{1n} = k\sqrt{x_1^2 + x_2^2} \quad . \qquad (5.7)$$

Преобразованные по методу Волдера вектора и в случае вращения, и в случае векторизации будут деформированы за счет коэффициента удлинения

$$k = \prod_{i=0}^{n-1} (1 + 2^{-2i})^{1/2} , \qquad (5.8)$$

который может быть учтен заранее, либо скомпенсирован специальными аппаратурными средствами [14, 50, 51].

Время выполнения одной итерации алгоритма (5.4) при аппаратурной реализации приближается к времени выполнения одного сложения.

Алгоритмы ДЛП удовлетворяют большинству требований к аппаратурно-ориентированным алгоритмам. Точность вычислений в методах ДЛП линейно зависит от разрядности операндов и достигается увеличением числа итераций. Численная устойчивость гарантируется унитарностью (ортогональностью) выполняемых преобразований. В этом отношении блоки, построенные на известных алгоритмах ДЛП вращений, предпочтительнее традиционных процессорных элементов, включающих быстрые умножители, делители и сумматоры. Если, например, рассмотреть матрицу плоского вращения (5.3), то при реализации преобразования с использованием стандартных умножителей как обычной процедуры умножения матрицы на вектор, мы имеем дело с 4-мя числовыми параметрами, а в случае реализации в виде ДЛП в силу особенностей метода фактически имеется один числовой параметр - угол α_0, что предпочтительнее из соображений численной устойчивости [52]. Сходимость известных методов ДЛП линейная, а для некоторых - квадратичная. Алгоритмы ДЛП имеют направленные регулярные графы, о чем свидетельствуют выражения (5.1, 5.2, 5.4). Эти алгоритмы рекурсивны,

каждая итерация выполняется в алгоритмах ДЛП за одинаковое время. Данные в алгоритмах ДЛП последовательно передаются с этапа на этап. В ДЛП методах обеспечивается побитовая рекурсия, они позволяют на каждой итерации получать очередной разряд (цифру) результата, поэтому такие методы и называют “цифра за цифрой”. Указанные свойства делают методы ДЛП пригодными для организации конвейеров как при параллельной обработке всех разрядов числе, так и при последовательной обработке по битам.

Два основных режима алгоритмов ДЛП, режим вращения (или режим применения преобразования) и векторизации, аналогичные соответствующим режимам CORDIC, позволяют параллельно обрабатывать ведущие столбцы (те, в которых на данном шаге обнуляются соответствующие компоненты) и остальные столбцы матриц, для которых ищутся необходимые разложения. В отличие от обычных методов выполнения типовых преобразований, для методов ДЛП не требуется предварительного вычисления параметра (или параметров) преобразования, поскольку в режиме векторизации одновременно осуществляется и расчет параметра, и само преобразование. Выполняемое на i-той итерации алгоритма элементарное преобразование задается направлениями (знаками ξ_i, так как остальные параметры в матрицах Ai фиксированы), которые могут непосредственно передаваться из блоков, выполняющих векторизацию в главных столбцах, блокам, выполняющим то же элементарное преобразование в режиме вращения (применения преобразования) для остальных столбцов. Эта особенность алгоритмов ДЛП делает их пригодными для параллельных систем, как в случае реализации каждой итерации алгоритма на отдельной ступени конвейера, так и в случае реализаций всех итераций на одной аппаратуре (последовательная реализация).

Алгоритмы ДЛП имеют ограниченный набор операций, который фактически сводится к операциям сложения, вычитания, сдвига, определения и изменения знака. Исключением является процедура масштабирования (нормализации), которая может выполняться для векторов в целом или даже для всей преобразованной матрицы в зависимости от реализуемого метода ЛА. Существуют варианты реализации ДЛП без масштабирования [14, 50, 51, 53].

Методы ДЛП пригодны не только для выполнения типовых ортогональных (унитарных) процедур, но также и для выполнения операций умножения, деления, извлечения квадратного корня (линейная функция CORDIC). При этом указанные операции выполняются медленнее, чем на специализированных аппаратных умножителях, однако, если в системе требуется и выполнять базовые унитарные преобразования, и вычислять тригонометрические функции, и выполнять умножение/деление, блоки ДЛП являются наиболее универсальным вариантом для выполнения всего набора процедур [14].

Таким образом, итерационные методы ДЛП позволяют строить специализированные устройства с достаточно высоким быстродействием и умеренными аппаратными затратами и обеспечивают при этом требуемую точность.

К настоящему времени известно большое число основных алгоритмов ДЛП и их модификаций для выполнения типовых ортогональных преобразований и некоторых унитарных преобразований [47 - 86]

Помимо многочисленных модификаций алгоритма CORDIC можно выделить алгоритмы ДЛП отражения Хаусхолдера [15, 18, 58, 75], ДЛП кронекеровского (прямого) произведения матриц вращения [56, 67, 77], кватернионное [73, 74, 76] и октонионное ДЛП [80, 83], комплексные (унитарные) ДЛП [20, 64, 66, 67, 69-71] и целый ряд других.

Несмотря на большое количество публикаций, посвященных ДЛП, большинство из них относятся к алгоритму CORDIC и его применениям. Описания реализаций ДЛП на заказных БИС и на элементной базе FPGA также в основном относятся к CORDIC [68]. Аппаратная реализация других алгоритмов ДЛП рассматривается авторами либо в общем виде, либо не относится к ПЛИС. Что касается описаний реализации этих алгоритмов с помощью средств высокоуровневого проектирования (HLS), такие публикации авторам не известны. В то же время нужно отметить, что данный вопрос представляет интерес, поскольку многие характеристики реализаций таких алгоритмов носят оценочный и часто очень приближенный характер. С другой стороны, многие из этих алгоритмов до сих пор не до конца изучены (например, вопросы сходимости многомерных ДЛП). В этих условиях было бы полезно оценить более реальные параметры алгоритмов при их аппаратной реализации, причем на наиболее доступной для этого элементной базе устройств FPGA. Относительно быстро это можно сделать с помощью HLS, что позволит одновременно оценить и применимость таких систем для прототипирования данного класса алгоритмов.

5.2 Многомерные алгоритмы ДЛП

Среди алгоритмов ДЛП можно выделить алгоритмы выполнения многомерных преобразований [18-20, 54-55, 58, 63-67, 69-83]. Большая часть упомянутых в предыдущем пункте алгоритмов ДЛП (кроме алгоритма CORDIC и его модификаций) относятся к таковым. Также к многомерным можно отнести по факту и алгоритмы комплексного вращения, основанные на кронекеровском произведении матриц вращения, либо на кватернионном преобразовании, поскольку при переходе от

комплексного пространства к вещественному мы также используем многомерное преобразование.

Все эти алгоритмы часто эксплуатируют один общий подход – укрупнение преобразования, выполняемого на каждой итерации ДЛП, как альтернатива применению традиционных итераций CORDIC в той или иной комбинации. Здесь необходимо отметить, что хотя во многих задачах обработки матриц (например, при выполнении QR разложения) ортогональное преобразование вращения может заменить другие ортогональные преобразования (например, отражения), в других случаях эти преобразования (отражение, кватернионное, октанионное) не могут быть заменены и их реализация представляет самостоятельный интерес. В данном разделе мы, однако, часто будем сравнивать параметры полученных реализаций многомерных алгоритмов именно с параметрами реализации CORDIC. Во-первых, действительно, многомерное ортогональное преобразование часто является альтернативой плоскому вращению. Во-вторых, интересно оценить в целом стоимость такого подхода, дает ли он какие-то реальные преимущества, в частности, при реализации на FPGA. Этот вопрос в литературе часто либо обходится вниманием, либо излагается в контексте реализации на специализированных кристаллах (ASIC). Известны также оценки, согласно которым многомерные ДЛП не эффективны при реализации на ПЛИС или ASIC.

Наглядным случаем, для которого можно сопоставить два подхода (использование плоских вращений или многомерного преобразования), является задача векторизации (поиска модуля / ориентирования вдоль одной из осей) N-мерного вектора (рис.5.1). Если мы решаем задачу с помощью пирамиды CORDIC-блоков, нам потребуется $\log_2 N$ этапов, на каждом из которых выполняется m итераций (на рисунке представлена пирамида CORDIC для N=4). Использование одного устройства ДЛП

отражения (например, Householder CORDIC) обеспечивает получения результата за один итерационный этап (те же m или m + k шагов, где k– количество повторений итераций).

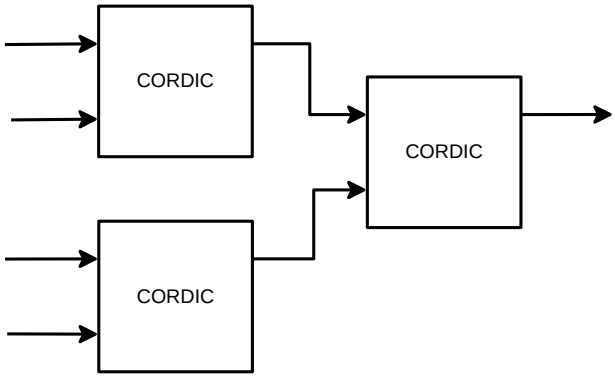


Рис.5.1. Структурная схема поиска модуля 4-мерного вектора

Само по себе избавление от $\log_2 N$ этапов не позволяет утверждать, что тем самым повышается быстродействие или производительность конвейера. За многомерность преобразования приходится платить усложнением каждой итерации, что приводит к увеличению времени выполнения одной итерации и к паритету по общему времени выполнения при уменьшении тактовой частоты конвейера (за счет увеличения длительности такта). Также сложно говорить о сокращении аппаратных затрат при реализации многомерных преобразований по сравнению с решениями на базе CORDIC, поскольку общее количество операций в алгоритмах при обоих подходах сопоставимо. Кроме того, нужно иметь в виду, что в многомерных ДЛП алгоритмах зачастую приходится вводить дополнительные итерации (например, в ДЛП отражения), соответственно возрастает общее время вычислений (и усложняется конвейер за счет увеличения сила его ступеней).

Все это в конечном итоге может объяснить относительно редкое применение таких алгоритмов по сравнению с CORDIC. Действительно, если мы проигрываем в частоте конвейера, увеличиваем общее время счета

и не выигрываем в аппаратных затратах, сложно говорить о практической применимости таких решений (еще раз отметим, что речь идет от тех случаях, когда плоские вращения являются альтернативой многомерным преобразованиям).

В то же время, в работах [19-20, 55, 58, 63-67 и др.] указывается на то, что многомерные ДЛП алгоритмы могут позволять добиваться уменьшения аппаратных затрат, прежде всего, за счет сокращения накладных расходов на регистры-защелки и особенно – на сумматоры-вычитатели. Это в том числе связано с применением такой схемотехнической базы, как сумматоры с сохранением переноса (ССП – Carry-Save Adders). Они позволяют достаточно эффективно экономить как время, так и площадь кристалла при реализации суммирования N чисел по сравнению с классической пирамидальной схемой. При этом, однако, чаще всего происходит увеличение времени выполнения каждой итерации ДЛП, и, как следствие – падение темпа конвейерной обработки. По этой причине конвейерные варианты ДЛП решений на базе CORDIC чаще всего выглядят предпочтительнее аналогичных многомерных решений, например, на базе отражений. И только в тех немногочисленных случаях, когда используется неконвейерный режим, с учетом сокращения накладных расходов многомерные ДЛП могут быть эффективны.

К сожалению, в большей части известных работ проводятся только оценки занимаемой площади и характеристики производительности многомерных ДЛП, либо не приводятся сравнения с CORDIC на одной элементной базе, что не позволяет проверить справедливость, либо ошибочность утверждения об их относительной эффективности.

Отдельно стоит остановиться на реализации многомерных ДЛП на ПЛИС. Насколько можно судить из доступных источников, такие реализации и их характеристики также не рассматривались (или почти не рассматривались) в отличие от многочисленных примеров использования

CORDIC и его модификаций. В то же время, по мнению авторов, специфическая схемотехника ПЛИС (FPGA) может наложить свой отпечаток на результирующие решения, и, возможно, повысить их эффективность.

В связи с этим представляется актуальным попробовать оценить характеристики многомерных ДЛП-решений при реализации на ПЛИС, как самих по себе, так и в сравнении с решениями на базе CORDIC, когда это возможно.

5.3 Оценка характеристик реализации алгоритмов ДЛП на ПЛИС

Для оценки характеристик реализации алгоритмов ДЛП на ПЛИС, прежде всего многомерных ДЛП, а также для дальнейшего изучения возможностей средств высокоуровневого проектирования для ПЛИС, рассмотренных в предыдущей главе, был проведен ряд экспериментов по реализации алгоритмов ДЛП с использованием следующих средств разработки:

- VerilogHDL;
- LegUp 3.0;
- Altera OpenCL SDK (с версией Quartus II 13).

Выбор высокоуровневых средств обусловлен выводами, приведенными в предыдущей главе, с учетом сложности реализации на Автокоде - HDL, а также в связи с тем, что на момент проведения работ тестовый стенд с Автокодом-HDL был недоступен по причине замены как оборудования, так и программного обеспечения в связи с переходом к новой версии. Включение в рассмотрение Altera OpenCL SDK объясняется, наоборот, доступностью этого средства разработки и соответствующего аппаратного обеспечения для авторов по академической программе Altera.

На первом этапе была проведена реализация классического алгоритма CORDIC с помощью перечисленных средств.

Прежде всего, необходимо отметить ряд особенностей выполнявшейся реализации, приведенных ниже.

1) Поскольку традиционно ДЛП алгоритмы строятся с использованием представления чисел с фиксированной запятой, была выбрана именно такая реализация. Это позволяет избежать трудностей с поддержкой плавающей запятой в различных средствах HLS и необходимости оценки качества и особенностей этой поддержки. Разрядность всех чисел – 32 бита.

2) Поскольку оба выбранных высокоуровневых средства реализации используют в качестве входного язык C (или C + OpenCL), исходные версии алгоритмов также реализовывались на C. Для оценки реализации с фиксированной запятой в качестве базового использовался тип integer (32 разряда).

3) Для исключения автоматической генерации средствами HLSблоков деления и умножения все операции сдвига реализовывались именно через сдвиг (операции <<и >>).

4) В качестве базового рассматривался вариант алгоритма CORDIC без вычисления углов («неявный CORDIC») только в режиме векторизации. Рассматривался как вариант с конвейером (реализуется при разворачивании цикла – каждая итерация выполняется на своей ступени), так и без него.

5) В качестве выходной информации для оценки реализации использовались данные системы САПР ПЛИС Quartus II по количеству сгенерированных логических блоков (LE), других занятых ресурсов ПЛИС (FF, DSP), а также данные о тактовой частоте, общем количестве тактов преобразования, либо количестве выполняемых сгенерированных ядер OpenCL в единицу времени (для Altera OpenCL SDK).

Также необходимо отметить, что на данном этапе использовались разные версии САПР ПЛИС Quartus II (9, 11 и 13), а также целевые устройства (семейства Cyclon II и Stratix V), что связано с доступностью лицензий и особенностями использованных средств синтеза. В итоге сравнивать полученные на разных средствах синтеза результаты между собой можно только отчасти (хотя логические блоки Cyclon II и Stratix V в принципе сопоставимы между собой), скорее качественно. Сравнение количественных показателей реализаций возможно только между разными вариантами алгоритмов, реализованных с помощью одного и того же средства.

В процессе выполнения экспериментов дополнительно уточнялись особенности использования каждого из средств разработки.

5.3.1 Реализация алгоритма CORDIC

В качестве реализации на Verlog был выбран проект, размещенный на открытом ресурсе www.opencores.org [86], с небольшими изменениями для реализации именно «неявного» CORDIC.

Параметры реализации на Verilog, полученные с помощью QuartusII 9.0SP2 Web Edition, приведены в таб.5.1.

Таблица 5.1

Характеристики реализации алгоритма CORDIC на Verilog

Вариант алгоритма	Сложность устройства (логических блоков,LE / регистров, FF)	Тактовая частота, МГц	Пропускная способность, (М задач/с)
Конвейерный	3565 / 2046	113	113
Без конвейера	1140 / 71	83,65	2,61

Из таблицы 5.1 видно, что конвейерный CORDIC при увеличении частоты в 1,35 раза, а производительности почти в 43 раза, требует только в 3,13 раз больше аппаратных затрат (в логических блоках, но при этом в 32 раза больше регистров). Это объясняется, в частности, упрощением схемы одной ступени (за счет фактически устранения сдвигателя) и уменьшением ее задержки в случае конвейерной реализации. Эти свойства аппаратной реализации алгоритма CORDIC хорошо известны.

При реализации того же алгоритма на LegUp мы воспользовались программой на C, приведенной в приложении А. В результате синтеза, который проводился с использованием образа для виртуальной машины, настроенной авторами LegUp, с установленной на нее версией системы 3.0 и QuartusII 11.1, получены результаты, приведенные в табл.5.2.

Необходимо отметить, что, несмотря на наличие в версии LegUp 3.0 средств конвейеризации циклов, получить конвейерный вариант CORDIC, конвейеризованный по итерациям, не удалось. Вызывает сомнение количество использованных логических блоков, оно в 13,6 раза меньше количества блоков, сгенерированных по версии алгоритма на Verilog. LegUp при использования режима Pure Hardware стремится минимизировать аппаратные затраты любым способом, возможно даже за счет подгонки схемы под конкретные параметры, передаваемые на ее вход в функции main(). В результате сгенерированная схема подходит только для очень приближенного начального прототипирования и сравнения разных вариантов ДЛП алгоритмов. В режиме Hybrid система строится в LegUp с использованием софт-процессора Tiger с системой команд RISC, а функция CORDIC создается в виде отдельного Verilog файла с аппаратным ускорителем этого процессора, однако и этот ускоритель при его анализе на Quartus требует слишком малого количества логических блоков.

Что касается частоты полученной схемы, она почти в два раза меньше, чем для версии на Verilog, что, вполне вероятно, объясняется

чрезмерной минимизацией аппаратных затрат при генерации данным средством.

Таблица 5.2

Характеристики реализации алгоритма CORDICна C / LegUp 3

Вариант алгоритма	Сложность устройства (логических блоков,LE / регистров,FF)	Тактовая частота, МГц	Пропускная способность, (М задач/с)
Конвейерный	-	-	-
Без конвейера	84 / -	46,58	1,37

Для оценки характеристик реализации алгоритма с помощью средства Altera OpenCL SDK необходимо подготовить код ядра OpenCL, которое фактически представляет собой обычную программу на C с некоторыми ограничениями. В частности, придется отказаться от вызова в ядре функций, не возвращающих void, использовать специальные указатели, не использовать вложенные циклы и так далее. Код ядра для CORDIC приведен в приложении Б. В отличие от LegUp, компилятор Altera OpenCL вполне успешно генерирует конвейеры путем разворачивания циклов с помощью прагмы

```
#pragma unroll<N>,
```

где

N – максимальное количество шагов, которые нужно разворачивать (по умолчанию без <N> используется максимально возможное количество шагов).

Также по поводу Altera OpenCL необходимо отметить, что в данном случае не использовалась генерация реальной прошивки для FPGA,

поскольку этот процесс занимает несколько часов даже на очень мощной машине. Вместо этого использовался режим быстрой генерации с предварительной оценкой параметров получающейся схемы:

```
aoc  –c  CORDIC.cl  --report
```

Характеристики реализации алгоритма CORDIC на Altera OpenCL SDK(версия QuartusII 13.1) приведены в табл.5.3 (здесь и далее приводятся только показатели ядра, без учета дополнительных схем окружения). Параметр процента общего использования FPGA приведен для сравнения с реализацией других алгоритмов ДЛП на Altera OpenCL для того же кристалла.

Таблица 5.3

Характеристики реализации алгоритма CORDIC на Altera OpenCL

Вариант алгоритма	Сложность устройства (логических блоков,LE / регистров, FF)	Использо-вание FPGA, %	Тактовая частота, МГц	Пропускная способность, (М задач/с)
Конвейерный	5790 / 5073	1.63	250	250
Без конвейера	2413 / 6747	1.23	250	7,81

В данном случае мы видим картину несколько идеализированную – конвейерный CORDIC имеет ровно в 32 раза большую производительность по сравнению с неконвейерным (в данном случае используется предварительная оценка средствами SDK). При этом сложность конвейера в логических блоках всего в 2,3 раза выше, чем у неконвейерного варианта (а общее использование кристалла с учетом регистров больше только на треть, увеличение же количества используемых регистров довольно неожиданно). Здесь не проявляется эффект увеличения тактовой частоты

для конвейерного устройства (как при реализации на Verilog), что связано, видимо, с особенностями реализации конвейеров в данном средстве HLS. Сложность данного варианта реализации выше, чем при реализации на HDL (примерно в 2 раза), что связано с максимальной тактовой частотой устройства и накладными расходами средства высокого уровня.

Тем не менее, мы видим, что Altera OpenCL SDK демонстрирует вполне объяснимые результаты синтеза, соответствующие параметрам реализации на Verilog (за исключением количества регистров в неконвейерном случае).

5.3.2 Реализация алгоритма Householder-CORDIC

При реализации ДЛП отражения Хаусхолдера возможны две формы алгоритма – векторная и матричная [18-20, 65].

Основные итерационные соотношения векторной формы алгоритма :

$$\left.\begin{aligned} y_1' &= \left(1+2^{-2i+l}\right) y_1 \quad - \quad 2\left(\sum_{k=1}^{m} f(k)\,\xi_k\, y_k\right) \\ y_j' &= \left(1+2^{-2i+l}\right) y_j \quad - \quad 2\left(\sum_{k=1}^{m} f(k)\,\xi_k\, y_k\right) 2^{-i}\,\xi_j \end{aligned}\right\} \qquad (5.9)$$

где

$$f(k) = \begin{cases} 1, & k=1 \\ 2^{-i}, & k>1 \end{cases}, \quad \xi_k = \begin{cases} 1, & k=1 \\ \operatorname{sgn}(y_1)\operatorname{sgn}(y_k), & k>1 \end{cases},$$

m- размерность вектора, $m = 2^l + 1$,

i – номер итерации, $i = \overline{0,\ n}$,

j – номер компоненты вектора, $j = \overline{2,\ m}$,

y_p – компоненты исходного,

аy_p' – преобразованного векторов.

В векторной форме скалярное произведение, стоящее в скобках справа в уравнениях, вычисляется один раз явно и затем используется во всех m уравнениях.

Для матричного алгоритма ДЛП Хаусхолдер-CORDIC основные итерационные соотношения выглядят следующим образом :

$$\left.\begin{aligned} y_1' &= \left(2^{-2i+l}-1\right)y_1 - \left(\sum_{k=2}^{m} 2^{-i+1}\xi_k y_k\right) \\ y_j' &= \left(1+2^{-2i+l}\right)y_j - 2^{-2i+1}y_j - 2^{-i+1}\xi_k y_1 - \\ &- \left(\sum_{k=2}^{j-1} 2^{-2i+1}\xi_{jk} y_k\right) - \left(\sum_{k=j+1}^{m} 2^{-2i+1}\xi_{jk} y_k\right) \end{aligned}\right\}, \qquad (5.10)$$

где

$$\xi_{jk} = \operatorname{sgn}\left(y_j\right)\operatorname{sgn}\left(y_k\right) \quad , \quad \xi_k = \operatorname{sgn}\left(y_1\right)\operatorname{sgn}\left(y_k\right)$$

В (5.9) и (5.10) предполагается, что для размерности m выполняется

$$m = 2^l + 1, \qquad (5.11)$$

в общем случае 2^{-2i+l} в (5.9 -5.10) трансформируется в $2^{-2i}\,(m-1)$.

Поскольку векторная форма предполагает увеличение времени выполнения каждой итерации и, возможно, как следствие – увеличение такта конвейера и уменьшение его производительности, сначала рассмотрим реализацию матричной формы.

Наиболее полезными с практической точки зрения являются реализации алгоритма Householder-CORDIC для векторов размерности от 2 до 10 (с ростом размерности векторов растет и количество дополнительных итераций алгоритма для обеспечения его сходимости [58]). При этом для размерностей m, удовлетворяющих условию (5.11) аппаратные затраты будут меньше. В диапазоне m=2..10 этому условию

соответствуют m = 3, 5, 9, поэтому будут рассмотрены именно эти случаи, а также m=4 для сравнения.

Для реализации с помощью средства HLS LegUp матричной формы алгоритма Householder-CORDIC отражения 4-мерного вектора разработана программа на C, приведенная в приложении B. В результате с помощью LegUp построено решение с параметрами, приведенными в табл.5.4.

Тактовая частота устройства в принципе та же, что и для CORDIC, а вот пропускная способность меньше почти в два раза. Таким образом, если сравнивать производительность данной схемы при векторизации 4-мерного вектора, она будет соответствовать производительности пирамиды из 3 CORDIC (рис. 5.1), при этом сложность схемы будет почти в 3 раза меньше. Правда, речь идет только о неконвейерном, то есть на практике менее полезном случае. Снова, как и при реализации CORDIC, показатели сложности устройства вызывают сомнение – вряд ли возможно получить решение для данного алгоритма, размещаемое всего в 85 логических блоках (при том, что CORDIC потребовал лишь на 1 логический блок меньше).

Таблица 5.4

Характеристики реализации матричной формы алгоритма Householder-CORDIC(4-мерное преобразование) на C / LegUp 3

Вариант алгоритма	Сложность устройства (логических блоков,LE / регистров, FF)	Тактовая частота, МГц	Пропускная способность, (М задач/с)
Конвейерный	-	-	-
Без конвейера	85 / -	48,18	0,73

По причине заниженной оценки сложности генерируемых схем ДЛП на LegUp дальнейшие эксперименты проводились только на Altera OpenCL SDK.

Для реализации алгоритма в матричной форме (размерность векторов равна 3) на Altera OpenCL SDK разработан код ядра OpenCL, приведенный в приложении Г. Для эффективной реализации потребовалось убрать внутренние необязательные циклы, развернув их явно. Кроме того, для сокращения сложности генерируемой схемы необходимо было избавиться от операций вида x_k * sign(x_i) * sign (x_j), где sign() принимает значения ±1. Несмотря на то, что согласно документации Altera по оптимизации кода умножение на ±1 реализуется просто, как и умножение на степени 2, фактически использование умножения на sign(x_i) * sign (x_j) приводило к увеличению сложности схемы при реализации конвейера на порядок (по-видимому, компилятор генерирует для выполнения этой операции умножитель). В итоге такое умножение было заменено на анализ логических переменных с помощью операции XOR и предварительное формирование операндов с нужными знаками :

```
// ...
// Определяем знаки в виде bool
ksil[0] = y[0] >= 0 ? 0 : 1;
ksil[1] = y[1] >= 0 ? 0 : 1;
ksil[2] = y[2] >= 0 ? 0 : 1;

// сочетания знаков ++ и -- = +, остальное -
ksi[1] = (ksil[0]&&(!ksil[1])) || ((!ksil[0])&&ksil[1]);
ksi[2] = (ksil[0]&&(!ksil[2])) || ((!ksil[0])&&ksil[2]);
ksi23 =  (ksi[1]&&(!ksi[2])) || ((!ksi[1])&&ksi[2]);

// подготавливаем операнды с нужными знаками сразу
Y[0][1] = ksi[1] ? -y[1] : y[1];
Y[0][2] = ksi[2] ? -y[2] : y[2];  //...
```

Если анализ знаков с помощью XOR близок к схемотехническому решению, которое в частности рассматривается в работе [20], то предварительное формирование операндов с нужным знаком расходится с ним, так как при рассмотрении реализации в работе [20] предполагалось использовать сумматоры - вычитатели. Реализовать генерацию сумматоров - вычитателей из кода ядра OpenCL без ущерба для наглядности исходного кода достаточно сложно, поскольку отсутствует возможность использования в коде ядра не void() функций.

В результате получены параметры решения, представленные в табл.5.5.

Таблица 5.5

Характеристики реализации матричной формы алгоритма Householder-CORDIC (3-мерное преобразование) на C /Altera OpenCL SDK

Вариант алгоритма	Сложность устройства (логических блоков,LE / регистров, FF)	Исполь-зование FPGA, %	По отно-шению к пирамиде из 2(3) CORDIC, %	Пропускная способность, (M задач/с)
Конвейерный	13937 / 11620	3.86	-21(+18)	250
Без конвейера	3537 / 9270	1.73	-53(-30)	7,81

Тактовая частота в табл.5.5 и далее не приводится, так как она для всех схем равна 250 МГц. Приведено сравнение с пирамидой CORDIC, решающей задачу обработки 3-мерного вектора, по использованию ресурсов FPGA. Такая пирамида может включать 3 блока CORDIC (рис. 5.2). Наличие дополнительного блока (выделен на рисунке) объясняется необходимостью масштабирования третьего элемента вектора на тот же коэффициент деформации K, который получается при обработке первых двух элементов на первом ярусе пирамиды. Такое масштабирование можно

выполнить и другими средствами, но обычно его выполняют аналогичным блоком. В скобках приведены значения для пирамиды из 2 CORDIC (без масштабирующего блока).

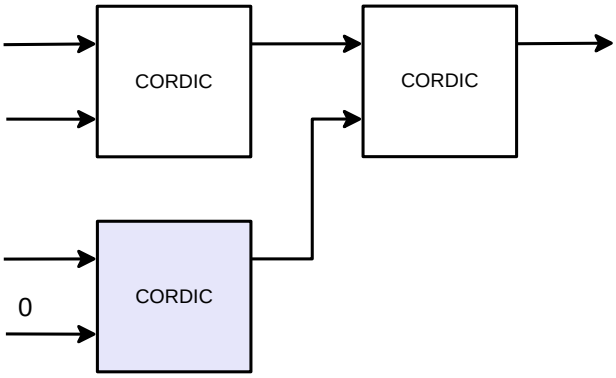


Рис. 5.2. Структурная схема обработки 3-мерного вектора

Из табл. 5.5 видно, что для алгоритма выполнения 3-мерного отражения сложность реализации конвейера немного меньше, чем сложность 3 блоков CORDIC (на 21%), но выше, чем для пирамиды из двух блоков CORDIC(на 18%), хотя для пирамиды из 2 CORDIC потребуется какое-то дополнительное средство масштабирования третьего элемента, сложность которого здесь не учтена.

Для неконвейерной реализации преимущество данной схемы по сравнению с пирамидой CORDIC более заметно и будет проявляться даже по сравнению с двумя блоками CORDIC.

При обработке больших векторов группами по 3 элемента преимущество рассмотренной схемы по сравнению с пирамидами из блоков CORDIC будет зависеть от общего количества дополнительных масштабирующих блоков и в общем случае будет меньше.

Вполне возможно, что приведенная в приложении Г реализация не является оптимальной для Altera OpenCL. Для более достоверной оценки параметров аппаратной реализации ДЛП отражения необходимо

использоватьVerilog/VHDL. В частности, необходимо предусмотреть применение сумматоров с сохранением переноса.

Для размерностей 4,5 и 9 также были разработаны реализации ядер OpenCL, аналогичные приведенному в приложении Г. Создание различных исходных кодов для разных размерностей, а не общей программы, объясняется рассмотренными выше ограничениями на код OpenCL ядер. В перспективе можно построить небольшое средство автоматизации создания ядер для разных размерностей алгоритма отражения.

Характеристики реализаций матричной формы алгоритма Householder-CORDIC для размерностей 4, 5, 9 приведены в табл. 5.6. Пирамиды CORDIC для обработки векторов размерностей 5 и 9 приведены на рисунках 5.3 и 5.4 (для размерности 4 – на рисунке 5.1). В скобках указана сложность пирамиды CORDIC, в которой не используются блоки масштабирования.

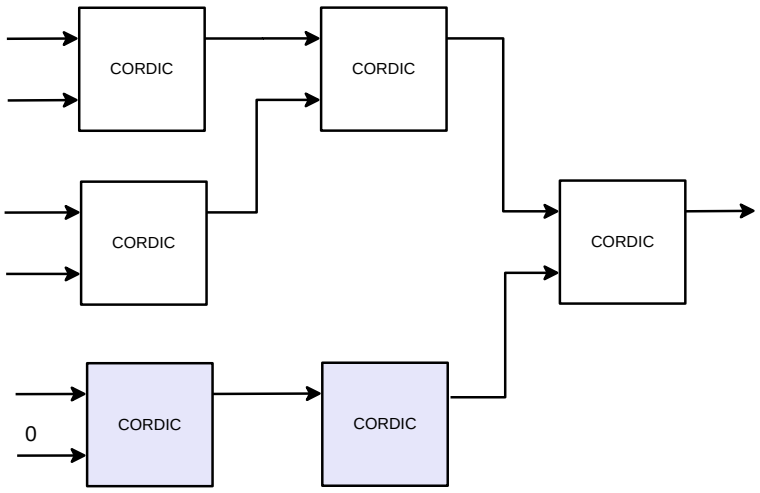


Рис. 5.3. Структурная схема обработки 5-мерного вектора

Как видно из полученных результатов, при реализации матричной формы алгоритма Householder - CORDIC для рассмотренных размерностей (за исключением размерности 5) сложность схемы (при реализации

конвейера) больше сложности пирамиды конвейерных CORDIC для обработки N - мерного вектора. Для размерности 5 аппаратные

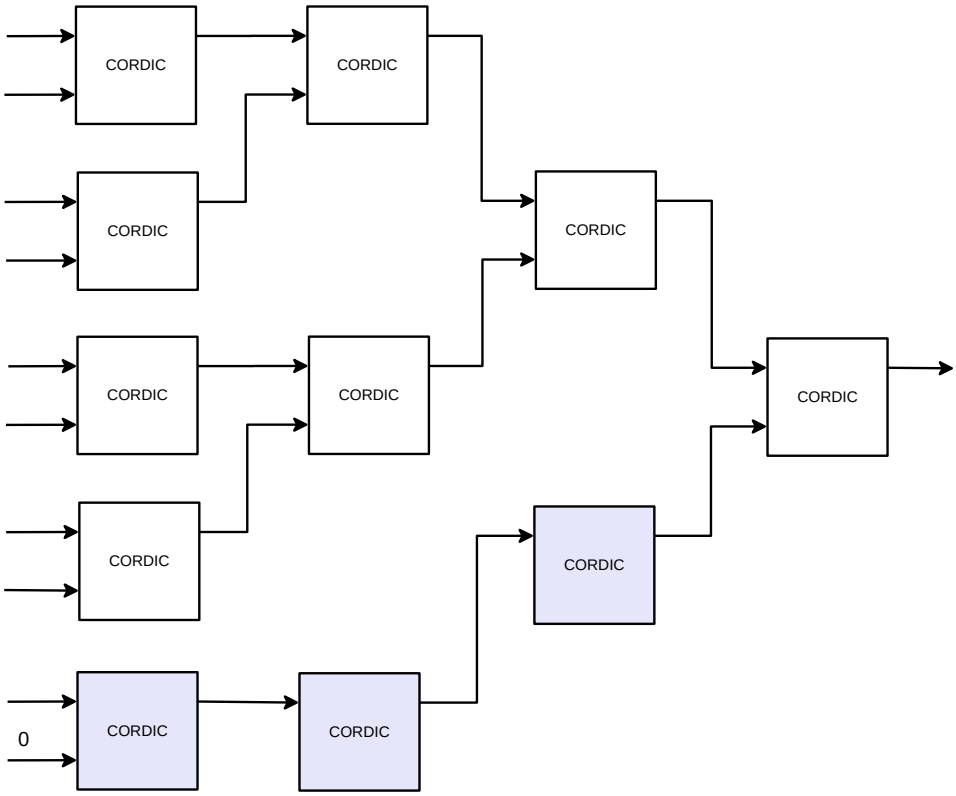


Рис.5.4. Структурная схема обработки 9-мерного вектора

затраты несколько ниже. В случае реализации без конвейера для всех размерностей наблюдается экономия аппаратных затрат на 40-50%. Но, как уже отмечалось, практическое применение неконвейерных вариантов ограничено, особенно для использования в сопроцессорах. Конвейер может не применяться, в частности, во встраиваемых системах, когда интенсивность поступления данных невысока, емкость кристалла невелика, но все же требуется ускорить CPU невысокой производительности (например, микроконтроллер), либо CPU вообще не используется.

Вполне возможно, что существуют другие размерности помимо 3 и 5, для которых также будет достигаться некоторая экономия аппаратных

затрат, но нужно учесть, что при размерностях > 10 увеличивается количество дополнительных итераций алгоритма, а соответственно – количество ступеней.

Таблица 5.6

Характеристики реализации матричной формы алгоритма Householder-CORDIC(размерности 4, 5, 9) на C /Altera OpenCL SDK

Вариант алгоритма и размерность	Сложность устройства (логических блоков,LE / регистров, FF)	Исполь-зование FPGA, %	По отно-шению к пирамиде CORDIC, %	Пропускная способность, (M задач/с)
Конвейерный, 4-мерный	24069 / 18924	6,54	+33,7	250
Без конвейера, 4-мерный	4570 / 12018	2.25	-39,02	7,81
Конвейерный, 5-мерный	31999 / 24675	8.63	-11,76 (+32,36)	250
Без конвейера, 5-мерный	5456 / 14642	2.72	-63,14 (-44.72)	7,81
Конвейерный, 9-мерный	86905 / 64045	23.09	+28,67 (+57,42)	250
Без конвейера, 9-мерный	10825 / 28040	5,27	-61,61 (-46,34)	7,81

Схемные решения для алгоритмов обработки векторов разной размерности, построенные с помощью Altera OpenCL SDK, имеют одинаковую производительность, хотя операционная сложность итераций алгоритмов разная. Это, по-видимому, связано с особенностями генерации схем компилятором и особенностями реализации конвейеров. Вполне возможно, что реализации векторных форм алгоритма окажутся столь же производительными, при этом в общем случае они обладают меньшей

операционной и аппаратурной сложностью, поскольку в них явно вычисляется норма вектора отражения.

Были разработаны версии OpenCL ядер для векторной формы алгоритма Householder-CORDIC размерностей 3, 4, 5, 9. Исходный код для размерности 3 приведен в приложении Д. Показатели реализации векторной формы алгоритма приведены в табл.5.7. Поскольку неконвейерный случай не представляет большого интереса, в таблице рассмотрены только характеристики конвейерных реализаций. При сравнении с пирамидами CORDIC в скобках указано сравнение с пирамидой без дополнительных блоков масштабирования.

Таблица 5.7

Характеристики конвейерной реализации векторной формы алгоритма Householder-CORDIC на C /Altera OpenCL SDK

Вариант алгоритма (размерность обрабатываемого вектора)	Сложность устройства (логических блоков,LE / регистров, FF)	Исполь-зование FPGA, %	По отно-шению к пирамиде CORDIC, %	Пропускная способность, (М задач/с)
3-мерный	12681 / 9743	3,42	-30 (+5)	250
4-мерный	23195 / 18132	6.29	+70.46	250
5-мерный	19152 / 15031	5.2	-46,8(-20)	250
9-мерный	39819 / 29639	10.62	-40,77(-18.6)	250 (222)

Показатели сложности реализации векторной формы, как и ожидалось, лучше, чем для матричной. Для всех размерностей, удовлетворяющих условию (5.11), наблюдается экономия аппаратных затрат по сравнению с решением на базе CORDIC от 18 до 47%. Для размерности 4, не удовлетворяющей условию (5.10), показатели даже хуже, чем для 5. Интересно, что при этом достигается та же тактовая

частота и производительность, что в общем случае при аппаратной реализации не всегда возможно.

Здесь нужно заметить, что реализация алгоритма Householder-CORDIC проводилась без учета необходимого числа итераций (которое в общем случае согласно [58] будет меньше, чем для алгоритма CORDIC) и числа повторяемых итераций для обеспечения сходимости, которых для размерностей < 10 немного (порядка 3) [58].

Подводя итог, можно заметить, что в результате проведенного прототипирования алгоритма ДЛП отражения на LegUp получены результаты, которые трудно считать достоверными. Более достоверны результаты, полученные при использовании Altera OpenCL SDK. Онип оказывают, что алгоритм Householder-CORDIC в матричной форме при аппаратной реализации с использованием данного средства HLS уступает по своим показателям решению для обработки вектора на базе пирамиды CORDIC. Для векторной формы аппаратная реализация алгоритма обеспечивает лучшие показатели использования кристалла для размерностей, удовлетворяющих условию (5.11), в нашем случае это размерности 3, 5, 9. В дальнейшем имеет смысл получить параметры реализации алгоритма для других размерностей, а также сравнить с реализацией на HDL языках. Также можно заключить, что с учетом определенных ограничений и выполнения ряда требований к исходному коду инструмент Altera OpenCL SDK может быть применен для прототипирования алгоритмов класса ДЛП.

5.3.3 Реализация алгоритма ДЛП кронекеровского произведения вращений

Другим примером алгоритма реализации многомерного ДЛП является алгоритм кронекеровского (прямого) произведения вращений (КП) [20, 56, 67, 70, 77].

Алгоритм прямого произведения двух элементарных плоских вращений при реализации в виде ДЛП выглядит следующим образом.

$$\frac{1}{cos^2 \Delta\varphi_i}\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}_{i+1} = \begin{bmatrix} 1 & -\gamma_{1i} & -\gamma_{2i} & \gamma_{1i}\gamma_{2i} \\ \gamma_{1i} & 1 & -\gamma_{1i}\gamma_{2i} & -\gamma_{2i} \\ \gamma_{2i} & -\gamma_{1i}\gamma_{2i} & 1 & -\gamma_{1i} \\ \gamma_{1i}\gamma_{2i} & \gamma_{2i} & \gamma_{1i} & 1 \end{bmatrix}\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix}_i;$$

$$\varphi_{j(i+1)} = \varphi_{ji} - \xi_{ji}\Delta\varphi_i; \quad \xi_{j(i+1)} = sgn\,\varphi_{ji}; \quad \gamma = \xi_{ji}2; \qquad (5.12)$$

$$j = \overline{1,2}; \quad i = \overline{1,n}; \quad \varphi_{10} = \alpha_1; \quad \varphi_{20} = \alpha_2;$$

$$\mathbf{y}_0 = \mathbf{x}; \quad \mathbf{y} = \mathbf{y}_n \quad .$$

Программная реализация ДЛП кронекеровского произведения двух плоских вращений в виде OpenCL приведена в приложении Е.

В результате синтеза алгоритма получены показатели сложности и производительности, приведенные в табл.5.8. Производительность неконвейерного варианта в два раза выше, чем для алгоритма CORDIC, что несколько неожиданно. Для неконвейерного варианта по сравнению с 4 блоками CORDIC (с помощью которых можно решать часть из тех задач, для которых применяется ДЛП КП вращений, например, реализовывать метод Якоби) можно достичь экономии аппаратных затрат более 50%.

Таблица 5.8

Характеристики реализации алгоритма ДЛП кронекеровского произведения вращений

Вариант алгоритма	Сложность устройства (логических блоков,LE / регистров, FF)	Исполь-зование FPGA, %	По отно-шению к решению на CORDIC, %	Пропускная способность, (М задач/с)
Конвейерный	22021 / 17304	5.98	-8,3	250
Без конвейера	4753 / 12981	2.39	-51,4	15,63

При этом сложность конвейера ДЛП КП вращений лишь немного меньше (на 8%) сложности четырех конвейеров CORDIC.

Для реализации комплексного вращения с помощью двух ДЛП КП вращений и 4 сумматоров, как описано в [20, 67, 70], потребуется схема вдвое большей сложности, чем приведено в табл.5.9, что значительно превосходит сложность реализации комплексного вращения с использованием 4 процессоров CORDIC. При этом преимуществом данной схемы (как и ДЛП КП вообще) является меньшее время получения конкретного решения (меньшая латентность).

5.3.4 Реализация ДЛП комплексного отражения

Рассмотрим реализацию комплексного (унитарного) ДЛП отражения на базе алгоритма Householder-CORDIC, описанную в [20, 64, 66, 71]. Поскольку более эффективной согласно пункту 5.3.2 представляется реализация векторной формы алгоритма, остановимся на ней.

Итерационные соотношения алгоритма [20] :

$$
\left.
\begin{aligned}
y'_{1Re} &= \left(2^{-2i+l}-1\right) y_{1Re} - 2^{-i+1}\sum_{k=2}^{m} S_1\left(\xi_{kRe},\xi_{kIm},y_{kRe},y_{kIm}\right) \\
y'_{1Im} &= \left(2^{-2i+l}-1\right) y_{1Im} - 2^{-i+1}\sum_{k=2}^{m} S_2\left(\xi_{kRe},\xi_{kIm},y_{kRe},y_{kIm}\right) \\
y'_{jRe} &= \left(1+2^{-2i+l}\right) y_{jRe} - 2^{-i+1} S_3\left(\xi_{jRe},\xi_{jIm},y_{1Re},y_{1Im}\right) - \\
&\quad - 2^{-2i+1} S_4\left(\xi_{jRe},\xi_{jIm},a'_{Re},a'_{Im}\right) \\
y'_{jIm} &= \left(1+2^{-2i+l}\right) y_{jIm} - 2^{-i+1} S_5\left(\xi_{jRe},\xi_{jIm},y_{1Re},y_{1Im}\right) - \\
&\quad - 2^{-2i+1} S_6\left(\xi_{jRe},\xi_{jIm},a'_{Re},a'_{Im}\right)
\end{aligned}
\right\} \qquad (5.13)
$$

где

S_t, $t=\overline{1,6}$ - функции выбора, определяемые следующим образом.

$$
\begin{aligned}
S_1\left(\xi_{kRe},\xi_{kIm},y_{kRe},y_{kIm}\right) &= \begin{cases} \xi_{kRe}y_{kRe}, & \xi_{kRe}\xi_{kIm}>0 \\ \xi_{kIm}y_{kIm}, & \xi_{kRe}\xi_{kIm}<0 \end{cases}, \\
S_2\left(\xi_{kRe},\xi_{kIm},y_{kRe},y_{kIm}\right) &= \begin{cases} \xi_{kRe}y_{kRe}, & \xi_{kRe}\xi_{kIm}<0 \\ \xi_{kIm}y_{kIm}, & \xi_{kRe}\xi_{kIm}>0 \end{cases}, \qquad (5.14) \\
S_3\left(\xi_{jRe},\xi_{jIm},y_{1Re},y_{1Im}\right) &= \begin{cases} \xi_{jRe}y_{1Re}, & \xi_{jRe}\xi_{jIm}>0 \\ \xi_{jRe}y_{1Im}, & \xi_{jRe}\xi_{jIm}<0 \end{cases}, \\
S_4\left(\xi_{jRe},\xi_{jIm},a'_{Re},a'_{Im}\right) &= \begin{cases} \xi_{jRe}\left(a'_{Re}-a'_{Im}\right), & \xi_{jRe}\xi_{jIm}>0 \\ \xi_{jRe}\left(a'_{Re}+a'_{Im}\right), & \xi_{jRe}\xi_{jIm}<0 \end{cases}, \\
S_5\left(\xi_{jRe},\xi_{jIm},y_{1Re},y_{1Im}\right) &= \begin{cases} \xi_{jIm}y_{1Im}, & \xi_{jRe}\xi_{jIm}>0 \\ \xi_{jIm}y_{1Re}, & \xi_{jRe}\xi_{jIm}<0 \end{cases}, \\
S_6\left(\xi_{jRe},\xi_{jIm},a'_{Re},a'_{Im}\right) &= \begin{cases} \xi_{jRe}\left(a'_{Re}+a'_{Im}\right), & \xi_{jRe}\xi_{jIm}>0 \\ \xi_{jRe}\left(a'_{Re}-a'_{Im}\right), & \xi_{jRe}\xi_{jIm}<0 \end{cases}.
\end{aligned}
$$

а a’ определяются через S_1 и S_2

$$
\begin{aligned}
a'_{Re}+a'_{Im} &= \sum_{k=2}^{m} S_1\left(\xi_{kRe},\xi_{kIm},y_{kRe},y_{kIm}\right), \qquad (5.15) \\
a'_{Re}-a'_{Im} &= \sum_{k=2}^{m} S_2\left(\xi_{kRe},\xi_{kIm},y_{kRe},y_{kIm}\right).
\end{aligned}
$$

В отличие от вещественного преобразования, в данном случае несколько усложняются логические функции для выбора знаков операндов :

```
ksir[0] = yr[0] >= 0 ? 0 : 1;
ksir[1] = yr[1] >= 0 ? 0 : 1;
ksir[2] = yr[2] >= 0 ? 0 : 1;
ksim[0] = yi[0] >= 0 ? 0 : 1;
ksim[1] = yi[1] >= 0 ? 0 : 1;
ksim[2] = yi[2] >= 0 ? 0 : 1;

// Повтор части итераций
if ((j % 2 == 0) || (j> 16)) i = i+1;

// сочетания знаков ++ и -- = +, остальное -
ksir0[1] = (ksir[0] && (!ksir[1])) || ((!ksir[0]) &&ksir[1]);
ksir0[2] = (ksir[0] && (!ksir[2])) || ((!ksir[0]) && ksir[2]);
ksim0[1] = (ksim[0] && (!ksim[1])) || ((!ksim[0]) && ksim[1]);
ksim0[2] = (ksim[0] && (!ksim[2])) || ((!ksim[0]) && ksim[2]);
ksi[1] = (ksir0[1] && (!ksim0[1])) || ((!ksir0[1])&& ksim0[1]);
ksi[2] = (ksir0[2] && (!ksim0[2])) || ((!ksir0[2])&& ksim0[2]);
//подготавливаем операнды для уравнений с нужными знаками сразу
Yr[0][1] = ksi[1] ? (ksim0[1] ? -yi[1] : yi[1]) :
           (ksir0[1] ? -yr[1] : yr[1]);
Yr[0][2] = ksi[2] ? (ksim0[2] ? -yi[2] : yi[2]) :
           (ksir0[2] ? -yr[2] : yr[2]);
Yi[0][1] = ksi[1] ? (ksir0[1] ? -yr[1] : yr[1]) :
           (ksim0[1] ? -yi[1] : yi[1]);
Yi[0][2] = ksi[2] ? (ksir0[2] ? -yr[2] : yr[2]) :
           (ksim0[2] ? -yi[2] : yi[2]);
// …
```

Исходный код ядра OpenCL приведен в приложении Ж.

Характеристики реализации векторной формы алгоритма ДЛП комплексного отражения (для размерности 3) приведены в табл.5.9.

Таблица 5.9

Характеристики реализации алгоритма ДЛП комплексного отражения (для размерности 3)

Вариант алгоритма	Сложность устройства (логических блоков,LE / регистров, FF)	Исполь-зование FPGA, %	По отно-шению к решению на CORDIC, %	По отно-шению к решению на Householder CORDIC,%	Пропускная способность, (М задач/с)
Конвейерный	41346 / 33277	11.32	-42	-3.5	250
Без конвейера	4725 / 13346	2.43	-83.5	-64	15,63

В данной реализации учтены повторы итераций, как описано в [20, 71] - примерно 8 итераций из первой половины (всего имеем 40 итераций для 32 – разрядных чисел). В таблице приведены также результаты сравнения сложности (по проценту использования FPGA) с альтернативными вариантами реализации комплексного отражения, рассмотренными в [20, 64, 66] :

- обработка вектора с помощью 3 блоков комплексного вращения (КВ), составленных из 4 блоков CORDIC (на рис. 5.5 показан блок КВ из 4 CORDIC, а пирамида блоков КВ соответствует рисунку 5.1 с заменой CORDIC на блоки КВ);

- обработка вектора с помощью двух блоков вещественных Householder-CORDIC и m блоков CORDIC (в данном случае m=3) (рис. 5.6) [64].

Как видно из табл.5.9, схема комплексного ДЛП отражения, предложенного в [20, 64, 71], позволяет уменьшить аппаратные затраты по сравнению с альтернативными вариантами даже с учетом повтора итераций, причем по сравнению с широко известным альтернативным вариантом (на базе CORDIC) затраты уменьшаются почти вдвое.

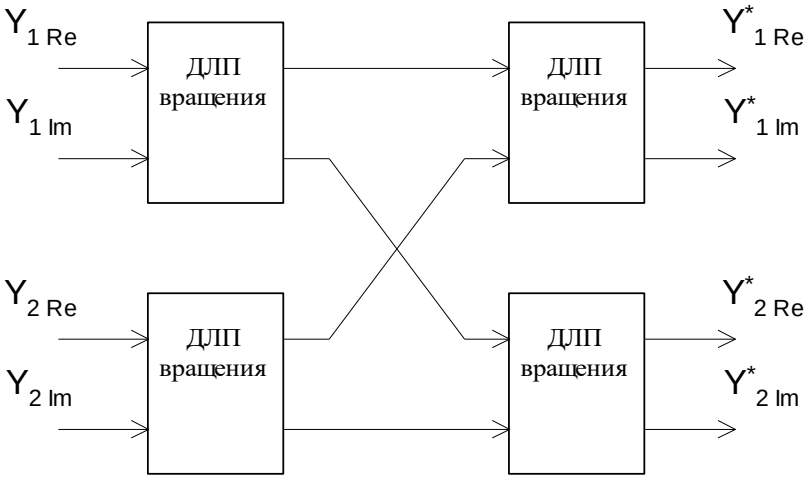


Рис. 5.5. Блок комплексного вращения из 4 CORDIC

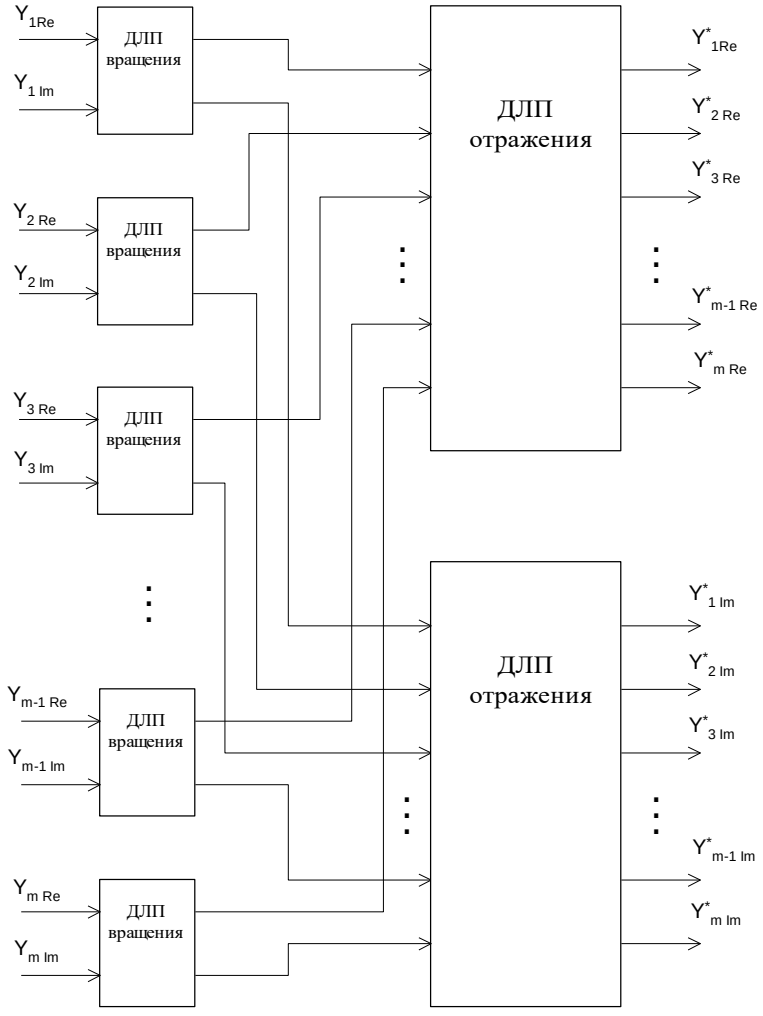


Рис. 5.6. Обработка вектора с помощью двух блоков вещественных Householder-CORDIC и m блоков CORDIC

5.4 Выводы

В данной главе рассмотрены оценки характеристик аппаратной реализации ряда алгоритмов дискретных линейных преобразований (ДЛП) на FPGA Altera с использованием высокоуровневых средств проектирования (HLS) LegUp и Altera OpenCL SDK.

На примере известного алгоритма ДЛП плоского вращения CORDIC проведено сравнение параметров реализации алгоритма на LegUp, Altera OpenCL SDKи Verilog HDL, в результате для оценки реализаций других алгоритмов принято решение использовать средствоAltera OpenCL SDK, для которого параметры сгенерированных схем в целом соответствуют параметрам решений, полученных спомощью Verilog. Указанное средство можно применять для прототипирования алгоритмов данного класса.

С помощью Altera OpenCL SDK авторами впервые проведена оценка параметров реализации на FPGA ряда многомерных алгоритмов ДЛП, описанных в виде OpenCL ядер на языке высокого уровня С. В том числе проведен анализ производительности и сложности схем, реализующих алгоритмы ДЛП отражения Householder-CORDIC, ДЛП кронекеровского произведения вращений, впервые получены конкретные параметры реализации разработанных авторами алгоритмов унитарных ДЛП комплексного вращения и отражения, для которых ранее были известны только приближенные аналитические оценки сложности и производительности.

Полученные результаты позволяют утверждать, что рассмотренные алгоритмы могут достаточно эффективно реализовываться на FPGA по описаниям на OpenCL.

Также показано, что многомерные ДЛП, прежде всего Householder-CORDIC, а также ДЛП комплексного отражения, реализуемые в векторной форме, для размерностей 3, 5, 9 позволяют сократить аппаратные затраты

при конвейерной обработке векторов соответствующей размерности по сравнению с использованием традиционного алгоритма CORDIC от 4 до 80% (в среднем на 30-40%).

Глава 6

РЕАЛИЗАЦИЯ КРИПТОЛОГИЧЕСКИХ АЛГОРИТМОВ НА ГВС-РС

6.1 Решение задачи поиска строки по MD5 хэш-сумме на FPGA сопроцессоре

По результатам оценки производительности реализаций алгоритмов, представленной в пунктах 2.3-2.5, было принято решение о разработке аппаратно-программного приложения для решения задачи восстановления строки по MD5 хэшу на реконфигурируемом ускорителе компании Altera, оснащенным FPGA Arria II GX EP2AGX125.

В главе 3 были рассмотрены ресурсы, имеющиеся в составе реконфигурируемого ускорителя, который будет использоваться для решения задач, и схемы систем взаимодействия приложения, работающего на CPU персонального компьютера, с аппаратной схемой, реализованной на кристалле FPGA. В данной главе приводится реализация блока, занимающегося непосредственно вычислениями в рамках рассматриваемого алгоритма. Она выполнена в рамках магистерского исследования И.М. Силкиным под руководством А.Е. Андреева.

6.1.1. Разработка общей структуры вычислительного блока

На рисунке 6.1 представлена структура вычислительного блока, осуществляющего решение задачи восстановления строки по хэш-сумме. Рассмотрим назначение модулей, входящих в состав данной схемы.

Управляющий модуль контролирует весь процесс вычисления. В регистрах данного модуля хранится искомый хэш, флаги готовности, найденный результат.

Генератор строк осуществляет полный перебор строк заданного алфавита. Сформированные строки данный модуль передает на входы модулей, осуществляющих хэширование по алгоритму MD5. Генератор производит одновременно количество строк равное количеству блоков, осуществляющих хэширование.

Модули хэширования реализуют алгоритм MD5. На входе они получают строку, для которой необходимо отыскать значение хэш-суммы, и её длину. В модуле проводятся операции в соответствии с алгоритмом, описанном в разделе 2.5. Найденные данными модулями значения хэш-сумм передаются в блок сравнения. Здесь они сопоставляются с эталонным значением хэша. В случае совпадения, блок сравнения подает сигнал устройству управления, который останавливает вычисления и фиксирует результат.

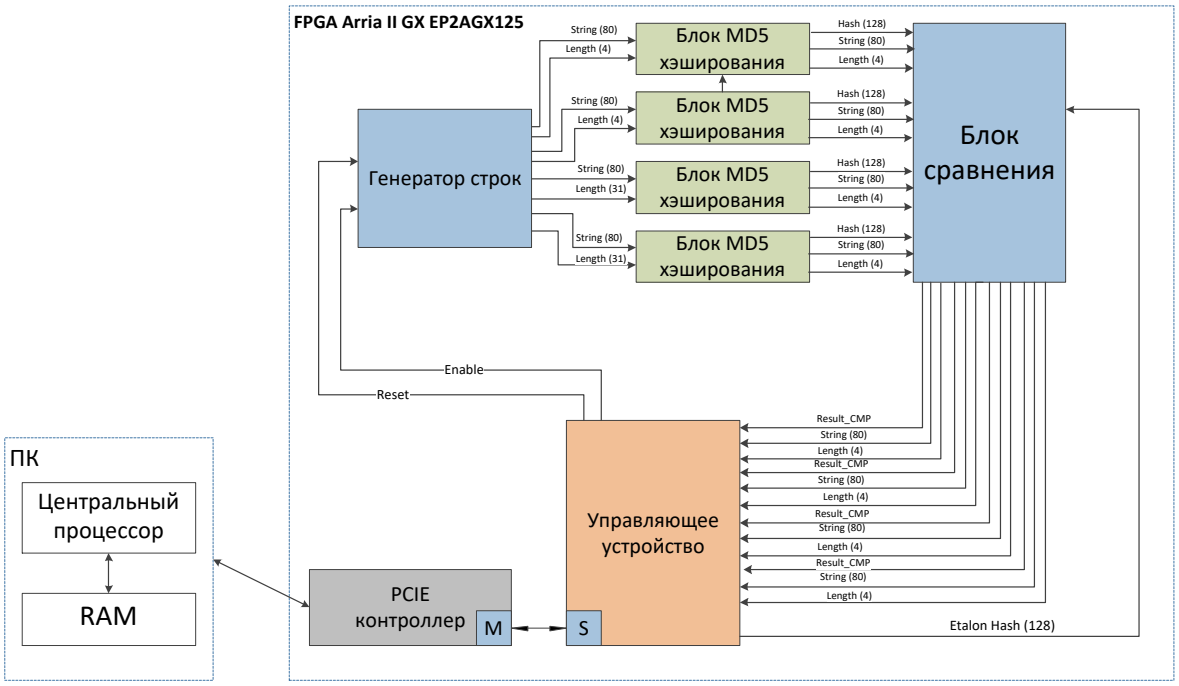


Рис. 6.1. Структура аппаратно-программного приложения

Приложение, выполняемое на CPU, может контролировать данное устройство путем чтения и записи регистров модуля управления.

В следующих разделах подробнее рассмотрим процесс реализации каждого из модулей. Каждый из модулей реализован на языке аппаратного описания схем Verilog.

6.1.2. Реализация модуля хэширования по алгоритму MD5

Интерфейс данного модуля представлен на рисунке 6.2. На вход подаются сигналы синхронизации clk и reset, строка, для которой необходимо получить значение хэш-суммы, и количество значимых битов данной строки. На выходе – строка, ее длина, хэш-сумма данной строки.

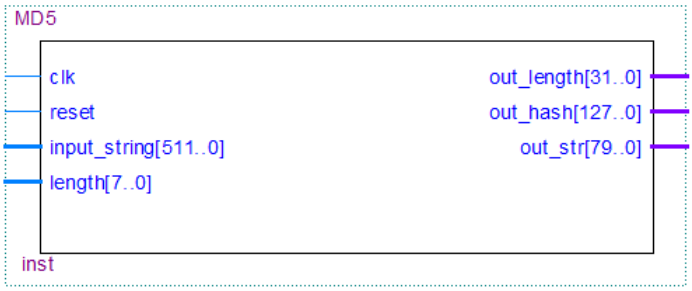


Рис. 6.2. Интерфейс модуля MD5 хэширования

Данное устройство собой конвейер для получения хэш-суммы входной строки. Как показано на рисунке 6.3, в состав рассматриваемого модуля входят следующие компоненты: модуль «Padding», модуль «Transformation», модуль «BlockToString». Помимо внутренних модулей на входах и выходах устройства расположены регистры для фиксирования данных и обеспечения синхронизации. Рассмотрим подробнее каждый из компонентов.

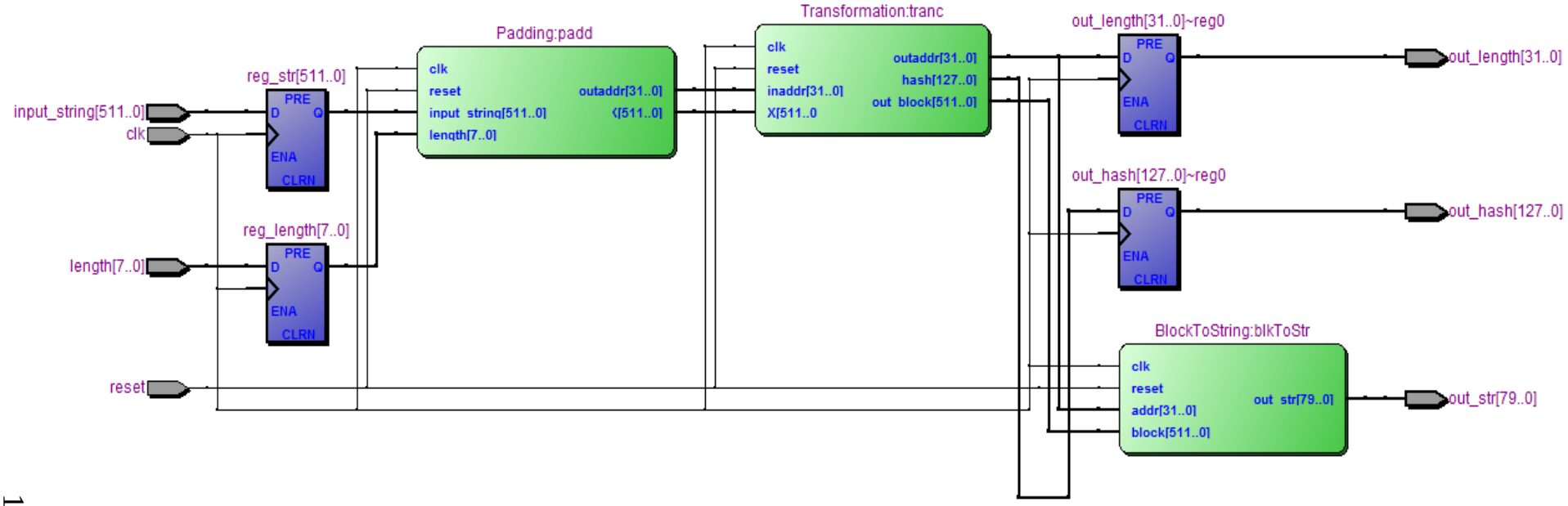


Рис. 6.3. Структура модуля MD5 хэширования

Модуль «Padding» осуществляет преобразование входной строки в блок для хэширования. Интерфейс данного блока представлен на рисунке 6.4. Здесь осуществляются операции выравнивания потока и добавление длины сообщения.

На вход данный блок принимает сигналы синхронизации, строку, полученную с помощью генератора строк, и длину этой строки в битах. На выход устройство выдает полученный из входной строки выровненный блок. Выходные данные модуля фиксируются в регистрах.

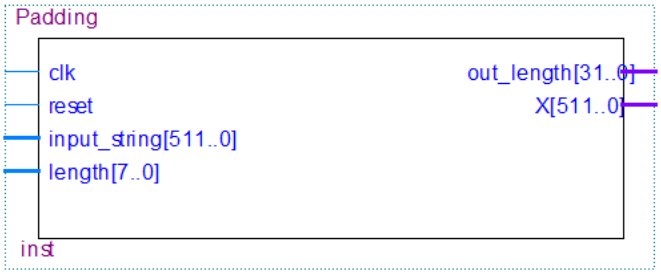


Рис. 6.4. Интерфейс модуля «Padding»

Рассмотрим модуль «Transformation». Его интерфейс представлен на рисунке 6.5, а структура – на рисунке 6.6. Входные данные: сигналы синхронизации, выровненный блок для хэширования, длина исходной строки. Выходные данные: хэш входной строки, длина входной строки и выровненный блок, над которым выполнялись преобразования (необходим для восстановления входной строки при выдаче результата).

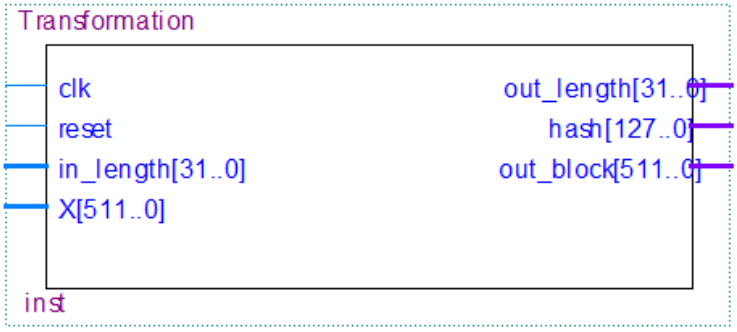


Рис. 6.5. Интерфейс модуля «Transformation»

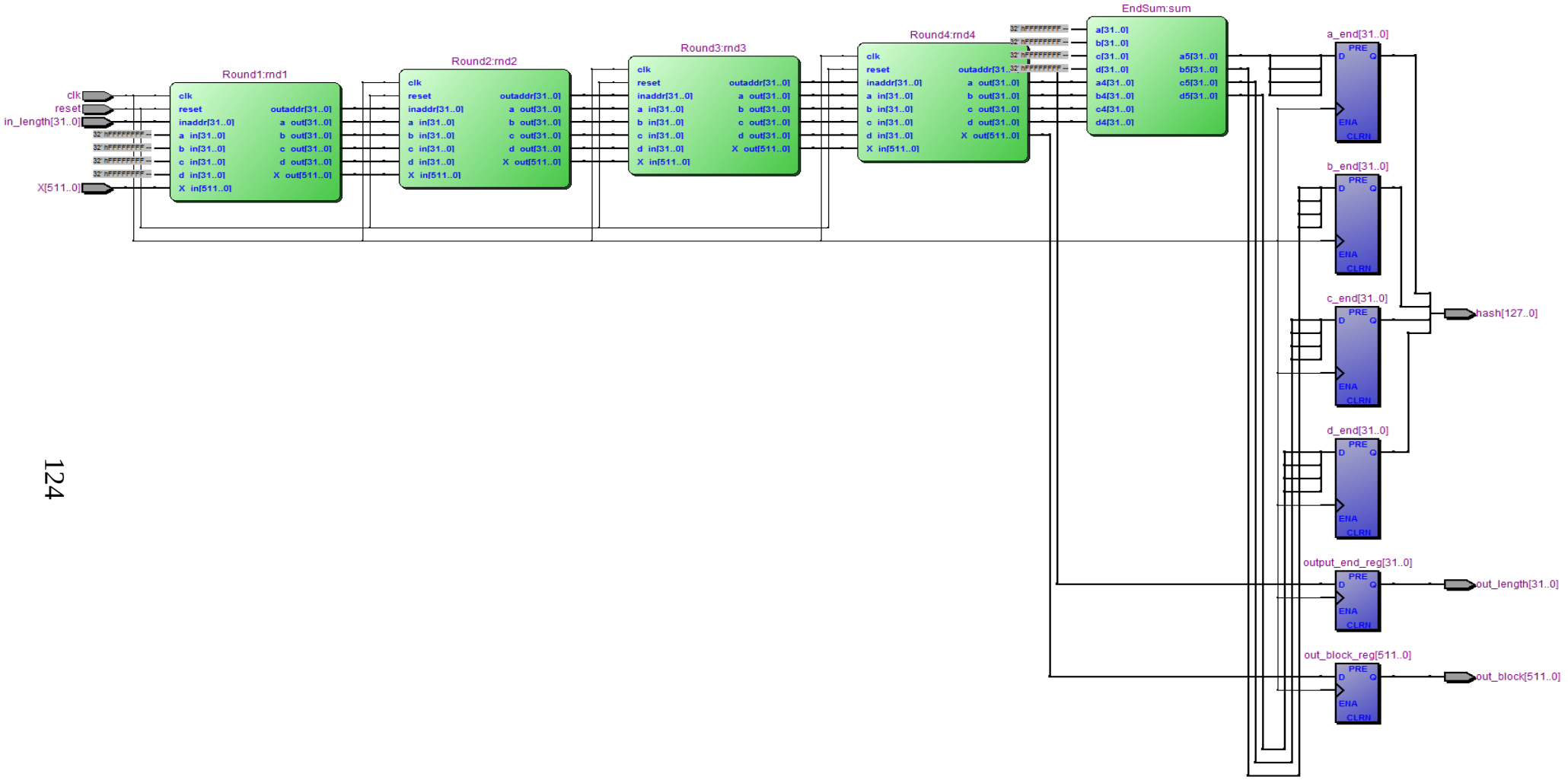


Рис. 6.6. Структура модуля модуля «Transformation»

Интерфейсы данных блоков «RoundX» идентичны. Входные данные: выровненный блок, полученный из исходной строки; значения a, b, c, d, в которых после выполнения всех преобразований окажется хэш-сумма; длина входной строки и сигналы синхронизации. Выходные данные: преобразованные значения a, b, c, d; выровненный блок; длина исходной строки.

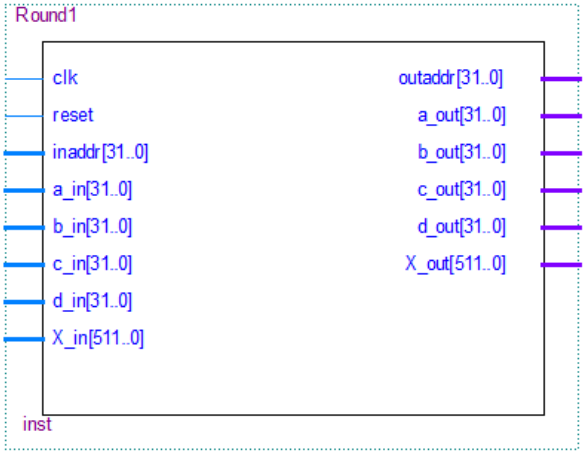


Рис. 6.7. Интерфейс модуля «Round1»

Структура данных модуля «Round1» представлена на рисунке 6.8. Из рисунка видно, что в состав данного модуля входят 16 последовательно соединенных модулей FunFF_7, которые реализуют вычисления согласно таблице 2.6 раздела 2.5. Остальные модули раундов устроены аналогично.

Структура модуля «FunFF_7» представлена на рисунке 6.9.

Модуль «FunFF_7» реализует вычисление значение функции:

FF(a, b, c, d, k, s, i) = b + ((a + F(b,c,d) + X[k] + T[i]) <<< 7) (6.1)

Как видно из структуры данного модуля, в его состав входит еще один модуль – «FunF», реализующий вычисление функции:

$$FunF(X,Y,Z) = (X \wedge Y) \vee (\overline{X} \wedge Z) \quad (6.2)$$

Остальные модули вида «FunXX_X» имеют аналогичную архитектуру и реализуют функции, описанные в разделе 2.5.



Рис. 6.8. Структура модуля «Round1»

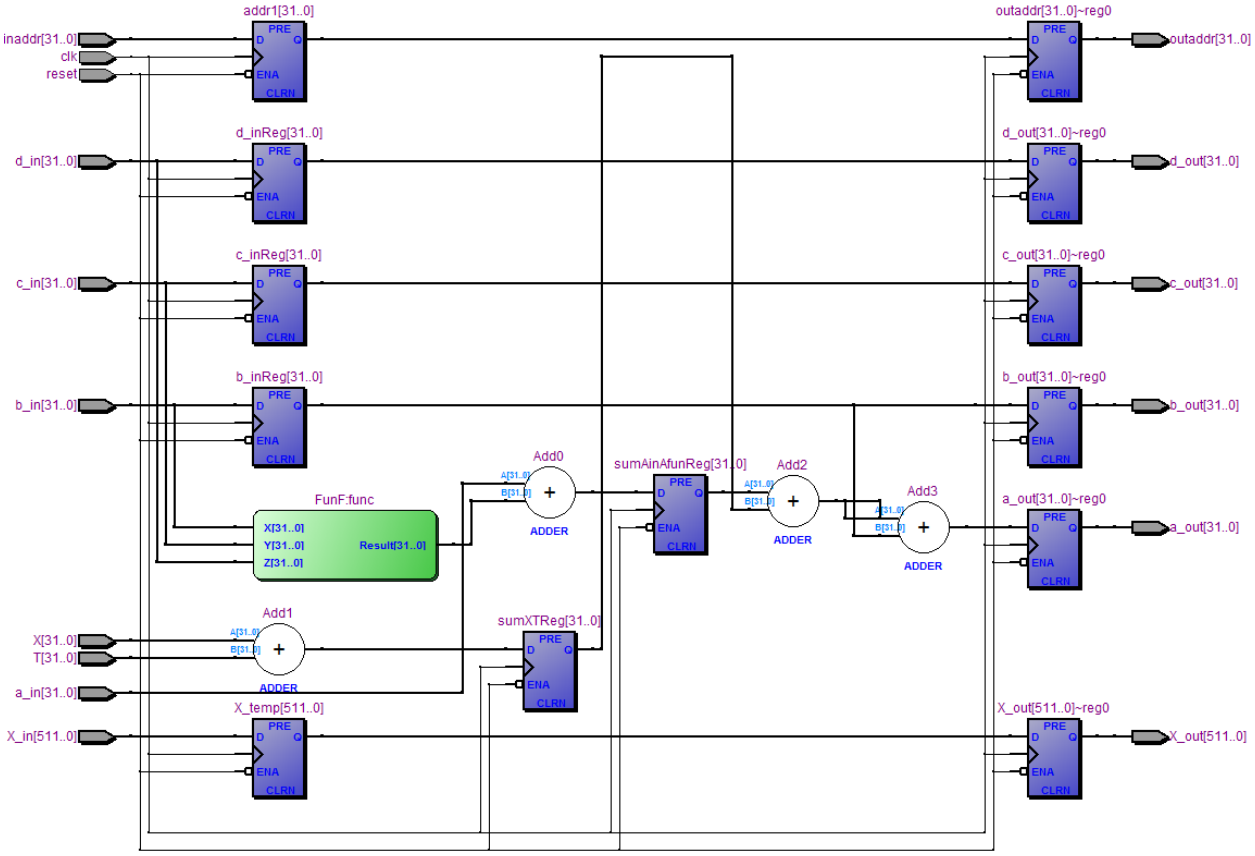


Рис. 6.9. Структура модуля «FunFF_7»

6.1.3. Реализация генератора строк

Генератор строк служит для перебора всевозможных строк заданного алфавита и их подачи на входы устройств вычисления хэш-суммы. Интерфейс данного модуля представлен на рисунке 6.10. На вход подаются сигналы синхронизации clk и reset, ограничение на максимальную длину генерируемых строк. На выходе – текущее состояние генератора; текущая длина перебираемых строк; сгенерированные строки для каждого из четырех вычислителей хэш-суммы.

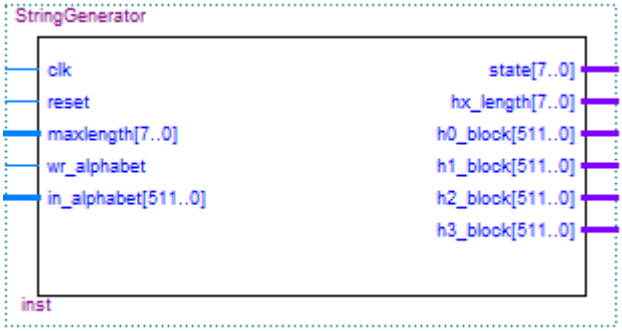


Рис. 6.10. Интерфейс модуля «StringGenerator»

6.1.4. Реализация блока сравнения

Блок сравнения сопоставляет данные, полученные от блоков хэширования, с искомой хэш-суммой. Интерфейс данного модуля представлен на рисунке 6.11. Входные данные: сигналы синхронизации clk и reset; искомое значение хэш-суммы, полученное от устройства управления; значения хэш-сумм, строк, для которых получены данные хэш-суммы, и длина этих строк подаются с устройств хэширования. Выходные данные: сигналы успешного сопоставления строк; строки и их длина. Все выходные сигналы подсоединяются к устройству управления.

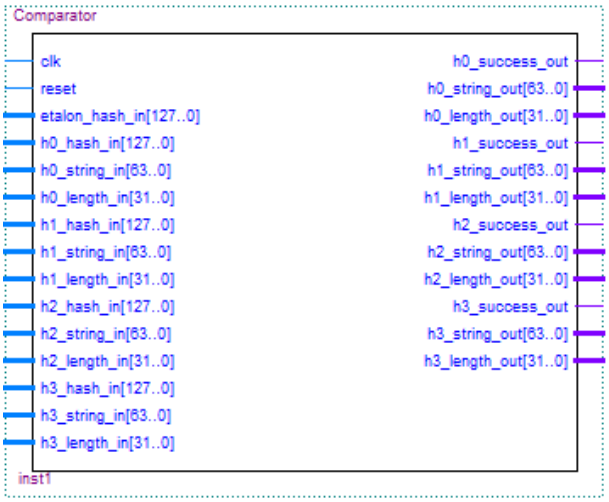


Рис. 6.11. Интерфейс модуля «Comparator»

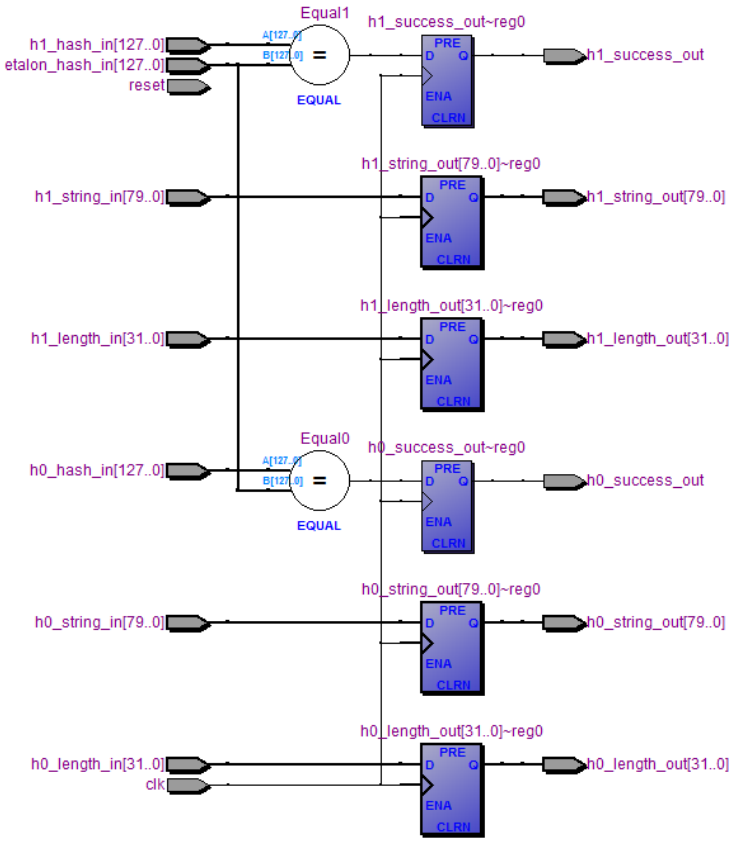


Рис. 6.12. Структура модуля "Comparator"

Структура модуля «Comparator» представлена на рисунке 6.12. Из рисунка видно, что для каждого из полученных хэшей происходит сравнение с искомым, а затем все данные сохраняются в регистрах на выходе. Сохранение данных в регистрах необходимо для организации конвейерной архитектуры.

6.1.5. Реализация управляющего устройства

Управляющее устройство объединяет все разработанные в предыдущих разделах модули в единую систему, решающую задачу восстановления строки по MD5-хэшу. Интерфейс данного блока представлен на рисунке 6.13.

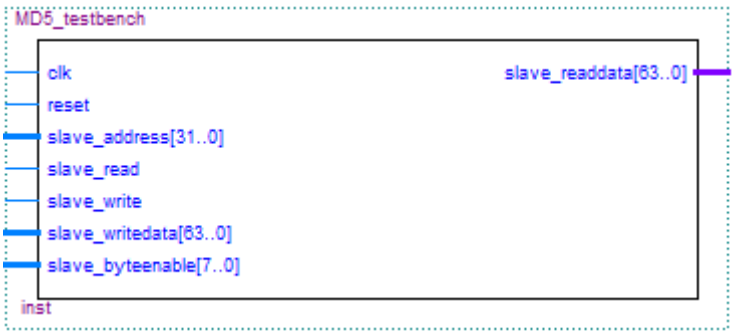


Рис. 6.13. Интерфейс модуля устройства управления

Как видно из рисунка 6.13, устройство управления реализует интерфейс Avalon Memory Mapped Slave. Таким образом, данный блок можно будет использовать в проектах QSys и, в частности, в схемах систем взаимодействия, разработанных в главе 3.

Управляющие регистры и регистры, хранящие исходные данные, настройки и результат, представлены в таблице 6.1.

Таблица 6.1

Регистры устройства управления

Название регистра	Назначение	Разрядность, бит	Доступ	Адрес для доступа через интерфейс Avalon Slave
alphabet	алфавит	64 x 8	запись чтение	0x00 – 0x07
wr_alphabet	флаг инициализации генератора строк	1	запись	0x08
etalon_hash_in	хэш-сумма искомой строки	64 x 2	чтение запись	0x09 0x0A
is_ready	флаг готовности результата	1	чтение	0x0B
result_string	найденная строка	64	чтение	0x0c
result_length	размер найденной строки	32	чтение	0x0d
reset_gen	перезапуск генератора строк	1	запись	0x0e
out_gen_state	текущее состояние генератора строк	1	чтение	0x0f

Записывая данные в представленные выше регистры, можно контролировать работу устройства. Считывая данные регистров, можно получить узнать текущее состояние различных компонентов, а также получить результат поиска.

6.1.6. Результаты экспериментов

В таблицу 6.2 сведены экспериментальные результаты поиска строк с использованием разработанного устройства.

Таблица 6.2

Экспериментальные результаты

Строка	Длина	Время поиска, с		Ускорение
		CPU	FPGA	
Ya93Z	5	4,58	1,76	2,60
dmtset	6	118,85	9,03	13,16
2_1cba	6	1592,24	120,27	12,57
brutfo	6	426,29	32,41	13,15
ab123ba	7	3565,24	278,97	12,78

Параметры тестирования представлены в таблице 6.3.

Таблица 6.3

Параметры тестирования

Параметр	Значение
Центральный процессор	Intel Core i7-920 (4 ядра, 2,66 ГГц)
FPGA	Altera Arria II GX EP2AGX125
Алфавит подбора	Английский алфавит в верхнем и нижнем регистрах, цифры и символы ‘-‘ и ‘_’ (64 символа)
Количество потоков центрального процессора	4 потока

Как видно из таблицы 6.2, среднее ускорение для 6 символьных строк составляет 12,92 раз. Ускорение для различных строк остается постоянным ввиду использования идентичных алгоритмов перебора строк

в версиях для CPU и FPGA. Ускорение для 5 символьной строки составило всего 2,6 раза. Такое отличие обуславливается тем, что время непосредственного поиска 5 символьной строки занимает время, сопоставимое с временем пересылок данных между FPGA и CPU. Таким образом, в данном случае время коммуникаций между сопроцессором и CPU составляет значительную долю от всего времени выполнения задачи.

Определим производительность разработанного устройства в количестве хэш-сумм, которые способна генерировать схема за одну секунду. С помощью анализа средства Quartus II определено, что реализованная схема может работать на частоте до 150 МГц. При разработке системы QSys на устройство подается сигнал тактовых импульсов от генератора на 125 МГц. Т.е. при наличии 4 конвейеров производительность схемы должна равняться 500000000 хэш-сумм/с. Проведем следующий эксперимент. Замеряем время нахождения строки «------». В разработанном алгоритме генерации строк, данная строка проверяется последней из 6 символьных строк. Т.е. к моменту нахождения данной строки устройство обработает следующее количество хэш-сумм: $(64^1+64^2+64^3+64^4+64^5+64^6 - 4) = 69810262076$. Эксперимент показал, что для нахождения данной строки по ее хэшу устройству понадобилось 145,56 секунд. Таким образом, производительность устройства в данном случае составляет: (69810262076 хэш-сумм) / (145,56 с) = 479597843,34 хэш-сумм/с. Различие с теоретической оценкой можно объяснить не учитываемым временем загрузки конвейера и временем передачи начальных данных устройству.

6.1.7. Практическая значимость разработки

Одной из областей, предоставляющих возможности эффективного применения параллельных и конвейерных вычислений с использованием

FPGA, является криптоанализ, в частности, задачи, использующие полный перебор возможных вариантов. В качестве задачи для разработки был выбран поиск исходной строки по её MD5-хэшу. В силу достаточной популярности использования именно алгоритма хэширования MD5 для хранения паролей (например, в интернет-пейджере QIP и многих системах управления содержимым веб-сайтов), данный аппаратно-программный комплекс может быть использован для их восстановления, а также выяснения времени подбора. Актуальность этого направления подтверждает также наличие и коммерческих (PasswordsPro и Elcomsoft Password Recovery Bundle), и бесплатных (BarsWF и Elcomsoft LHC) программ со схожими функциональными возможностями. Для ускорения поиска ряд данных пакетов позволяет использовать графические акселераторы в качестве сопроцессора. В данной же работе исследуются возможности применения реконфигурируемых вычислителей. Проведем сравнительный анализ показателей ускорения по сравнению с временем решения данной задачи на CPU для реконфигурируемого и графического ускорителя [87]. Результаты экспериментов приведены в таблице 6.4.

Таблица 6.4

Сравнение производительности FPGA и GPU

Строка	Длина	Время поиска, с			Ускорение	
		CPU	GPU	FPGA	FPGA/CPU	FPGA/GPU
Mdfive	6	148,63	12,86	11,48	12,95	1,12
ls23jF	6	911,43	75,15	69,09	13,19	1,09
slKd6j	6	307,39	25,74	23,46	13,10	1,10

Параметры тестирования приведены в таблице 6.5.

Из результатов экспериментов видно, что для используемого в качестве сопроцессоров оборудования, получены близкие показатели ускорения. Но если рассматривать показатели энергопотребления, то для GPU значение данного показателя будет значительно выше.

Таблица 6.5

Параметры тестирования

Параметр	Значение
Центральный процессор	Intel Core i7-920 (4 ядра, 2,66 ГГц)
FPGA	Altera Arria II GX EP2AGX125
Видеокарта	NVIDIA GeForce GTX 260
Алфавит подбора	Английский алфавит в верхнем и нижнем регистрах, цифры и символы ‘-‘ и ‘_’ (64 символа)
Количество потоков центрального процессора	4 потока

Помимо всего выше сказанного, несомненной практической значимостью данной разработки является возможность повторного использования отдельных аппаратных модулей для решения других задач. Например, конвейерный блок, осуществляющий вычисление хэша для входной строки может быть использован в других задачах, где требуется вычисление MD5 хэш-суммы. Это может быть аппаратный комплекс, реализующий алгоритмы, использующиеся в SSL (Secure Sockets Layer — уровень защищённых сокетов). SSL представляет собой криптографический протокол, который обеспечивает установление безопасного соединения между клиентом и сервером. Помимо MD5 SSL включается в себя такие алгоритмы, как DES, Triple DES, MD4, MD2, RC2, RC4 и другие, которые в свою очередь также возможно эффективно реализовать на реконфигурируемой архитектуре.

6.2. Реализация отдельных режимов криптоалгоритма ГОСТ

В данном разделе рассмотрена реализация некоторых важных фрагментов режимов криптоалгоритма ГОСТ. Необходимо отметить, что реализация не выполнялась в полном объеме, так как целью работы являлось сравнительное исследование производительности, а не

построение работоспособной криптографической системы. По этой же причине во всех реализациях была выбрана разрядность ключа 32 бита.

Для реализации режимов простой замены, гаммирования и гаммирования с обратной связью на FPGA был выбран язык описания аппаратуры Verilog HDL. Такой выбор обусловлен сложностью адаптации модулей, полученных с помощью высокоуровневых средств.

Реализация выполнена в рамках магистерского исследования Ю.В. Шафрана под руководством А.Е. Андреева.

6.2.1. Реализация режима простой замены на FPGA

В качестве реализации режима простой замены алгоритма ГОСТ 28147-89 (с 32-битным ключом, используемым во всех раундах) на FPGA были разработаны модули SS28147SboxMux, SS28147SboxPhase, SS28147AddressRound и SS28147SimpleReplacementPipeline, изображения интерфейсов которых приведены на рисунках 6.14, 6.15, 6.16 и 6.17.

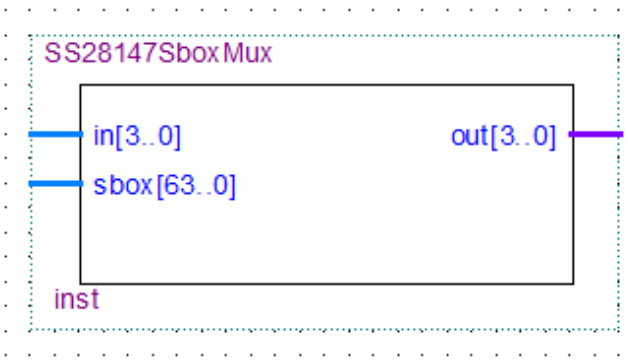


Рис. 6.14. Интерфейс SS28147SboxMux

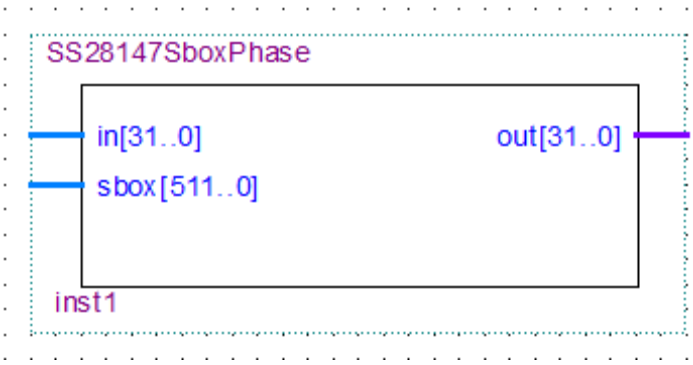


Рис. 6.15. Интерфейс SS28147SboxPhase

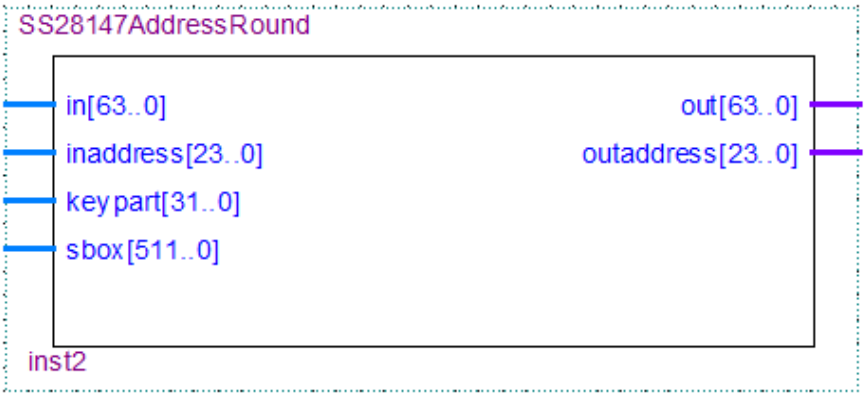


Рис. 6.16. Интерфейс SS28147AddressRound

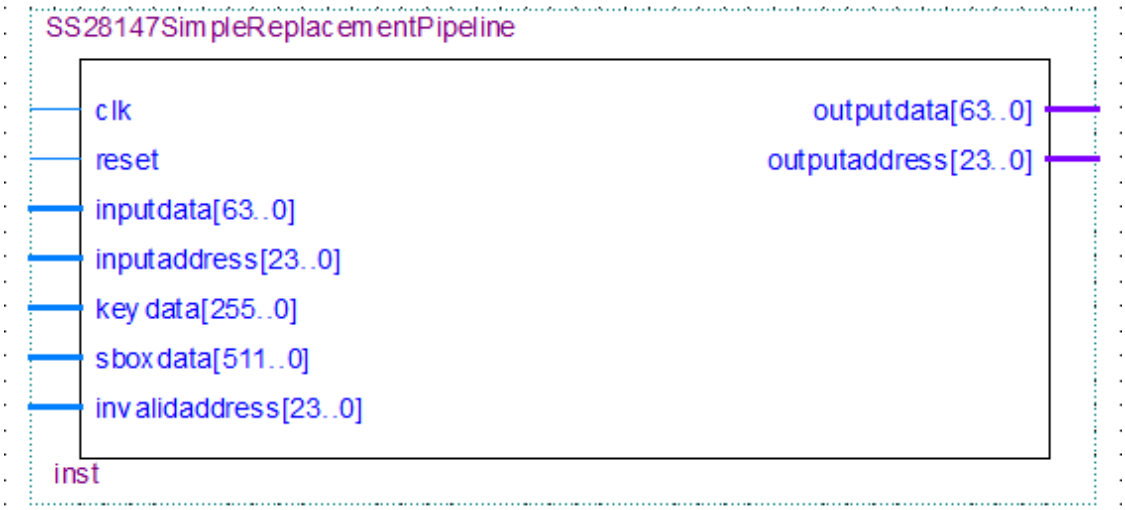


Рис. 6.17. Интерфейс SS28147SimpleReplacementPipeline

Модуль шифрования SS28147SimpleReplacementPipeline для режима простой замены представляет собой устройство, производящее конвейерную обработку блоков исходных данных. На вход модуля должна

подаваться ключевая информация и таблица подстановки. После запуска модуль на каждом такте получает на вход сопровождаемый адресом блок исходных данных. Полученный блок вместе со своим адресом подаётся на вход конвейера шифрования, который состоит из 32 ступеней (по количеству раундов шифрования ГОСТ 28147-89). Через 32 такта обработанный (зашифрованный) блок покидает конвейер и выдаётся на выход схемы также вместе с идентифицирующим его адресом.

6.2.2. Реализация режимов гаммирования на FPGA

Для реализации режимов гаммирования и гаммирования с обратной связью алгоритма ГОСТ 28147-89 (с 32-битным ключом, используемом во всех раундах) на FPGA использованы модули SS28147SboxMux и SS28147SboxPhase, а также разработаны модули SS28147Round, SS28147Rounds и SS28147XorModesTunedRounds, изображения интерфейсов которых приведены на рисунках 6.18, 6.19 и 6.20 соответственно.

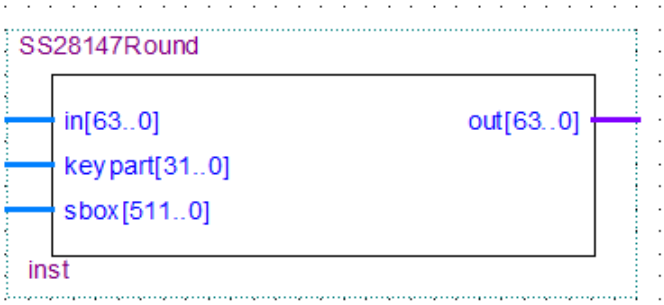


Рис. 6.18. Интерфейс модуля SS28147Round

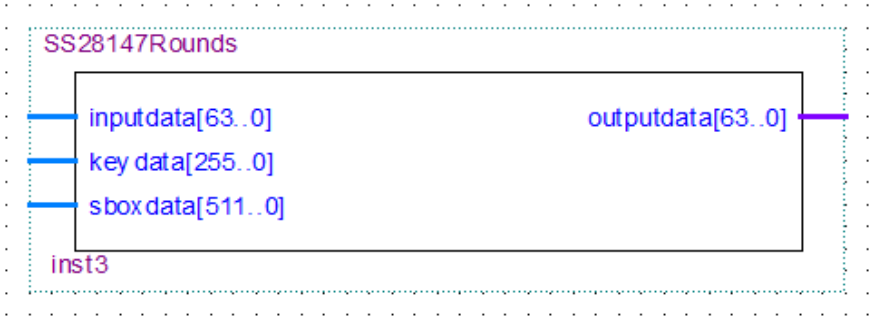


Рис. 6.19. Интерфейс модуля SS28147Rounds

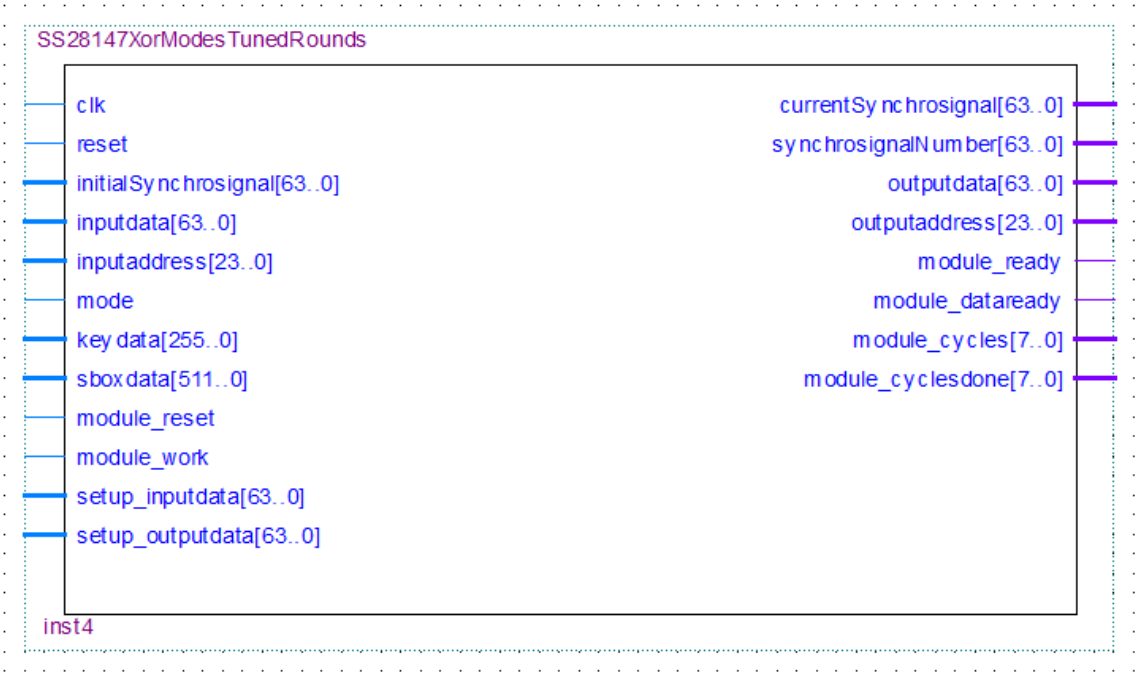


Рис. 6.20. Интерфейс модуля SS28147XorModesTunedRounds

Модуль шифрования для режимов гаммирования и гаммирования с обратной связью SS28147XorModesTunedRounds при запуске схемы по подаваемым значениям ключевой информации, таблицы подстановки, исходного и конечного настроечных блоков данных подсчитывает количество тактов, необходимое для шифрования одного блока. После этого устройство получает на вход сопровождаемый адресом блок исходных данных и производит его шифрование. По завершении шифрования выдаётся сигнал о готовности выходных данных и загружается новый блок исходных данных.

6.2.3. Результаты реализации алгоритма шифрования ГОСТ 28147-89 (с 32-битным ключом) в режиме простой замены на GPU и FPGA

Для использования разработанного модуля шифрования по алгоритму ГОСТ 28147-89 (с 32-битным ключом) в режиме простой замены в составе системы для средства разработки Altera Qsys,

предназначенного для создания систем на чипе, необходимо адаптировать его к интерфейсу Avalon Memory Mapped. С этой целью разработан модуль SS28147SimpleReplacement, интерфейс которого представлен на рисунках 6.21 и 6.22. Описание сигналов интерфейса Avalon Memory Mapped приведено в спецификации [31].

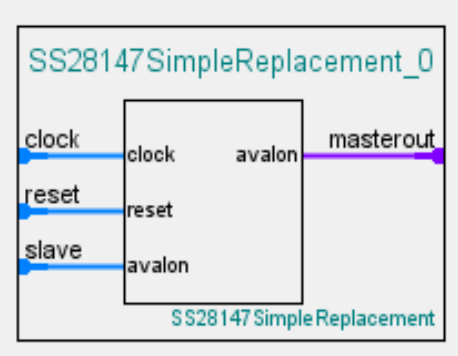


Рис. 6.21. Интерфейс модуля SS28147SimpleReplacement

Входные данные для шифрования подаются в модуль по адресам от 0 до FFFFCF и по этим же адресам выходные данные записываются в память.

Схема разработанной системы Qsys приведена на рисунке 6.23.

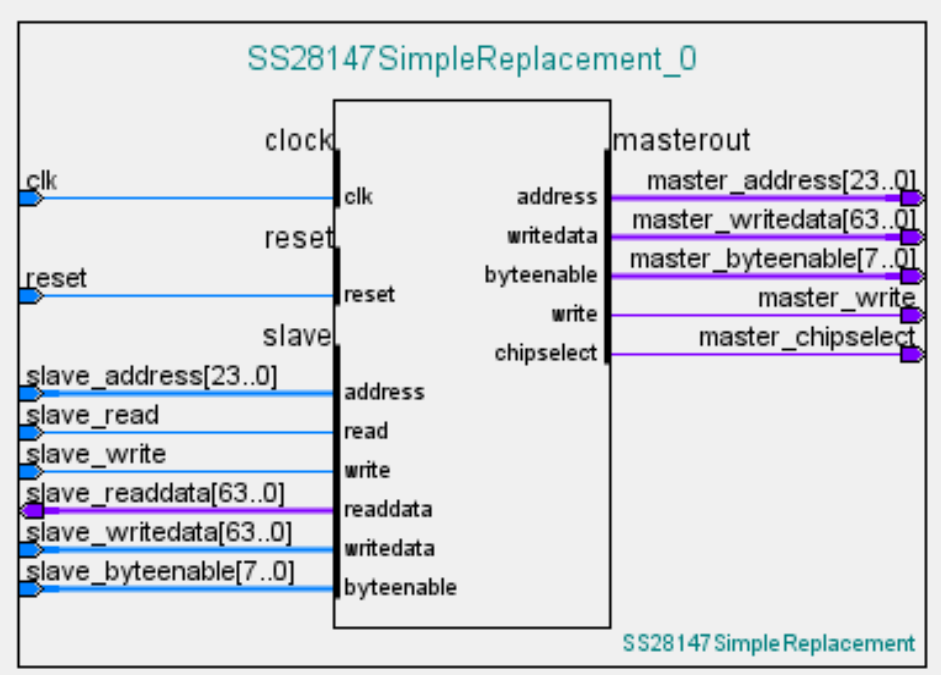


Рис. 6.22. Полный интерфейс модуля SS28147SimpleReplacement

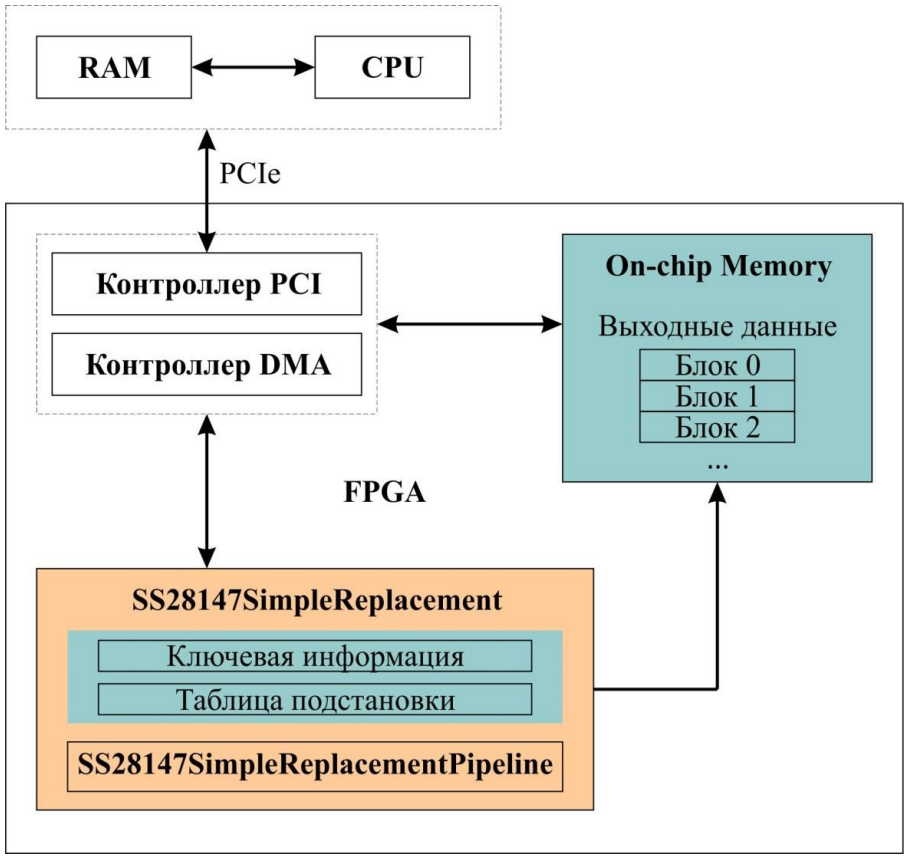


Рис. 6.23. Система для режима простой замены

Алгоритм работы с устройством представлен в таблице 6.6.

Таблица 6.6

Алгоритм работы в режиме простой замены

Стадия	Операция	Описание
Инициализация	Передача ключевой информации	Запись ключевой информации в соответствующие регистры SS28147SimpleReplacement.
	Передача таблицы подстановок	Запись таблицы подстановок в соответствующие регистры SS28147SimpleReplacement.
Шифрование	Передача исходных данных	Запись исходных данных по адресам от 0 до FFFFCF из модуля SS28147SimpleReplacement.
	Чтение зашифрованных данных	Чтение зашифрованных данных по адресам, использованным при передаче исходных данных, из памяти выходных данных.

6.2.3.1. Сравнение экспериментальной производительности с прогнозируемой

Использование разработанной системы из программы, выполняемой на CPU, осуществлялось с использованием библиотек Jungo WinDriver, обеспечивающих взаимодействие с устройствами, подключенными к шине PCI.

Сравнение экспериментальной и прогнозируемой производительностей для реализаций режима простой замены по алгоритму ГОСТ 28147-89 (с 32-битным ключом) приведено в таблице 6.7. Оно показывает приемлемую точность оценки производительности по методике RAT как для реализаций на GPU, так и для реализаций на FPGA. Максимальная разница прогнозируемой и экспериментальной производительностей составила не более 8%. С помощью всех сопроцессоров (и GPU, и FPGA) получено значительное ускорение.

Таблица 6.7

Сравнение результатов для реализаций режима простой замены

Устройство	GPU			FPGA
	NVIDIA GT 335M	NVIDIA GTX 260	NVIDIA Tesla C1060	Altera Arria II GX EP2AGX125
$N_{elements}$	134 217 728			
$N_{bytes/elements}$, байт	8			
$t_{RC_{DB}}$, с	8,50	2,21	1,91	2,74
t_{comp} (CPU Intel Core i3-2125), с	24,07			
Расчётное ускорение (двойная буферизация)	2,83	10,89	12,60	8,78
Экспериментальное $t_{RC_{DB}}$, с	7,90	2,34	1,99	2,88
Экспериментальное ускорение (двойная буферизация)	3,05	10,29	12,10	8,36
Разница с оценкой, %	7,59	5,56	4,02	4,86

6.2.4. Результаты реализации алгоритма шифрования ГОСТ 28147-89 (с 32-битным ключом) в режимах гаммирования на FPGA

6.2.4.1. Адаптация реализации на FPGA для интерфейсов Avalon

Для использования разработанного модуля шифрования по алгоритму ГОСТ 28147-89 (с 32-битным ключом) в режиме простой замены в составе системы для средства разработки Qsys необходимо адаптировать его к интерфейсу Avalon Memory Mapped. С этой целью разработан модуль SS28147XorModes, интерфейс которого представлен на рисунках 6.24 и 6.25. Описание сигналов приведено в таблице 6.8. Описание сигналов интерфейса Avalon Memory Mapped приведено в спецификации [31].

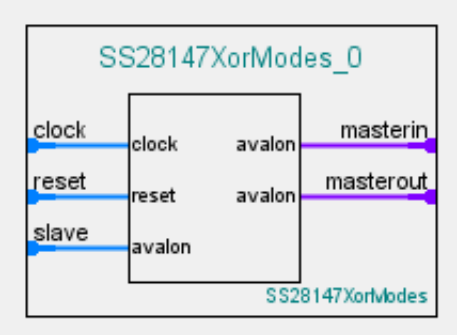


Рис. 6.24. Интерфейс модуля SS28147XorModes

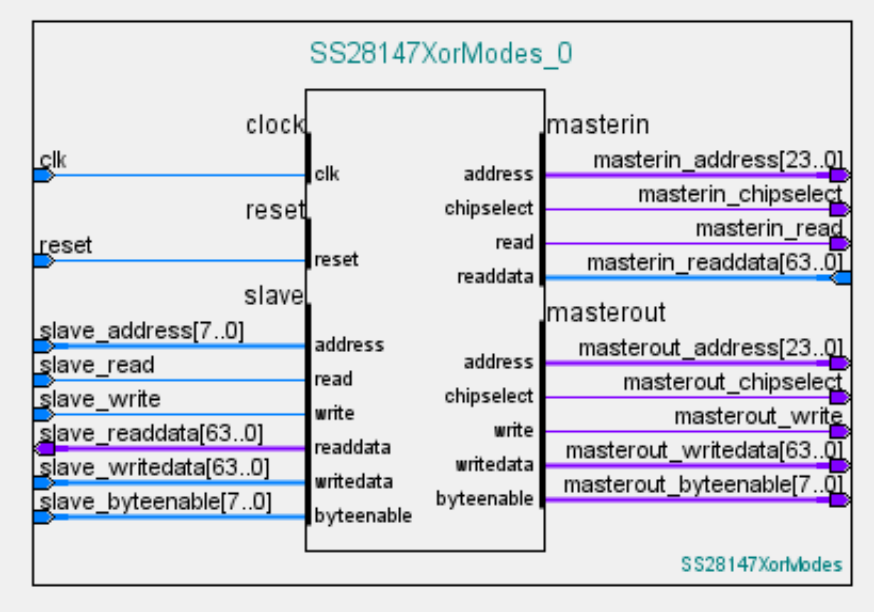


Рис. 6.25. Полный интерфейс модуля SS28147XorModes

Таблица 6.8

Описание сигналов SS28147XorModes

Сигнал	Вид	Размер,бит	Описание
Clk	Вход	1	Тактовый сигнал.
Reset	Вход	1	Сигнал перезапуска.
Masterin	Avalon Memory Mapped Master		Используется для подключения модуля шифрования к памяти на чипе или контроллеру внешней памяти, из которой модуль читает входные блоки данных.
Masterout	Avalon Memory Mapped Master		Используется для подключения модуля шифрования к памяти на чипе или контроллеру внешней памяти, в которую модуль записывает выходные блоки данных.
Slave	Avalon Memory Mapped Slave		Используется для подключения модуля шифрования к управляющему устройству.

Схема разработанной системы Qsys приведена на рисунке 6.26.

Алгоритм работы с устройством представлен в таблице 6.9.

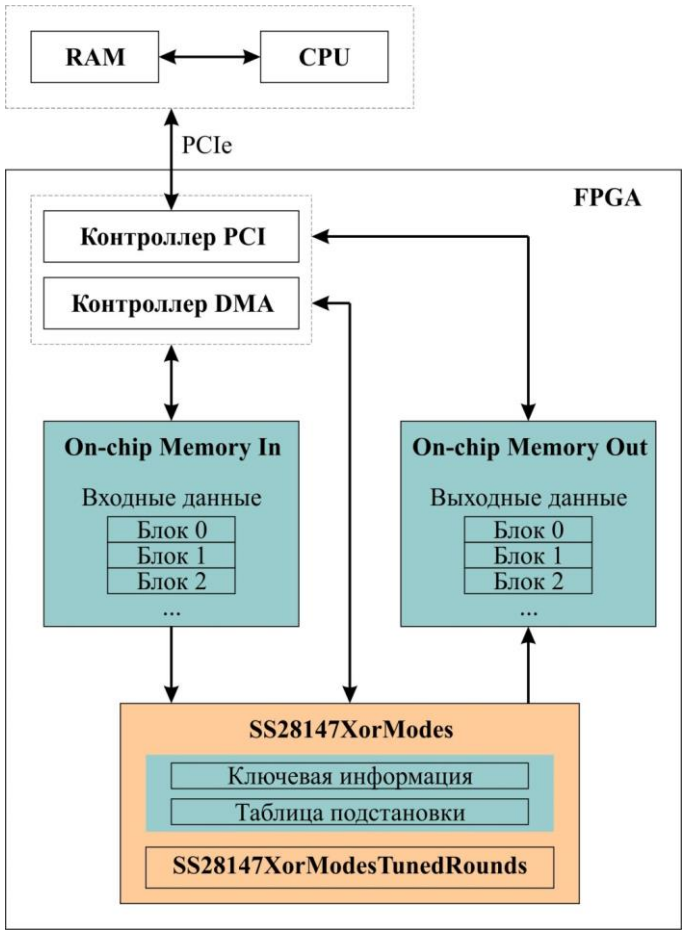


Рис. 6.26. Система для режимов гаммирования

Таблица 6.9

Алгоритм работы в режимах гаммирования

Стадия	Операция	Описание
Инициали-зация	Передача ключевой информации	Запись ключевой информации в соответствующие регистры SS28147XorModes.
	Передача таблицы подстановок	Запись таблицы подстановок в соответствующие регистры SS28147XorModes.
	Передача настроечных блоков	Запись настроечного входного блока и соответствующего ему настроечного выходного блока в соответствующие регистры SS28147XorModes.
	Отключение модуля	Обнуление сигнала работы модуля SS28147XorModes.
	Перезапуск модуля	Обнуление номера текущего состояния SS28147XorModes.
	Ожидание готовности модуля	Ожидание текущего состояния SS28147XorModes, равного 4.
	Выбор режима работы	Передача значения режима работы SS28147XorModes.
	Задание начального адреса	Передача в SS28147XorModes адреса текущего блока входных данных, соответствующего первому блоку шифруемых данных в памяти входных данных.
	Запись синхропосылки	Передача в SS28147XorModes первоначальной синхропосылки.
Шифрова-ние	Установка флага продолжения шифрования	Передача в SS28147XorModes флага продолжения шифрования данных: 0 – в случае, если необходимо загрузить синхропосылку из регистра первоначальной синхропосылки; 1 – в случае если необходимо продолжать шифрование с текущим значением синхропосылки.
	Передача объёма исходных данных	Передача в SS28147XorModes количества блоков данных, которое необходимо зашифровать.
	Передача исходных данных	Запись исходных данных по адресам память входных данных.
	Запуск шифрования	Установка сигнала работы модуля SS28147XorModes.
	Ожидание завершения шифрования	Ожидание текущего состояния SS28147XorModes, равного 11.
	Отключение модуля	Обнуление сигнала работы модуля SS28147XorModes.
	Перевод модуля в состояние готовности	Передача 4 в SS28147XorModes в качестве номера текущего состояния.
	Чтение зашифрованных данных	Чтение зашифрованных из памяти выходных данных.

6.2.4.2. Сравнение экспериментальной производительности с прогнозируемой

Использование разработанной системы из программы, выполняемой на CPU, осуществлялось с использованием библиотек Jungo WinDriver. Сравнение экспериментальной и прогнозируемой производительностей для реализаций режимов гаммирования и гаммирования с обратной связью по алгоритму ГОСТ 28147-89 (с 32-битным ключом) приведено в таблице 6.10. Оно подтверждает обнаруженную при реализации режима простой замены ГОСТ 28147-89 (с 32-битным ключом) приемлемую точность оценки производительности по методике RAT для реализаций на FPGA. Разница прогнозируемой и экспериментальной производительностей снова составила не более 8%. С помощью FPGA получено приемлемое ускорение. Большего ускорения можно достичь при использовании реализации с двойной буферизацией.

Проведённые эксперименты показали, что на частоте 125МГц 32 раунда шифрования по алгоритму ГОСТ 28147-89 (с 32-битным ключом) срабатывают за 10 тактов.

Таблица 6.10

Сравнение результатов для реализации режимов гаммирования

Устройство	FPGA Altera Arria II GX EP2AGX125			
$N_{elements}$	134 217 728			
$N_{bytes/elements}$, байт	8			
Количество тактов, необходимых для шифрования одного блока данных	8	9	10	11
$t_{RC_{SB}}$, с	11,33	12,40	13,48	14,55
t_{comp} (CPU Intel Core i3-2125), с	24,66			
Расчётное ускорение (одиночная буферизация)	2,18	1,99	1,83	1,69
Экспериментальное $t_{RC_{SB}}$, с			14,55	
Экспериментальное ускорение (одиночная буферизация)			1,69	
Разница с оценкой, %			7,39	

6.3 Выводы

В данной главе рассмотрены реализации двух задач из области криптологии на ГВС РС: поиск строки по MD5 хэш-сумме и реализация фрагментов криптографического алгоритма ГОСТ. Обе реализации не предназначены для практического применения как средства криптозащиты, только для сравнительной оценки параметров реализации на реконфигурируемом сопроцессоре с реализациями на CPU или GPU, поэтому выполнялись не в полном объеме, а криптографический алгоритм - с использованием небольших ключей.

Полученные результаты подтверждают эффективность использования реконфигурируемых сопроцессоров даже оснащенных ПЛИС среднего уровня, а также демонстрируют достаточно высокую степень соответствия полученных результатов оценочным параметрам реализации, полученным во второй главе при предварительной оценке целесообразности реализации алгоритмов на ПЛИС.

Глава 7

РЕАЛИЗАЦИЯ НА ПЛИС ПРОТОТИПА ТЕПЛОВОГО РАСЧЕТА ДЛЯ СИСТЕМЫ МЕЖДИСЦИПЛИНАРНОГО ИНЖЕНЕРНОГО АНАЛИЗА

В заключительной главе рассматривается прототип теплового расчета для системы междисциплинарного инженерного анализа, реализованный в рамках магистерского исследования М.А. Оболониным под руководством А.Е. Андреева [88].

7.1 Постановка задачи теплового расчета

В процессе междисциплинарного моделирования движения автомобиля по неровной поверхности требуется решить ряд задач, относящихся к различным областям физики, но связанных между собой. Моделирование движения автомобиля осуществляется с помощью уравнений динамики системы связанных тел, основанных на уравнениях Лагранжа первого рода. Уравнения динамики механической системы слагаются из уравнений свободного движения тел и уравнений связей от кинематических пар и упруго демпфирующих элементов, соединяющих тела. В процессе своего движения механическая энергия, возникающая в демпфирующем элементе автомобиля, переходит в тепловую, нагревая жидкость в амортизаторе. Также при этом воздействии рычаги подвески испытывают большие динамические нагрузки, поэтому при проектировании подвески автомобиля актуальным является определение оптимальных параметров ее работы в различных режимах.

Для решения данной задачи требуется решить задачи, связанные с моделированием динамики системы связанных тел, процесса превращения

механической энергии в тепло в амортизаторе, а также определения напряжений в деталях подвески автомобиля. В данной главе представлено решение задачи расчета теплового процесса в амортизаторе автомобиля с использованием ПЛИС.

Решение междисциплинарной задачи, включающей расчет динамики системы тел, а также расчет теплопроводности носит итерационный характер. На каждом шаге расчета модели определяется положение амортизатора, на основе которого рассчитывается работа, совершенная демпфером. Затем, считая, что вся энергия идет на равномерный нагрев жидкости амортизатора, можно определить ее среднюю температуру, которая будет использоваться при решении задачи нестационарной теплопроводности в качестве граничных условий с внутренней стороны амортизатора. С внешней части граничным условием будет служить температура воздуха.

Технология расчета состоит в использовании нестационарной дискретной модели, аппроксимирующей решение уравнения теплопроводности для произвольной геометрии с использованием кубической сетки для дискретизации геометрии [89-94].

На каждой итерации междисциплинарного расчета происходит интегрирование системы обыкновенных дифференциальных уравнений теплопроводности, соответствующих рассматриваемому изделию методом Рунге Кутты 4го порядка. Дискретная математическая модель соответствует уравнению теплопроводности в частных производных [94]:

$$\gamma\rho\frac{\partial u}{\partial t} - q\nabla^2 u = Q(x,y,z,t), \tag{7.1}$$

где u – температура, q – коэффициент теплопроводности материала, γ – теплоемкость, ρ – плотность, с начальными условиями

$$u\,(x,y,z,0) = f(x,y,z), \tag{7.2}$$

и граничными условиями

$$u(x, y, z, t)|_{\Gamma} = T_b, \tag{7.3}$$

Приближенное значение температуры в каждой точке будет вычисляться в 4 этапа по формуле:

$$y(n+1) = y(n) + \frac{k_1 + 2k_3 + 2k_3 + k_4}{6}, \tag{7.4}$$

В рассматриваемой реализации метода предполагается, что решетка разбиения – кубическая, с кубиками одинакового размера с длиной ребра h.

7.2 Реализация алгоритма расчета распределения тепла на ПЛИС

Последовательное вычисление четырех этапов алгоритма (7.4) для каждой точки расчетной сетки занимает значительное время, линейно увеличивающееся с ростом размера сетки. При использовании сеток большой размерности, время работы алгоритма на каждой итерации междисциплинарного расчета будет занимать большую часть времени, в сравнении с расчетом многотельной модели, поэтому ускорение работы данного алгоритма позволит сократить общее время расчетов.

Выполнение расчета на графических ускорителях с использованием технологии CUDA позволяет получить 5-ти кратное ускорение расчетов в сравнении со временем работы на центральном процессоре. Однако серьезным ограничением для его эффективного использования является необходимость размещать данные для работы алгоритма в памяти ГПУ. При решении задачи, объем данных которой целиком укладывается в объем памяти ГПУ, возможно предварительное копирование массивов данных в память устройства, что позволяет избежать блокировок работы алгоритма для синхронизации результатов. Однако, в случае нехватки

памяти устройства, требуются дополнительные накладные расходы на пересылку недостающих данных. Алгоритм усложняется, и эффективность его использования снижается.

Архитектура ПЛИС предполагает разработку потоковых алгоритмов, минимально использующих внутреннюю память устройства. С учетом того, что объем памяти современных плат на порядок ниже объемов памяти графических ускорителей, применение данного подхода не представляется возможным. Поэтому для использования в качестве вычислительного элемента ПЛИС, данный алгоритм требуется оптимизировать под особенности архитектур данных плат.

Основным подходом к разработке алгоритмов для ПЛИС является конвейеризация вычислительного процесса. Алгоритм Рунге-Кутты выполняется в 4 этапа, перед каждым этапом производится пересчет правой части дифференциальных уравнений для каждой точки расчетной сетки. Для выполнения расчета одного этапа алгоритма, требуется передать устройству значения температуры и ее производной в искомой точке, а также в 6 соседних точках, произвести необходимые вычисления и вернуть обновленное значение температуры в искомой точке. Для решения данной задачи требуется разработать вычислительный блок для каждого этапа алгоритма. В каждом вычислительном блоке организуется конвейер, принимающий данные, производящий расчет и возвращающий результат, по достижению загрузки конвейера результат получается на каждом такте. Между этапами алгоритма требуется синхронизация данных для всех расчетных точек сетки.

На рисунке 7.1 представлена общая схема вычислений температуры каждой точки. Главный процесс, выполняющийся на центральном процессоре, управляет вычислениями на ПЛИС, передавая данные для каждого вычислительного элемента, производящего расчет своего этапа метода Рунге-Кутта.

Расчет каждой точки происходит независимо от остальных, поэтому возможно использование параллельных конвейерных линий для повышения производительности. Алгоритмы этапов метода Рунге-Кутты представлены на рисунках 7.2 – 7.5.

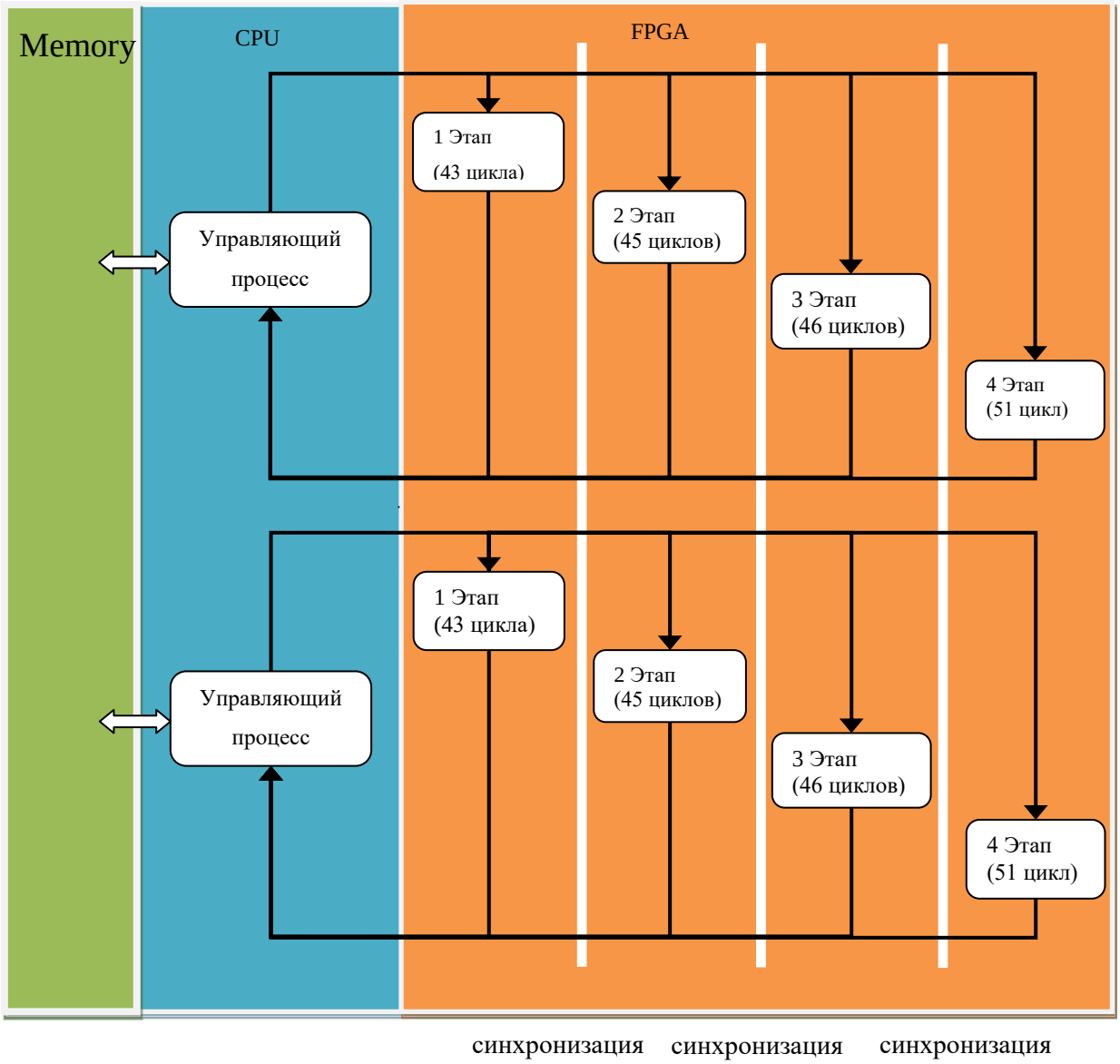


Рис. 7.1 – Схема работы системы

Каждый этап включает в себя стадию чтения, вычисления и записи результатов. Расчетная сетка и данные о температуре каждой ее точки хранится в оперативной памяти, и управляющий процесс передает все необходимые данные для вычисления 1 точки.

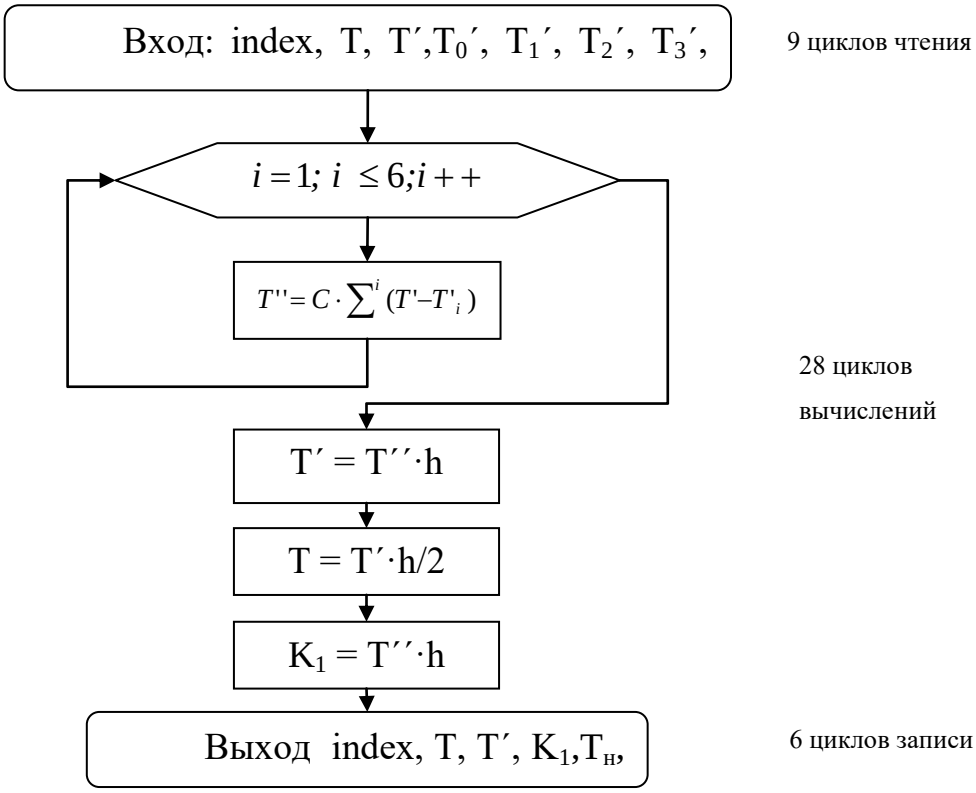


Рис. 7.2 – Алгоритм 1 этапа Рунге-Кутты

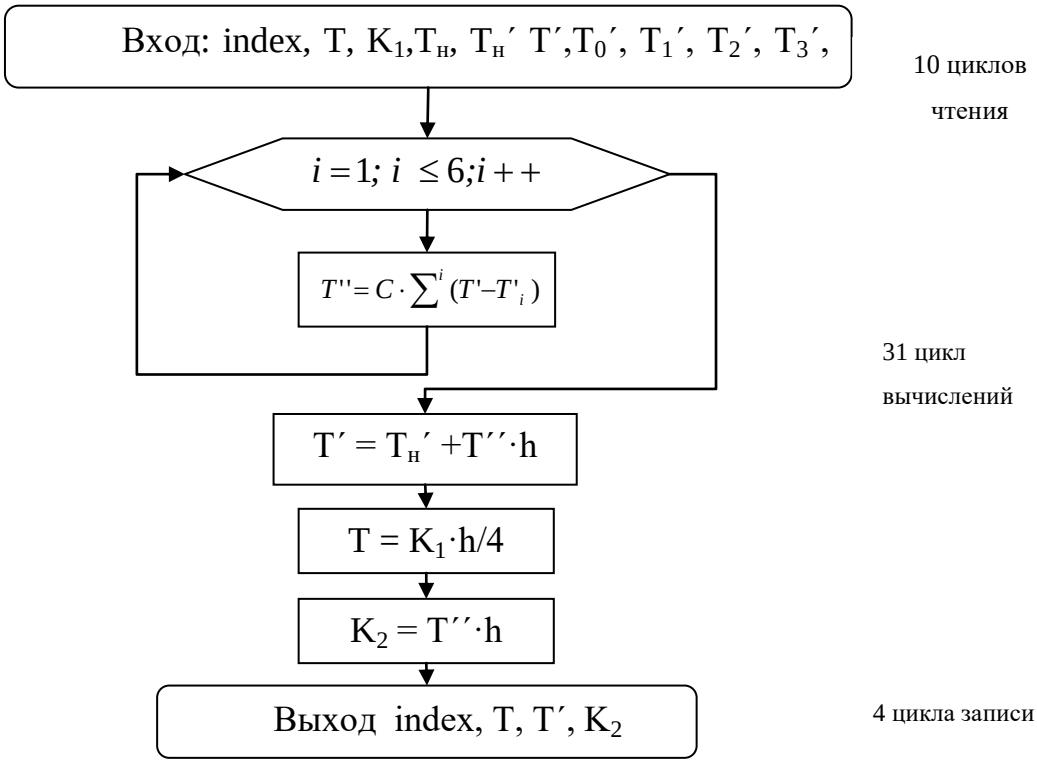


Рис. 7.3 – Алгоритм 2 этапа Рунге-Кутты

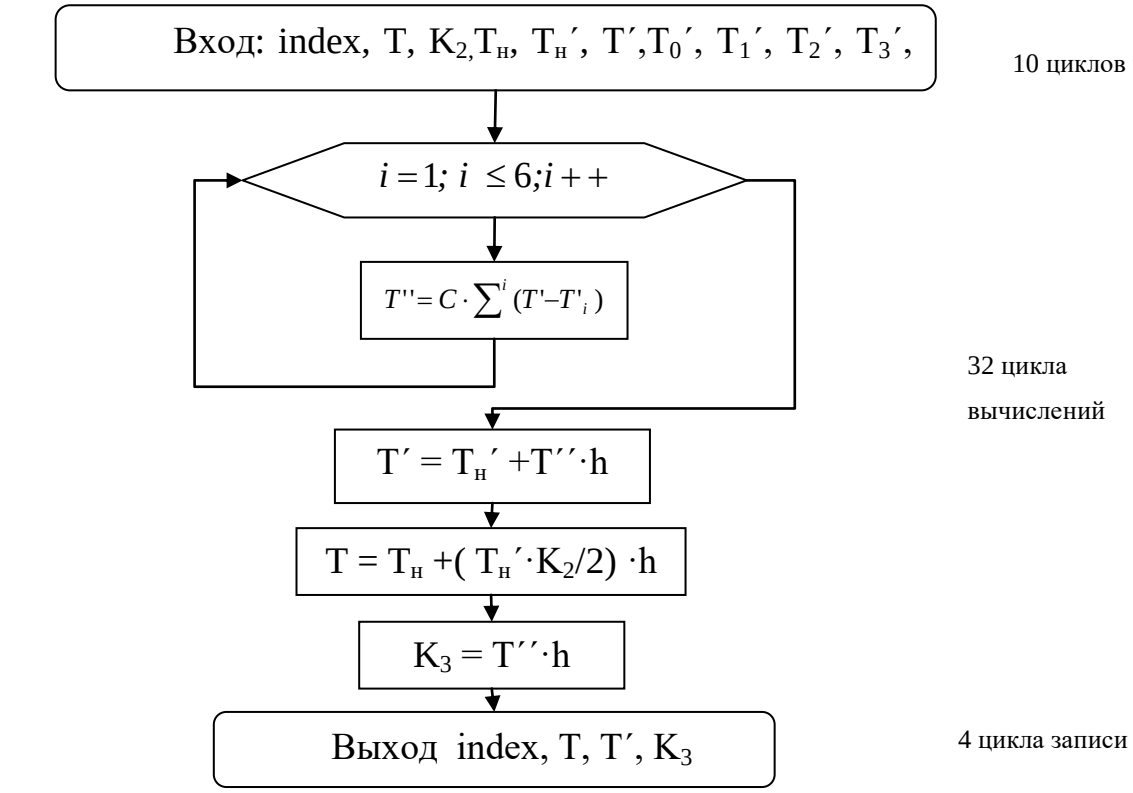


Рис. 7.4 – Алгоритм 3 этапа Рунге-Кутты.

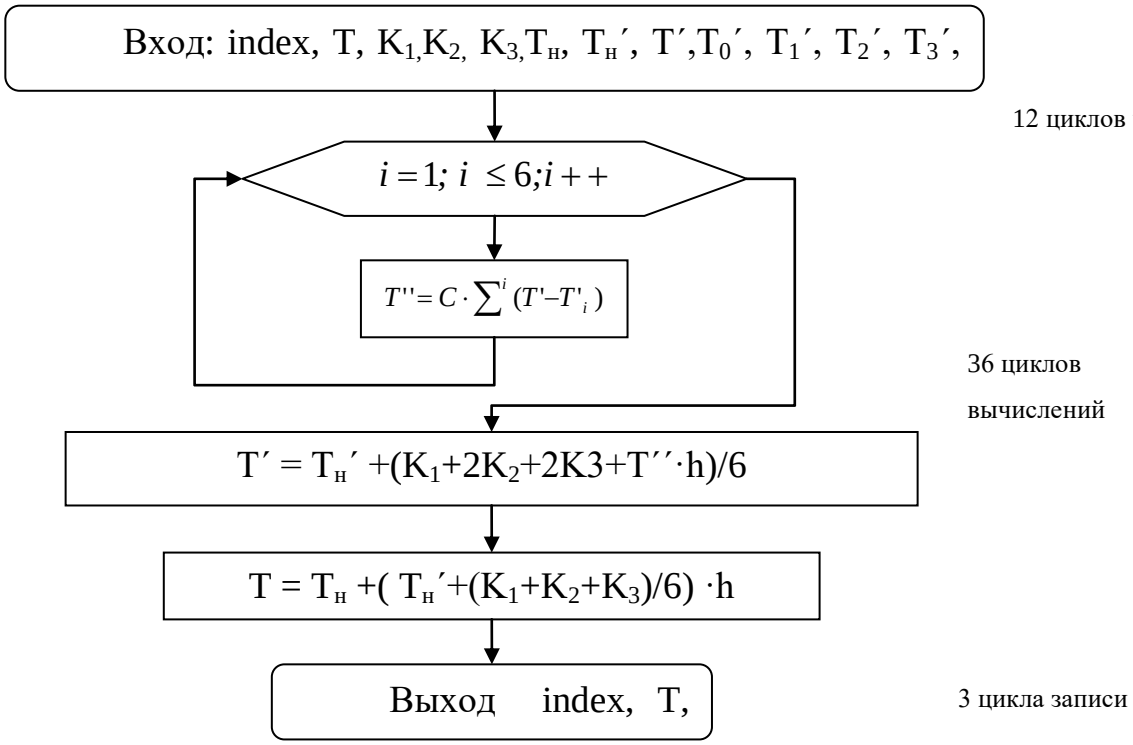


Рис. 7.5 – Алгоритм 4 этапа Рунге-Кутты.

В разработанных вычислительных блоках пересылка данных занимает существенное время, примерно 30% от всего времени расчета, однако конвейеризация этого процесса позволяет уменьшить этот негативный эффект. Каждому вычислительному элементу для расчета текущей точки требуется передать ее индекс, значения температуры T и T′, а также значения T′ шести соседних точек и по окончании расчетов записать новые значения T и T′. Все данные, необходимые для работы алгоритма, хранятся в оперативной памяти, и в каждый вычислительный блок передаются только требуемые для расчета данные, что позволяет преодолеть ограничение встроенной памяти устройства ПЛИС.

7.3 Реализация алгоритма с помощью средства Impulse C

Так как разрабатываемый алгоритм требует эффективной организации передачи данных, то для его реализации были рассмотрены различные средства высокоуровнего синтеза, описанные в работе [35]. На основе данного анализа было выбрано средство высокоуровневой разработки для ПЛИС Impulse C, позволяющее создавать эффективные схемы с множеством потоков, абстрагируя программиста от деталей их создания. Разрабатываемая программа делится на 2 части:

Программная часть – часть программы, управляющая работой схемы. Данный модуль генерирует данные для расчетов и передает их на схему в требуемый процесс. Она состоит из процессов Producer и Consumer. Producer управляет внутренними потоками, соединяющими программную и схемную части, записывает данные для всех этапов в соответствующие потоки, и считывает данные из первых трех этапов. Consumer получает результат всего расчета с выхода четвертого этапа и запускает расчеты для следующего кадра.

Схемная часть – представлена 4 процессами, являющимися реализацией четырех этапов алгоритма. По сигналу от управляющего процесса рабочий блок (в данный момент) начинает считывание входных данных, производит вычисления и возвращает результат в Producer. Эти 4 процесса работают последовательно, синхронизация происходит в управляющем процессе.

На рисунке 7.6 представлена архитектура модуля, скомпилированного в среде Impulse CoDeveloper. Producer и Consumer представляют программную часть, процессы Point0 – Point3 4 этапа алгоритма, реализованного на ПЛИС. Зелеными стрелками указаны потоки данных, соединяющие программную и схемную части. В схеме представлен алгоритм, использующий 1 конвейер для организации вычислений. Производительность данной схемы может быть повышена размещением нескольких параллельных конвейеров путем увеличения количества процессов Point0 – Point3. Однако данные изменения повлекут за собой рост количества логических элементов, требуемых для реализации схемы на кристалле.

7.4 Результаты работы теплового расчета для различных архитектур

Для получения результатов была произведена симуляция работы схемы в ModelSim. Так как при реализации алгоритма производятся вычисления с плавающей точкой, то для симуляции его работы требуется использование библиотеки работы с плавающей точкой. Impulse C поддерживает такую библиотеку только для плат Xilinx, поэтому они были выбраны в качестве вычислительных устройств. Результаты тестирования алгоритма сведены в таблицу 7.1. В таблице представлено сравнение версии теплового расчета для ПЛИС Xilinx семейства Virtex 5 100 МГц в

трех вариантах – с одним конвейером, с двумя параллельно работающими конвейерами и с четырьмя, а также на CPU – Intel Core i7 2700 Mhz, и GPU ATI Radeon HD7670m

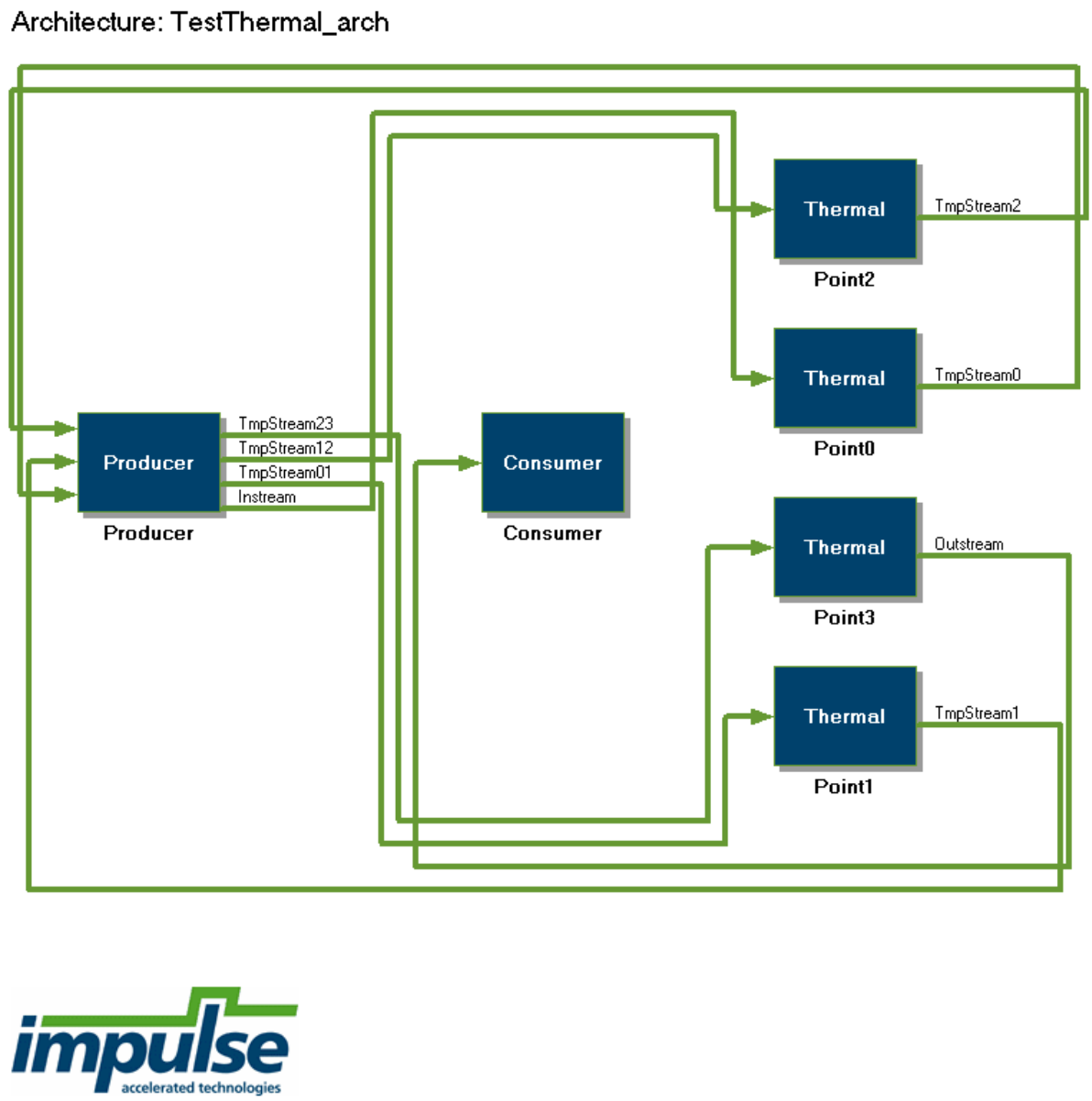


Рис. 7.6 – Архитектура схемы, полученная в Impulse CoDeveloper.

При сравнении самого производительного варианта с 4 параллельными конвейерами с версией, работающей на CPU, было получено ускорение от 1,7 до 1,33 раза, т.е. сокращение на 33-70% времени расчетов одной итерации алгоритма. Однако версия,

разработанная для графического ускорителя, работает, приблизительно, в 6 раз быстрее.

Таблица 7.1

Время работы алгоритма на различных вычислителях

N size	FPGA 1 pipeline,10^{-3}c	FPGA 2 pipeline,10^{-3}c	FPGA 4 pipeline,10^{-3}c	CPU, 10^{-3}c	GPU, 10^{-3}c
10000	7,34	3,66	1,85	3,25	0,53
20000	14,65	7,32	3,74	5,64	0,77
30000	21,91	11,05	5,62	8,32	1,37
50000	36,76	19,12	9,66	13,1	1,52
100000	73,82	36,94	18,67	24,85	2,95

Тесты показали, что четырех параллельно работающих конвейеров достаточно для получения ускорения расчетов относительно центрального процессора, однако при этом увеличивается использование логических элементов, которое достигает 83%. Таким образом, дальнейшее наращивание количества параллельно работающих конвейеров не позволит разместить схему на выбранном устройстве.

Таблица 7.2

Объем ресурсов, требуемый для схемной реализации алгоритма на плате Virtex 5 XC5VLX330T.

	Number of LUT's	Number of I/O buf	Utilization
FPGA 1 pipeline	42076	282	20%
FPGA 2 pipeline	83836	562	41%
FPGA 4 pipeline	172108	947	83%

Однако при размещении 8 конвейеров, данные, передаваемые на плату по шине PCI-Express, уже могут не укладываться в один кадр, пересылаемый в единицу времени по шине. В соответствии с этим, работа конвейеров будет приостанавливаться для ожидания данных, что замедлит работу алгоритма. Поэтому нельзя утверждать, что увеличение

конвейерных линий позволит добиться пропорционального увеличения ускорения. Платы Xilinx нового поколения содержат увеличенное количество LUT, поэтому на таких устройствах можно ожидать увеличения ускорения работы алгоритма, относительно CPU версии. В таблице 7.2 представлены характеристики разработанной схемы для решения задачи расчета теплового процесса. На основе этих данных в Xilinx Power Estimator была получена оценка энергопотребления платы при напряжении V_{ccint} = 1В, тактовой частоте F = 100 МГц – потребляемая энергия W = 2,95 Вт.

7.5 Выводы

С использованием одного из рассмотренных в главе 4 средств проектирования высокого уровня для ПЛИС – Impulse C – построен прототип системы для выполнения теплового расчета амортизатора при движении автомобиля по неровной поверхности, выполняемого в контексте междисциплинарного расчета динамики, нагрева и прочности в системе инженерного анализа.

Показано, что система, выполняющая расчет на ПЛИС, в целом позволяет ускорить расчет по сравнению с CPU, уступая по показателям ускорения графическому сопроцессору, но при этом значительно опережая его по энергоэффективности.

ЗАКЛЮЧЕНИЕ

В рамках данной работы рассмотрен ряд вопросов построения и использования гибридных вычислительных систем на базе штатных рабочих станций и серверов, оснащенных реконфигурируемыми устройствами на базе микросхем программируемой логики (ПЛИС), используемыми в качестве сопроцессоров.

Рассмотрено понятие о гибридных реконфигурируемых вычислительных системах (РВС), аппаратное обеспечение таких систем – современные семейства ПЛИС Altera и Xilinx, а также использовавшиеся при проведении работ платы комплектов разработчика Altera Arria II GX и Terasic DE5. Также рассмотрена общая структура гибридной системы, особенности построения аппаратных конфигураций в ПЛИС для различных классов задач и используемого системного программного обеспечения. Определены свойства алгоритмов, которые целесообразно реализовывать на подобных системах, приводятся примеры оценки такой целесообразности.

Значительная часть работы посвящена созданию собственно схемных конфигураций для решения задач на РВС. Приведены примеры решения задач на языке описания аппаратуры (Verilog). Проведен небольшой анализ средств высокоуровневого проектирования для создания схемных решений на языках высокого уровня (прежде всего – C и OpenCL). С использованием одного из рассмотренных высокоуровневых средств проектирования – Impulse C - разработан прототип системы теплового расчета для системы междисциплинарного инженерного анализа, а с использованием других средств – LegUp, а также Altera OpenCL SDK проведен анализ параметров аппаратной реализации ряда алгоритмов дискретных линейных преобразований (ДЛП), причем для некоторых их них аппаратная реализация рассмотрена впервые.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Большая Советская Энциклопедия М.: "Советская энциклопедия", 1969-1978.
2. Visions for Application Development on Hybrid Computing Systems / R.D. Chamberlain и др. / Parallel Computing Vol. 34 Issue 4-5, May, 2008, pp 201-216
3. Amar Shan Heterogeneous Processing: a Strategy for Augmenting Moore's Law [Электронный ресурс] .- 2006.- Режим доступа: http://www.linuxjournal.com/article/8368
4. Каляев, И. А. Реконфигурируемые вычислительные системы [Электронный ресурс] / И. А. Каляев, И. И. Левин, Е. А. Семерников. – 2009. – Режим доступа : http://parallel.ru/sites/default/files/papers/kaljaev4.pdf
5. Каляев И.А. Реконфигурируемые мультиконвейерные вычислительные структуры / И. А. Каляев, И. И. Левин, Е. А. Семерников, В.И. Шмойлов // Монография .- ЮНЦ РАН, Ростов-на-Дону, 2008.-393с.
6. Каляев И.А. Реконфигурируемые вычислительные системы на основе ПЛИС семейства VIRTEX-6 / И. А. Каляев , И. И. Левин , Е. А. Семерников , А. И. Дордопуло // Вестник УГАТУ.- Уфа, 2011.- т.15, № 5(45)- С.148-154.
7. A Quantitative Analysis of the Speedup Factors of FPGAs over Processors [Электронный ресурс] / Zhi Guo [и др.]. – 2004. – Режим доступа: http://www.cs.ucr.edu/~najjar/papers/2004/fpga2004.pdf
8. The Performance Prediction Model – A Methodology for Estimating the Performance of an FPGA Implementation of an Algorithm [Электронный ресурс] / Tobias Jeger [и др.]. – 2000. – Режим доступа : http://e-collection.library.ethz.ch/eserv/eth:27115/eth-27115-01.pdf
9. Steffen, C. P. Parametrization of Algorithms and FPGA Accelerators To Predict Performance [Электронный ресурс] / Craig P. Steffen. – 2007. – Режим доступа : http://rssi.ncsa.illinois.edu/2007/proceedings/papers/rssi07_18_paper.pdf
10. Smith, M. C. Parallel application performance on shared high performance reconfigurable computing resources [Электронный ресурс] / Melissa C. Smith, Gregory D. Peterson. – 2005. – Режим доступа : http://www.sciencedirect.com/ science / article / pii / S0166531604001361
11. RAT: a methodology for predicting performance in application design migration to FPGAs [Электронный ресурс] / B. Holland [и др.] // NSF Center for High-Performance Reconfigurable Computing (CHREC). – 2007. – Режим доступа : http://www.chrec.org/pubs/HPRCTA2007_F3.pdf
12. Holland, B. RAT: RC Amenability Test for Rapid Performance Prediction [Электронный ресурс] / Brian Holland, Karthik Nagarajan, Alan D. George. – 2009. – Режим доступа : http://www.hcs.ufl.edu/pubs/TRETS08_F3.pdf
13. Holland, B. Lessons learned with performance prediction and design patterns on molecular dynamics / B. Holland, K. Nagarajan, S. Merchant, H. Lam, and A. George // Proc. Military and Aerospace FPGA and Applications (MAFA) Meeting, Palm Beach, FL. – 2007.

14. Сверхбольшие интегральные схемы и современная обработка сигналов: Пер. с англ./Под ред. С.Гуна, Х.Уайтхауза, Т.Кайлата.- М.: Радио и связь, 1989.- 472с.:ил.

15. Духнич Е.И. Синтез класса алгоритмов и вычислительных структур для реализации дискретных линейных преобразований. Диссерт. на соискание уч. степени д.т.н., Таганрог, 1985.- 306с.

16. Духнич Е.И. Класс аппаратурно-реализованных алгоритмов для построения проблемно-ориентированных процессоров с новой архитектурой //Computers-89, Bratislava, Chechoslovakia, 1989.- p.42-47.

17. Духнич Е.И. Класс алгоритмов для реализации процедур линейной алгебры с помощью комплекта СБИС // Многопроцессорные вычислительные структуры. - Таганрог , 1991. - вып.13(XXII). - с.25-28

18. Духнич Е.И., Егунов В.А. Алгоритмы многомерных отражений, ориентированные на систолическую реализацию // Проектирование ЭВМ: Межвузовский сборник научных трудов. - Рязань : РГРА, 1994. - с. 57-63.

19. Егунов В.А. Разработка и исследование аппаратурно-ориентированных алгоритмов быстрого преобразования Хаусхолдера. Диссерт. на соискание уч. степени к.т.н., Волгоград, 1996. - 199с.

20. Андреев А.Е. Аппаратурно-ориентированные алгоритмы типовых унитарных преобразований линейной алгебры. Диссерт. на соискание уч. степени к.т.н., Волгоград, 1998. – 203 с.

21. Андреев, А.Е. Прогнозирование производительности при реализации алгоритмов на гибридных архитектурах с сопроцессорами [Электронный ресурс] / Андреев А.Е., Силкин И.М., Шафран Ю.В. // Современные проблемы науки и образования : электрон. журнал. - 2012. - № 3. - С. Режим доступа : http://www.science-education.ru/103-6389.

22. Петров А.А., Масловский Е.К. Англо-русский словарь по робототехнике / М: Русский язык. – 1989, 494с.

23. Misra, M. Design of Systolic arrays for QR Decomposition [Electronic Resource] / Manoj Misra, Rajat Moona. - 1994. - Mode of access: http://www.cse.iitk.ac.in/users/moona/papers/iccse94.pdf

24. Intel® Data Protection Technology with AES-NI and Secure Key [Электронный ресурс] – Mode of access: http://www.intel.com/content/www/us/en/architecture-and-technology/advanced-encryption-standard--aes-/data-protection-aes-general-technology.html

25. Yu, S. Improved collision attacks on MD4 and MD5/ Yu S., Naito Y., Kuhiniro N. // IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences. – Oxford, 2007.

26. Altera FPGAs [Electronic Resource] – Mode of access: http://www.altera.com/devices/fpga/

27. All Programmable FPGAs [Electronic Resource] – Mode of access: http://www.xilinx.com/products/silicon-devices/fpga/index.htm

28. Arria II GX FPGA Development Kit [Electronic Resource] – Mode of access: http://www.altera.com/products/devkits/altera/kit-aiigx-pcie.html

29. Terasic – All FPGA Main Boards – Stratix V - DE5-Net FPGA Development Kit [Electronic Resource] – Mode of access: http://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=158&No=526

30. Qsys - Altera's System Integration Tool [Electronic Resource] – Mode of access: http://www.altera.com/products/software/quartus-ii/subscription-edition/qsys/qts-qsys.html

31. Avalon Interface Specifications [Electronic Resource] // Altera Corporation. - 2011. - Mode of access : http://www.altera.com/literature/manual/ mnl_avalon_spec.pdf

32. Pellerin, D. Practical FPGA Programming in C / Pellerin D., Thibault S. – Prentice Hal. – 2005.

33. Embedded processor [Electronic Resource] – Mode of access: http://www.altera.com/devices/processor/nios2/ni2-index.html

34. Андреев, А.Е. Высокоуровневые средства разработки для FPGA [Электронный ресурс] / Андреев А.Е., Силкин И.М., Шафран Ю.В. // Современные научные исследования и инновации : электрон. журнал. - 2012. - Июнь. - С. Режим доступа : http://web.snauka.ru/issues/2012/06/14365.

35. Сравнительный анализ высокоуровневых средств разработки для ПЛИС [Электронный ресурс] / Андреев А.Е., Малолетков В.А., Оболонин М.А., Фролов Г.О., Чернояров Д.А. // Современные проблемы науки и образования : электрон. науч. журнал / РАЕ. - 2013. - № 2. - С. Режим доступа : http://www.science-education.ru/108-8748.

36. Rotem, N. C-to-Verilog FAQ [Электронный ресурс] / Nadav Rotem. - 2009. - Режим доступа : http://www.c-to-verilog.com/faq.html

37. Языки высокого уровня Impulse C, Mitrion-C и Handel-C [Электронный ресурс]. – URL :http://fpga.parallel.ru/lang.html

38. LegUp: High-Level Synthesis for FPGA-Based Processor/Accelerator Systems [Электронный ресурс] / A. Canis [и др.] // University of Toronto, Altera Toronto. – 2011. – URL: http://legup.eecg.utoronto.ca/fpga60-legup.pdf

39. Руководство программиста Автокод HDL [Электронный ресурс]/ С. С. Андреев [и др.]. – 2012. – URL : http://www.kiam.ru/MVS/research/avtokod/index.html

40. Технологии разработки прикладного ПО для реконфигурируемых вычислительных структур [Электронный ресурс] / С. С. Андреев [и др.]. –2010. – URL: http://sdat.ispras.ru/archive/2010/20100318_Latsis.pdf.

41. Mitrion Users' Guide [Electronic Resource] // Mitrionics AB. - Mode of access : http://forum.mitrionics.com/uploads/Mitrion_Users_Guide.pdf

42. Achieve Power-Efficient Acceleration with OpenCL on Altera FPGAs [Electronic Resource] – Mode of access: http://www.altera.com/ products/software/opencl/opencl-index.html

43. Holland B., Vacas M., Aggarwal V., De Ville R., Troxel I., and George A. D., "Survey of C-based Application Mapping Tools for Reconfigurable Computing" [Электронный ресурс]. – URL: https://wiki.ittc.ku.edu/eecs700/images/b/b8/Holland.pdf.

44. El-Araby E., Taher M., Abouellail M., El-Ghazawi T., and Newby G. B., "Comparative analysis of High Level Programming for Reconfigurable Computers: Methodology and Empirical Study" [Электронный ресурс]. – URL: http://www.ann.ece.ufl.edu/courses/eel6935_10spr / student_talks /

Comparative_analysis_of_High_Level.pptx.

45. El-Araby E., Merchant S. and El-Ghazawi T. “A Framework for Evaluating High-Level Design Methodologies for High-Performance Reconfigurable Computers” [Электронный ресурс]. – URL : http://www.chrec.org/pubs/TPDS10_G4.pdf.

46. Hara Y. and others “CHStone: A Benchmark Program Suite for Practical C-Based High-Level Synthesis” [Электронный ресурс]. – URL: http://ir.nul.nagoya-u.ac.jp/jspui/bitstream/2237/12085/1/iscas2008-300_1027.pdf.

47. Духнич Е.И. Об одном подходе к выполнению цифровых линейных преобразований // Кибернетика. - 1985.- N5. - с.95-98.

48. Духнич Е.И. Два обобщенных алгоритма дискретных линейных преобразований // Многопроцессорные вычислительные структуры. - Таганрог, 1984. - Вып.6. - С.9-11.

49. Volder J.E., The CORDIC Trigonometric Computing Technique.// IRE Trans. On Electronic Computers. - 1959. - Vol. EC-8 (3).- pp. 330-334.

50. Gotze J. Parallel methods for iterative matrix computations //In Proc. IEEE Int. Symp. On Circuits and Systems, 1991.-pp.233-236.

51. Духнич Е.И., Лукашева Г.Н., Серов А.А. Модифицированные алгоритмы дискретных линейных преобразований вращения//Многопроцессорные вычислительные структуры.-Таганрог, 1990.-Вып.12 (XXI). - с37-38.

52. Van Dijk H.W., Hekstra G.J., Deprettere E.F. Scalable parallel processor array for Jacobi-type matrix computations. // Integration, The VLSI Journal. Vol.20. Issue 1.- p.41-61. -1995.

53. Paul S., Gotze J, Sauer M. Error Analysis of CORDIC-Based Jacobi Algorithms. // IEEE Transactions on Computers. - 1995. - Vol. 44(7). - pp. 947-951.

54. Духнич Е.И., Деревенсков С.О. ДЛП-алгоритмы многомерных вращений для СБИС-реализации // Многопроцессорные вычислительные структуры: Междуведомственный тематический научный сборник сборник.-Таганрог: ТРТИ, 1995. Вып. 15-16. с25-27.

55. Hsiao S.-F., Delosme J.-M. Parallel Processing of Complex Data Using Quaternion and Pseudo-Quaternion CORDIC Algorithms. // In Proceedings of the ASAP’94 Conf., University of California.-1994.-pp.125-130.

56. Духнич Е.И., Каляев А.В. Алгоритмы для аппаратурной реализации преобразования компонент тензоров //Автоматика и вычислительная техника.-1977.-N2.-с.79-82.

57. Духнич Е.И., Мурашов С.В. Два варианта алгоритмов для быстрой аппаратурной реализации линейного преобразования отражения //Проектирование ЭВМ: Межвуз. Сб. Научн. трудов.-Рязань, 1992.-с47-51.

58. Hsiao S.-F., Delosme J.-M. Householder CORDIC Algorithms. // IEEE Transactions on Computers. - 1995. - Vol. 44(8). - pp. 990-1001.

59. Cavallaro J.R., Elster A.C., Complex Matrix Factorizations with CORDIC Arithmetic.//Technical Report 89-1071, Department of Computer Science, Cornell University.- 1989.-22p.

60. Cavallaro J.R., Elster A.C., A CORDIC Processor Array for the SVD of a Complex Matrix.//R. Vaccaro, Ed., SVD and Signal Processing II.-Amsterdam: Elsevier Science.-1991.-pp. 227-239.

61. Cavallaro J.R., Hemkumar N.-D. Efficient complex matrix transformations with CORDIC // In Proceedings of the 11th IEEE Symposium on Computer Arithmetic, Windsor, Ontario.- 1993.- p.122-129.

62. Freeman S., O'Donnell M., A Complex Arithmetic Digital Signal Processor Using CORDIC Rotators. // In Proceedings of the APPLICATION IMPLEMENTATIONS AND MAPPINGS Conf. - 1995. - Volume 5, pp. 319-326.

63. Hsiao S.-F., Delosme J.-M. Parallel Singular Value Decomposition of Complex Matrices Using Multidimensional CORDIC Algorithms.// IEEE Trans. On Signal Processing.-1996. - Vol. (3). - pp. 256-272.

64. Духнич, Е.И. Аппаратная реализация унитарных преобразований с помощью алгоритмов ДЛП / Духнич Е.И., Андреев А.Е. // Концептуальное проектирование в образовании, технике и технологии: Сб. науч. тр. / ВолгГТУ. - Волгоград,1999.-С.68-70.

65. Андреев, А.Е. Аппаратурная реализация алгоритмов многомерных ДЛП на ПЛИС / Андреев А.Е., Андреева М.И., Земцов А.Н. // Информационные технологии в образовании, технике и медицине: Сб. науч. тр. Междунар. науч.-техн. конф., 24-26 сент. 2002 / ВолгГТУ и др. - Волгоград, 2002. - Часть I. - С. 41-44.

66. Андреев, А.Е. Алгоритмы комплексных ДЛП и их аппаратурная реализация на ПЛИС / Андреев А.Е., Андреева М.И., Стрельников О.И. // Информационные технологии в образовании, технике и медицине: Сб. науч. тр. Междунар. науч.-техн. конф., 24-26 сент. 2002 / ВолгГТУ и др. - Волгоград, 2002. - Часть I. - С. 45-48.

67. Духнич, Е. Effective Processor Architecture for Matrix Decomposition [Электронный ресурс] / Духнич Е., Salamah M., Андреев А.Е. // Arabian Journal for Science and Engineering. - 2013. - September. - С. URL : http://download.springer.com / static/pdf/330/art%253A10.1007%252Fs13369-013-0759-.pdf?auth66 = 1385641261_ 46245b333719f38bfada8e612b558e0c&ext=.pdf.

68. Ray Andraka, A Survey of CORDIC Algorithms for FPGAs // FPGA '98. Proceedings of the 1998 ACM/SIGDA sixth international symposium on Field programmable gate arrays, Feb. 22-24, 1998, Monterey, CA. pp 191-200

69. Духнич, Е.И. Алгоритмы для аппаратурной реализации комплексного вращения / Духнич Е.И., Андреев А.Е. // I межвузовская научно-практическая конференция студентов и молодых учёных г. Волжского (г. Волжский, 24-28 апр. 1995 г.) : тез. докл. / ВГИ (филиал) ВолГУ [и др.]. - Волжский, 1995. - С. 107-109.

70. Андреев, А.Е. Дискретное линейное преобразование комплексного вращения/Андреев А.Е. // II Межвуз. науч.-практ. конф. студентов и молодых ученых Волгоградской области, 27 нояб. - 1 дек. 1995 г.: Сб. науч. ст. / Волгогр. ассоц. проф. орг. и др. - Волгоград, 1997. - Вып.5. - С. 84-89.

71. Духнич, Е.И. Сходимость процесса ДЛП- отражения в комплексном случае / Духнич Е.И., Андреев А.Е. // II межвузовская научно-практическая конференция студентов и молодых учёных г. Волжского : тез. докл. / ВПИ (филиал) ВолгГТУ [и др.]: Тез. докл. - Волжский, 1997. - С. 98-99.

72. E. Doukhnitch "Highly parallel multidimensional CORDIC-like algorithms", Artificial Intelligence, No3, pp.284-293, Ukraine, 2001, ISSN 1561-5359.

73. E. Doukhnitch, E. Ozen, “Hardware-oriented Algorithm for Quaternion Valued Matrix Decomposition”, IEEE Transactions on Circuits and Systems--II: Express Briefs, Vol. 58, no. 4, pp.225-229, April 2011, (SCIE)

74. E. Doukhnitch, V. Podbelskiy, “Higher Parallel Hardware-oriented Algorithm for Jacobi SVD of Quaternion Valued Matrix”, Parallel and Cloud Computing Research, vol. 1, no.3, pp.41-49, October 2013

75. E. Doukhnitch, “Hardware-oriented Algorithms for Fast Householder Transform”, in Proc. of 1 Intern. Conference on Signal Processing and Applications “DSPA-98”, v.2e, pp.129-132, Moscow, July, 1998

76. E. Doukhnitch, “Synthesis for the Discrete Quaternion Transform Algorithms Class”, in Proc. of Intern. Conf. On Intelligent Multiprocessor Systems”, pp.44-48, Taganrog, Russia, 1999

77. E. Doukhnitch, O. Strelnikov, A Andreev, “Application of Kronecker Matrix Product for the Synthesis of Hardware-oriented DSP Algorithms”, in Proc. of Intern. Conf. on Signal Proc. “DSPA-99”, pp. 78-83, Moscow, Sept. 1999

78. E. Doukhnitch, O. Strelnikov, A Andreev, “Use of Pseudo-unitary Transform for Eigenvalue Decomposition", in Proc. of Intern. Conf. "Information Technologies in Education, Machinery and Medicine", pp.53-57, Volgograd, Sept. 2000

79. E. Doukhnitch "Multidimensional CORDIC-like algorithms for DSP", in Proc. of the Sixteenth Intern. Symp. on Computer and Information Sciences, pp.368-375, Nov. 2001, Antalya, Turkey

80. E. Doukhnitch, “Octonion CORDIC Algorithms for DSP,” in Proc. of the 6th Symp. on Signal Processing, DSPCS’2002, pp.159-163, Sydney, Australia, Jan. 2002.

81. E. Doukhnitch, E. Osen, "Hardware-oriented Algorithms for Fast Matrix Triangularization", in Proc. of Inter. Conf. on Supercomputers and Multiprocessor Computer Systems, pp.212-215, Taganrog, Russia, 2002.

82. E. Doukhnitch, “Hardware-Oriented Algorithm for Householder Bidiagonalization with Minimal Latency Time”, in Proc. of Intern. Conf. on Multiprocessor Computing and Control Systems,” MCCS-2009, pp. 120 -123, Taganrog, Russia, Sept. 2009, ISBN 978-5-8327-0341-1

83. E.I. Doukhnitch, O.I. Strelnikov, “Convergence Check Procedure for Octonion CORDIC Algorithm,” Интеллектуальные и многопроцессорные системы-2003 // Материалы Международной научной конференции. Т.2. Таганрог: Изд-во ТРТУ, 2003, с.110-112

84. Егунов, В.А. Моделирование алгоритмов дискретных линейных преобразований / Егунов В.А., Андреев А.Е., Стрельников О.И. // Изв. ВолгГТУ. Серия "Актуальные проблемы управления, вычислительной техники и информатики в технических системах": межвуз. сб. науч. ст. / ВолгГТУ. - Волгоград, 2007. - Вып.1, №1. - С. 51-52.

85. Егунов, В.А. Определение условия оптимальности процесса дискретного линейного преобразования отражения / Егунов В.А., Андреев А.Е., Стрельников О.И. // Изв. ВолгГТУ. Серия "Актуальные проблемы управления, вычислительной техники и

информатики в технических системах": межвуз. сб. науч. ст. / ВолгГТУ. - Волгоград, 2007. - Вып.1, №1. - С. 53-54.

86. Configurable cordic core in Verilog [Электронный ресурс] / Режим доступа : http://opencores.org/project,verilog_cordic_core

87. Шафран, Ю.В. Восстановление строк по MD5-хэшу полным перебором на процессорах общего назначения и графических процессорах / Шафран Ю.В., Андреев А.Е. // Тезисы докладов юбилейного смотра-конкурса научных, конструкторских и технологических работ студентов ВолгГТУ, Волгоград, 11-14 мая 2010 г. / ВолгГТУ, Совет СНТО. - Волгоград, 2010. - С. 196-197.

88. Оболонин, М.А. Система распределения нагрузки в неоднородных вычислительных комплексах / Оболонин М.А., Андреев А.Е. // Тезисы докладов смотра-конкурса научных, конструкторских и технологических работ студентов Волгоградского гос. технического университета, 14–17 мая 2013 г. / ВолгГТУ, Совет СНТО. - Волгоград, 2013. - С. 151.

89. Solving Equations of Nonstationary Heat Conduction in Multibody System Dynamics / Андреев А.Е., Шаповалов О.В., Резников М.В., Громов Е.Г. // Multibody System Dynamics : the First Joint International Conference (Lappeenranta, Finland, May 25-27, 2010 : book of abstracts / Lappeenranta University of Technology.- [Lappeenranta], 2010.- 6 p.- Англ.

90. Решение задачи нестационарной теплопроводности в многотельной модели автомобиля / Шаповалов О.В., Резников М.В., Громов Е.Г., Андреев А.Е. // Сборник тезисов докладов VII всерос. межвуз. конференции молодых учёных, посвящ. 110-годовщине со дня создания СПбГУ ИТМО. Вып. 5 / Санкт-Петербургский гос. ун-т информ. технологий, механики и оптики. - СПб., 2010. - С. 99-100.

91. Решение задачи теплопроводности в многотельной модели автомобиля / Шаповалов О.В., Шамов Е.А., Жариков Д.Н., Андреев А.Е. // Высокопроизводительные параллельные вычисления на кластерных системах (HPC-2010) : матер. X междунар. конф. (г. Пермь, 1-3 нояб. 2010 г.). В 2 т. Т. 2 / Суперкомпьютерный консорциум ун-тов России, ГОУ ВПО "Пермский гос. техн. ун-т" [и др.]. - Пермь, 2010. - С. 309-315.

92. Система для расчёта тепловых технологических процессов машиностроения / Андреев А.Е., Громов Е.Г., Резников М.В., Шаповалов О.В. // Известия ВолгГТУ. Серия "Актуальные проблемы управления, вычислительной техники и информатики в технических системах". Вып. 7 : межвуз. сб. науч. ст. / ВолгГТУ. - Волгоград, 2009. - № 12. - С. 124-126.

93. Шаповалов, О.В. Методы решения задачи теплопроводности в системе многотельной динамики "ФРУНД" / Шаповалов О.В., Андреев А.Е. // Тезисы докладов смотра-конкурса научных, конструкторских и технологических работ студентов ВолгГТУ, Волгоград, 10-13 мая 2011 г. / ВолгГТУ, Совет СНТО. - Волгоград, 2011. - С. 185-186.

94. Шаповалов О.В. Разработка методов расчета задачи теплопроводности в контексте мультифизического подхода // Вестник УГАТУ, Уфа. – 2012. – № 3. – С. 173-179.

ПРИЛОЖЕНИЕ А

Исходный код на С для реализации алгоритма CORDIC на LegUp

```
#include <stdio.h>
#include <math.h>

const int CNT=32;
int shr(int val, int n)   {   return val >> n; }
int sign(int val) { return val >= 0 ? 1 : -1;  }

int CORDIC(int* px, int* py, int n) {
      int x, y, xi, yi, mul;
      int ksi, i, xs, ys;
      xi = *px;
      yi = *py;
      ksi = 1;

loop: for (i=0; i<n; i++)     {
            x = xi;
            y = yi;
            ksi = sign(y);
            xi  = x + shr(y*ksi, i);
            yi  = y - shr(x*ksi, i);
      }
      *px = x;
      *py = y;
      return x;
}


int main()  {
      int x, y;
      x = 3765479;
      y = 4235609;
      CORDIC(&x, &y, CNT);
      printf("%d %d \n", x, y);

      return 0;
}
```

ПРИЛОЖЕНИЕ Б

Исходный код ядра OpenCL для реализации алгоритма CORDIC на Altera OpenCL SDK

```
__kernel
void CORDIC_try( // Input and output parameters
                 __global const int *x,
                 __global const int *y,
                 __global int *restrict xo,
                 __global int *restrict yo)
{
    // Local storage
    __local int X, X2;
    __local int Y, Y2;
    __local int sign;

    X2 = *x;
    Y2 = *y;

    #pragma unroll 32
    for (int i = 0; i < 32; i++)
    {
        X = X2;
        Y = Y2;
            sign = Y >= 0 ? 0 : 1;

        X2 = X + (sign ? -Y : Y) >> i;
        Y2 = Y - (sign ? -X : X) >> i;
    }

    *xo = X2;
    *yo = Y2;
}
```

ПРИЛОЖЕНИЕ В

Исходный код на С для реализации алгоритма Hausholder CORDIC в матричной форме для размерности 4 на LegUp

```
#include <stdio.h>
#include <math.h>

const int CNT=4;
int sign(int val) { return val >= 0 ? 1 : -1; }
int shl(int val, int n) { return val << n; }
int shr(int val, int n) {     return val >> n; }

int haushold_CORDIC(int* v, int n) {
      int y[4];
      int i, j, k, i2;
      int ksi[4];

      for(i=0; i<4; i++)      {
            y[i] = v[i];
            ksi[i] = 1;
      }

loop: for (i=0; i<n; i++) {
            for (j=0; j<4; j++)
                  y[j] = v[j];

            for (j=0; j<4; j++)  {
                  if (y[j] >= 0)
                        ksi[j] =  1;
                  else
                        ksi[j] = -1;
            }

           i2 = shl(i,1);
           v[0] = y[0] - shl(shr(y[0], i2),1) - shr(y[0], i2) +
                        shl(shr(ksi[1]*y[1],i),1) +
                        shl(shr(ksi[2]*y[2],i),1) +
                        shl(shr(ksi[3]*y[3],i),1);
            v[1] = y[1] + shr(y[1], i2) - shl(shr(ksi[1]*y[0],i),1) -
```

```
                    shl(shr(ksi[1]*ksi[2]*y[2],i2),1) –
                    shl(shr(ksi[1]*ksi[3]*y[3],i2),1);
            v[2] = y[2] + shr(y[2], i2) - shl(shr(ksi[2]*y[0],i),1) -
                    shl(shr(ksi[1]*ksi[2]*y[1],i2),1) –
                    shl(shr(ksi[2]*ksi[3]*y[3],i2),1);
            v[3] = y[3] + shr(y[3], i2) - shl(shr(ksi[3]*y[0],i),1) -
                    shl(shr(ksi[1]*ksi[3]*y[1],i2),1) –
                    shl(shr(ksi[2]*ksi[3]*y[2],i2),1);
      }
      return v[0];
}

int main()
{
      int x[4];
      int i;

      x[0] = 0x01000000;
      x[1] = 0x016a09e5;
      x[2] = 0x01000000;
      x[3] = 0x016a09e5;
      printf("%d \n", haushold_CORDIC(x, 32));
      for (i=1; i<4; i++)
            printf("%d \n", x[i]);
      return 0;
}
```

ПРИЛОЖЕНИЕ Г

Исходный код ядра OpenCL для реализации алгоритма Hausholder CORDIC в матричной форме для размерности 3 на Altera OpenCL SDK

```
__kernel
void Haus3( // Input and output parameters
                 __global const int *xi,
                 __global const int *yi,
                 __global const int *zi,
                 __global int *restrict xo,
                 __global int *restrict yo,
                 __global int *restrict zo)
{
  __local int y[3];
  __local int v[3];
  __local int Y[3][3];
  __local int i, j, k;
  __local int i2;
  __local bool ksi23;
  __local int  ksi[3];
  __local bool ksil[3];

  i=-1;

  v[0] = *xi;
  v[1] = *yi;
  v[2] = *zi;

  #pragma unroll 32
  for (j=0; j<32; j++)   {
     y[0] = v[0];
     y[1] = v[1];
     y[2] = v[2];

     // Определяем знаки в виде bool
     ksil[0] = y[0] >= 0 ? 0 : 1;
     ksil[1] = y[1] >= 0 ? 0 : 1;
     ksil[2] = y[2] >= 0 ? 0 : 1;
```

```
    //i = j % 2 == 0 ? i + 1 : i;
    i = j;
    i2 = i << 1;

    // сочетания знаков ++ и -- = +, остальное -
    ksi[1] = (ksil[0]&&(!ksil[1])) || ((!ksil[0])&&ksil[1]);
    ksi[2] = (ksil[0]&&(!ksil[2])) || ((!ksil[0])&&ksil[2]);
    ksi23 =  (ksi[1]&&(!ksi[2])) || ((!ksi[1])&&ksi[2]);

    // подготавливаем операнды для матрицы
    // с нужными знаками сразу
    Y[0][1] = ksi[1] ? -y[1] : y[1];
    Y[0][2] = ksi[2] ? -y[2] : y[2];

    Y[1][2] = ksi23  ? -y[2] : y[2];
    Y[1][0] = ksi[1] ? -y[0] : y[0];

    Y[2][1] = ksi23  ? -y[1] : y[1];
    Y[2][0] = ksi[2] ? -y[0] : y[0];

    if(i < 16) {
        // пока 2^-2i еще актуально
        v[0] = -y[0] + ((y[0] >> i2) << 1) - ((Y[0][1] >> i) << 1)
           - ((Y[0][2] >> i) << 1);
        //v[0] =  y[0] - ((y[0] >> i2) << 1) - ((Y[0][1] >> i) << 1)
           - ((Y[0][2] >> i) << 1);
        v[1] =  y[1] - ((Y[1][0] >> i)<< 1) - ((Y[1][2] >> i2)<< 1);
        v[2] =  y[2] - ((Y[2][0] >> i)<< 1) - ((Y[2][1] >> i2)<< 1);
    }
    else   {
        // 2^-2i уходят за разрядную сетку
         v[0] = -y[0] - ((Y[0][1] >> i)<< 1) - ((Y[0][2]>> i) << 1);
         v[1] =  y[1] - ((Y[1][0] >> i) << 1);
         v[2] =  y[2] - ((Y[2][0] >> i) << 1);
    }
  } // for

  *xo = v[0];
  *yo = v[1];
  *zo = v[2];
}
```

ПРИЛОЖЕНИЕ Д

Исходный код ядра OpenCL для реализации алгоритма Hausholder CORDIC в векторной форме для размерности 3 на Altera OpenCL SDK

```
void Haus3v( // Input and output parameters
                  __global const int *xi,
                  __global const int *yi,
                  __global const int *zi,
                  __global int *restrict xo,
                  __global int *restrict yo,
                  __global int *restrict zo)
{
  __local int y[3];
  __local int v[3];
  __local int Y[3][3];
  __local int i, j, k;
  __local int i2;
  __local int N;
  __local bool ksi12;
  __local bool ksi13;
  __local int  ksi[3];
  __local bool ksil[3];

  v[0] = *xi;
  v[1] = *yi;
  v[2] = *zi;

  #pragma unroll 32
  for (i=0; i<32; i++)
  {
      y[0] = v[0];
      y[1] = v[1];
      y[2] = v[2];

      // Определяем знаки в виде bool
      ksil[0] = y[0] >= 0 ? 0 : 1;
      ksil[1] = y[1] >= 0 ? 0 : 1;
      ksil[2] = y[2] >= 0 ? 0 : 1;

      i2 = i << 1;
```

```
      // сочетания знаков ++ и -- = +, остальное -
      ksi12 = (ksil[0]&&(!ksil[1])) || ((!ksil[0])&&ksil[1]);
      ksi13 = (ksil[0]&&(!ksil[2])) || ((!ksil[0])&&ksil[2]);

        // подготавливаем операнды для матрицы
        // с нужными знаками сразу
        Y[0][1] = ksi12 ? -y[1] : y[1];
        Y[0][2] = ksi13 ? -y[2] : y[2];

      N = (y[0] + (Y[0][1] >> i) + (Y[0][2] >> i)) << 1;
        Y[1][1] = ksi12 ? -N : N;
        Y[1][2] = ksi13 ? -N : N;

     if(i < 16) {
            // пока 2^-2i еще актуально
            v[0] = y[0] + ((y[0] >> i2) << 1) - N;
            v[1] = y[1] + ((y[1] >> i2) << 1) - (Y[1][1] >> i);
            v[2] = y[2] + ((y[2] >> i2) << 1) - (Y[1][2] >> i);
     }
     else     {
            // 2^-2i уходят за разрядную сетку
            v[0] = y[0] - N;
            v[1] = y[1] - (Y[1][1] >> i);
            v[2] = y[2] - (Y[1][2] >> i);
     }
  } // for

  *xo = v[0];
  *yo = v[1];
  *zo = v[2];
}
```

ПРИЛОЖЕНИЕ Е

Исходный код ядра OpenCL для реализации алгоритма ДЛП кронекеровского произведения вращений на Altera OpenCL SDK

```
__kernel
void KRONEKER_Prod( // Input and output parameters
                __global const int *y,  __global const int *x,
                __global const int *z,  __global const int *w,
                __global int *restrict xo,
                __global int *restrict yo,
                __global int *restrict zo,
                __global int *restrict wo,
            int numElements) {
    // Local storage
    __local int X1, X2, X3, X4;
    __local int Y[4][4];
    __local int y1, y2, y3, y4;
    __local bool s1, s2, s12;
    __local int i2;

    X1 = *x;    X2 = *y;
    X3 = 0;     X4 = 0;

    #pragma unroll 16
    for (int i = 0; i < 16; i++)
    {
      y1 = X1;          y2 = X2;
      y3 = X3;          y4 = X4;

      s1  = y2 >= 0 ? 0 : 1;
      s2  = y3 >= 0 ? 0 : 1;
      s12 = ((s1 && !s2) || (!s1 && s2));

      Y[0][1] = s1  ? -y2 : y2;
      Y[0][2] = s2  ? -y3 : y3;
      Y[0][3] = s12 ? -y4 : y4;

      Y[1][1] = s1  ? -y1 : y1;
      Y[1][2] = s2  ? -y4 : y4;
      Y[1][3] = s12 ? -y3 : y3;
```

```
  Y[2][1] = s1  ? -y4 : y4;
  Y[2][2] = s2  ? -y1 : y1;
  Y[2][3] = s12 ? -y2 : y2;

  Y[3][1] = s1  ? -y3 : y3;
  Y[3][2] = s2  ? -y2 : y2;
  Y[3][3] = s12 ? -y1 : y1;

  i2 = i << 1;

      X1 = y1 + (Y[0][1] >> i) + (Y[0][2] >> i) + (Y[0][3] >> i2);
      X2 = y2 - (Y[1][1] >> i) - (Y[1][2] >> i) + (Y[1][3] >> i2);
      X3 = y3 + (Y[2][1] >> i) - (Y[2][2] >> i) + (Y[2][3] >> i2);
      X4 = y4 + (Y[3][1] >> i) - (Y[3][2] >> i) + (Y[3][3] >> i2);
}

#pragma unroll 16
for (int i = 16; i < 32; i++)
{
  y1 = X1;
  y2 = X2;
  y3 = X3;
  y4 = X4;

  s1  = y2 >= 0 ? 0 : 1;
  s2  = y3 >= 0 ? 0 : 1;
  X1 = y1 + ((s1  ? -y2 : y2) >> i) + ((s2  ? -y3 : y3) >> i);
  X2 = y2 - ((s1  ? -y1 : y1) >> i) - ((s2  ? -y4 : y4) >> i);
  X3 = y3 + ((s1  ? -y4 : y4) >> i) - ((s2  ? -y1 : y1) >> i);
  X4 = y4 + ((s1  ? -y3 : y3) >> i) - ((s2  ? -y2 : y2) >> i);
}

*xo = X1 + X2;
*yo = X2;
*zo = X3;
*wo = X4 + X3;
}
```

ПРИЛОЖЕНИЕ Ж

Исходный код ядра OpenCL для реализации алгоритма комплексного ДЛП отражения на базе Hausholder CORDIC в векторной форме для размерности 3 на Altera OpenCL SDK

```
__kernel
void Haus3vc(__global const int* vir,  __global const int* vii,
           __global int* restrict vor, __global int* restrict voi)  {
__local int yr[3];     __local int yi[3];
__local int vr[3];     __local int vi[3];
__local int Yr[3][3];  __local int Yi[3][3];
__local int i, j, k;   __local int i2;
__local int Ar;        __local int Ai;
__local bool ksir[3];  __local bool ksim[3];
__local bool ksir0[3]; __local bool ksim0[3];
__local bool ksi[3];   __local bool ksi0[3];

vr[0] = vir[0];   vr[1] = vir[1];
vr[2] = vir[2];

vi[0] = vii[0];   vi[1] = vii[1];
vi[2] = vii[2];

i=-1;

#pragma unroll 40
for (j=0; j<40; j++)
{
      yr[0] = vr[0];          yr[1] = vr[1];
      yr[2] = vr[2];

      yi[0] = vi[0];          yi[1] = vi[1];
      yi[2] = vi[2];

      // Определяем знаки в виде bool
      ksir[0] = yr[0] >= 0 ? 0 : 1;
      ksir[1] = yr[1] >= 0 ? 0 : 1;
      ksir[2] = yr[2] >= 0 ? 0 : 1;
```

```
ksim[0] = yi[0] >= 0 ? 0 : 1;
ksim[1] = yi[1] >= 0 ? 0 : 1;
ksim[2] = yi[2] >= 0 ? 0 : 1;

if ((j % 2 == 0) || (j > 16)) i = i+1;
i2 = i << 1;

// сочетания знаков ++ и -- = +, остальное -
ksir0[1] = (ksir[0] && (!ksir[1])) || ((!ksir[0]) && ksir[1]);
ksir0[2] = (ksir[0] && (!ksir[2])) || ((!ksir[0]) && ksir[2]);
ksim0[1] = (ksim[0] && (!ksim[1])) || ((!ksim[0]) && ksim[1]);
ksim0[2] = (ksim[0] && (!ksim[2])) || ((!ksim[0]) && ksim[2]);

ksi[1] = (ksir0[1] && (!ksim0[1])) || ((!ksir0[1])&& ksim0[1]);
ksi[2] = (ksir0[2] && (!ksim0[2])) || ((!ksir0[2])&& ksim0[2]);

// подготавливаем операнды для уравнений с
// нужными знаками сразу
Yr[0][1] = ksi[1] ? (ksim0[1] ? -yi[1] : yi[1]) :
                  (ksir0[1] ? -yr[1] : yr[1]);
Yr[0][2] = ksi[2] ? (ksim0[2] ? -yi[2] : yi[2]) :
                   (ksir0[2] ? -yr[2] : yr[2]);

Yi[0][1] = ksi[1] ? (ksir0[1] ? -yr[1] : yr[1]) :
                  (ksim0[1] ? -yi[1] : yi[1]);
Yi[0][2] = ksi[2] ? (ksir0[2] ? -yr[2] : yr[2]) :
                  (ksim0[2] ? -yi[2] : yi[2]);

Ar = Yr[0][1] + Yr[0][2];
Ai = Yi[0][1] + Yi[0][2];

Yr[1][1] = ksi[1] ? (ksir0[1] ? -yi[0] : yi[0]) :
                   (ksir0[1] ? -yr[0] : yr[0]);
Yr[1][2] = ksi[1] ? (ksir0[1] ? -Ar : Ar):(ksir0[1]? -Ai : Ai);
Yi[1][1] = ksi[1] ? (ksim0[1] ? -yr[0]: yr[0]):
                   (ksim0[1] ? -yi[0] : yi[0]);
Yi[1][2] = ksi[1] ? (ksir0[1] ? -Ai : Ai):(ksir0[1]? -Ar : Ar);

Yr[2][1] = ksi[2] ? (ksir0[2] ? -yi[0] : yi[0]):
                   (ksir0[2] ? -yr[0] : yr[0]);
Yr[2][2] = ksi[2] ? (ksir0[2] ? -Ar : Ar):(ksir0[2]? -Ai : Ai);
```

```
        Yi[2][1] = ksi[2] ? (ksim0[2] ? -yr[0] : yr[0]) :
                           (ksim0[2] ? -yi[0] : yi[0]);
        Yi[2][2] = ksi[2] ? (ksir0[2] ? -Ai : Ai):(ksir0[2]? -Ar : Ar);

        if(i < 16) {
              // пока 2^-2i еще актуально
              vr[0] = -yr[0] + ((yr[0] >> i2) << 1) - ((Ar >> i) << 1);
              vi[0] = -yi[0] + ((yi[0] >> i2) << 1) - ((Ai >> i) << 1);

              vr[1] =  yr[1] + ((yr[1] >> i2) << 1)
                             - ((Yr[1][1] >> i) << 1)
                             - ((Yr[1][2] >> i2) << 1);
              vi[1] =  yi[1] + ((yi[1] >> i2) << 1)
                             - ((Yi[1][1] >> i) << 1)
                             - ((Yi[1][2] >> i2) << 1);

              vr[2] =  yr[2] + ((yr[2] >> i2) << 1) -
                        ((Yr[2][1] >> i) << 1)- ((Yr[2][2] >> i2) << 1);
              vi[2] =  yi[2] + ((yi[2] >> i2) << 1) -
                        ((Yi[2][1] >> i) << 1)- ((Yi[2][2] >> i2) << 1);
              }
              else        {
                    // 2^-2i уходят за разрядную сетку
                    vr[0] = -yr[0] - ((Ar >> i) << 1);
                    vi[0] = -yi[0] - ((Ai >> i) << 1);
                    vr[1] =  yr[1] - ((Yr[1][1] >> i) << 1);
                    vi[1] =  yi[1] - ((Yi[1][1] >> i) << 1);
                    vr[2] =  yr[2] - ((Yr[2][1] >> i) << 1);
                    vi[2] =  yi[2] - ((Yi[2][1] >> i) << 1);
              }
        } // for

        vor[0] = vr[0];   vor[1] = vr[1];
        vor[2] = vr[2];   voi[0] = vi[0];
        voi[1] = vi[1];   voi[2] = vi[2];
}
```

Научное издание

Андрей Евгеньевич **Андреев**
Виталий Алексеевич **Егунов**
Дмитрий Николаевич **Жариков**
Владимир Алексеевич **Малолетков**

РЕАЛИЗАЦИЯ ВЫЧИСЛИТЕЛЬНО-ИНТЕНСИВНЫХ АЛГОРИТМОВ НА ГИБРИДНЫХ СИСТЕМАХ С РЕКОНФИГУРИРУЕМЫМИ СОПРОЦЕССОРАМИ

Монография

Редактор *Л.П. Кузнецова*
Компьютерная верстка *авторов*

Темплан 2013 г. (научные издания). Поз. № 35н.
Подписано в печать 24.12.2013. Формат 60x84 1/16. Бумага офсетная.
Гарнитура Times. Печать офсетная. Усл. печ. л. 11,25. Уч.-изд. л. 11,25.
Тираж 100 экз. Заказ

Волгоградский государственный технический университет.
400005, г. Волгоград, просп. В.И. Ленина, 28, корп. 1.

Отпечатано в типографии ИУНЛ ВолгГТУ.
400005, г. Волгоград, просп. В.И. Ленина, 28, корп. 7.